

Methods S1: FLIM Playground Manual

(related to STAR Methods)

Table of contents

1	Quick Start	8
1.1	Try Data Analysis	8
1.2	Try Data Extraction locally	9
1.3	Installation	11
1.4	Introduction	11
1.5	Challenges	12
1.6	Method	13
1.6.1	Feature Classes	13
1.6.2	Design	14
1.6.3	Summary	16
1.6.4	Citation	16
I	Data Extraction	17
2	Overview	18
2.1	Stages	18
2.2	Input File Types	19
2.2.1	Decay	19
2.2.2	Mask	20
2.2.3	IRF	20
2.2.4	SPCImage t1	20
2.2.5	Fluorescence Lifetime Standard	21
2.2.6	Intensity (2D)	21
2.3	Limitations	21
3	Configuration	22
3.1	Extraction Profiles	23
3.2	Cross-Channel Configurations	24
3.2.1	Categorical Features	24
3.2.2	Decay Types	24
3.2.3	Identifiers	25
3.2.4	Decay Info	25
3.2.5	Calibration Method	26
3.2.6	Number of Channels	26

3.3	Per-Channel Configurations	27
3.3.1	Channel Name	27
3.3.2	Imaging Modality	27
3.3.3	Feature Extractor	27
3.3.4	Number of Components	28
3.3.5	Fixed Lifetimes	28
3.3.6	File Suffix	29
3.4	Derived Features	29
3.5	Save	29
4	Field of View Metadata	30
4.1	Metadata	31
4.1.1	Decay Type	31
4.1.2	Channel Names	31
4.1.3	Feature Extractors	31
4.1.4	Decay Info	31
4.1.5	File Suffix	32
4.1.6	Fluorescence Lifetime Standard	32
4.1.7	Folder Path	32
4.2	Metadata Extraction	32
4.2.1	FOV File Paths	32
4.2.2	Decay Info	33
4.2.3	Result Preview and Export	34
5	Numerical Feature Extraction	36
5.1	Input	36
5.2	Feature Extraction	37
5.3	Save Results	38
6	IRF Calibration	39
6.1	When this stage is needed	39
6.2	Input	39
6.3	Fit Calibration	40
6.3.1	Gather High SNR Decay Curves	41
6.3.2	Reconvolution Fitting	41
6.4	Fit-Free IRF Calibration	46
6.4.1	Shift IRF	46
6.5	Confirm Calibration	47
7	Lifetime Features (Fit)	48
7.1	Fitting	48
7.1.1	Preprocessing	49
7.1.2	Fraction of components	49

7.1.3	Mean lifetime	50
7.1.4	Intensity-weighted mean lifetime	50
7.1.5	Goodness of fit	50
7.2	Pixel pre-fit	51
8	Lifetime Features (Fit-Free)	52
8.1	Preprocessing	52
8.2	Calibration method	53
8.2.1	IRF	53
8.2.2	Fluorescence lifetime standard	53
8.3	Phasor features	53
8.3.1	Step 1: raw phasor coordinates	53
8.3.2	Step 2: calibrated phasor coordinates	54
8.3.3	Step 3: phasor-derived lifetime	55
9	Morphological Features	58
10	Textural Features	60
10.1	Granularity	60
10.2	Radial Distribution	60
10.3	Mass Displacement	60
10.4	Intensity Sum	61
11	Derived Features	62
11.1	Build Derived Features	62
11.1.1	Templates	63
11.1.2	Custom Formulas	63
11.2	Compute Derived Features	64
11.3	Derived Feature In Data Analysis	64
12	Categorical Feature Extraction	65
12.1	Merging Datasets	65
12.2	Assigning Categorical Features	66
II	Data Analysis	68
13	Overview	69
13.1	General Workflow	69
13.2	Methods	69
13.3	Input	70
13.3.1	Requirements	70
13.3.2	Column roles	70
13.3.3	Warning Messages	71

13.3.4	Error Messages	71
13.4	Shared Interactive Widgets	72
13.4.1	Numerical Selection Widgets	72
13.4.2	Filter Widgets	73
13.4.3	Visual Channels Widgets	75
13.4.4	Unique ID Hover	77
13.4.5	Plotting Configuration Widgets	77
13.4.6	Feature Labels	78
13.4.7	Export Workflow as Python Script	78
14	Configuration	79
1.	Upload Iris	79
2.	Review column roles	80
3.	Feature group management	81
	How group names are inferred	81
	Change groups in bulk	82
4.	Save as Iris	83
5.	Reopen from the Iris summary	83
6.	Manage saved profiles	84
III	Univariate Analysis	85
15	Feature Comparison	86
15.1	Shared Interface Components	87
15.2	Plot Options	87
15.3	Collapse by	87
15.3.1	SuperPlot	88
15.4	Separate by	89
15.5	Subcolor by	90
15.6	Reorder X-axis Groups	91
15.7	Comparison Widget	92
15.7.1	Effect Size Threshold	92
15.8	Effect Size	92
15.8.1	Glass's Delta	93
15.8.2	Absolute Cohen's d	93
15.9	Statistical Test	94
15.10	Comparison Annotation	94
16	Feature Histogram	95
16.1	Shared Interface Components	95
16.2	Histogram Mode	96
16.3	Separate by	97

16.4	Gaussian Mixture Model Mode	98
16.4.1	Fit Gaussian Mixture Models	98
16.4.2	Heterogeneity index	99
16.4.3	Classification	99
16.4.4	Example	101
IV	Bivariate Analysis	103
17	2D Feature Distribution	104
17.1	Shared Interface Components	104
17.2	Log-scale Axes	104
17.3	Separate by	104
17.4	Collapse by	106
17.5	Marginal Distribution	106
17.6	2D Gaussian Mixture Model	106
17.6.1	Export labeled data	108
17.7	Correlation coefficient	109
17.8	Regression line	109
18	Phasor Plot	111
18.1	Shared Interface Components	112
18.2	Select Phasor Coordinate	112
18.3	Separate by	113
18.4	Export the plot	114
V	Multivariate Analysis	115
19	Dimension Reduction	116
19.1	Shared Interface Components	117
19.2	Separate by	118
19.3	Hyperparameter Widget	119
19.3.1	UMAP	119
19.3.2	t-SNE	119
19.4	Export the analysis	119
20	Classification	120
20.1	Shared Interface Components	121
20.2	Method-specific Components	121
20.2.1	Classifier Hyperparameters	122
20.2.2	Class Imbalance Handling	126
20.2.3	Confusion Matrix	128
20.2.4	Performance Metrics	128

20.2.5 ROC Curve	129
20.2.6 Feature Importance	131
20.3 Export the analysis	132
21 Common Q & A	133

1 Quick Start

docs synced v1.13.1

Welcome to the FLIM Playground 🥳🎉! This is an interactive graphical user interface (GUI) that allows you to extract single-cell¹ features from fluorescence lifetime imaging microscopy (FLIM) raw data ([Data Extraction](#)) and analyze extracted features or your own datasets using a built-in repertoire of methods ([Data Analysis](#)).

To quickly try out different analysis methods, download this [sample dataset](#) and try the [online demo](#). If you prefer to use your own datasets, read [this chapter on data analysis configuration](#) to learn how to configure the system.

Several talks have been given about FLIM Playground:

- [FLIM Playground @ SPIE 2026: 13 mins](#)
- [FLIM Playground @ the Fluorescence Microscopy Summit by Photonics Media: 31 mins](#)

1.1 Try Data Analysis

1. **Load the sample.** In the online demo, open **Data Analysis** → **Univariate** → **Feature Comparison**. Keep **Use Dataset from Data Extraction** checked and upload `inhibitors.csv` from the sample link above. For your own table, clear the checkbox and follow the [column review walkthrough](#).
2. **Compare groups.** Select **tm** under **Lifetime fit_nadh**, set **Color by** to **treatment**, and **Separate by** to **cell_line**. Change a [filter](#) or select another measurement to explore the [Feature Comparison](#) plot.

¹in general, any region of interest works, including the whole field of view

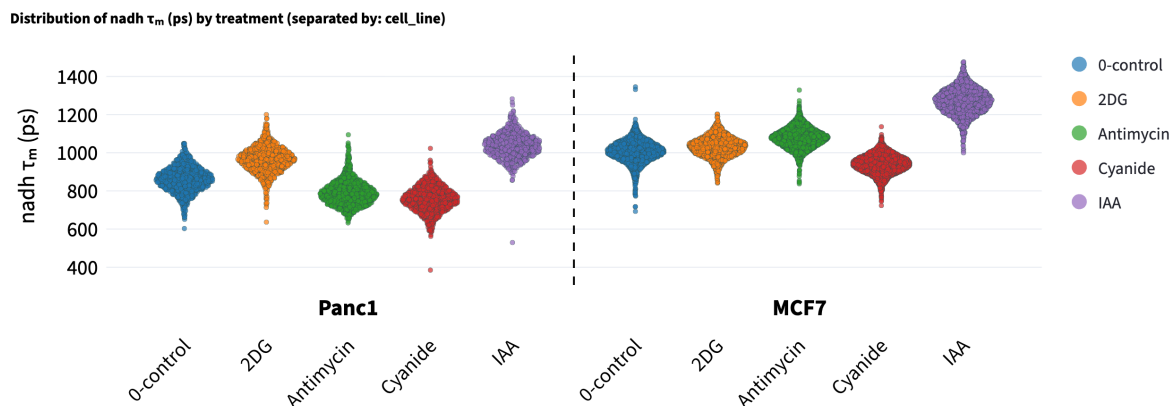


Figure 1.1: Compare a lifetime measurement across treatments, with a separate section for each cell line.

3. **Save the workflow.** Below **Plot Styling**, click **Export the entire analysis as a Python script**. Keep the original CSV and follow the [script workflow](#) to reproduce the plot.

Export the entire analysis as a Python script

Figure 1.2: Download a Python script containing the current analysis settings.

1.2 Try Data Extraction locally

Use a [local installation](#) for Data Extraction so the app can read input folders and save CSV files on your computer. The **Mosaic example** contains 25 NADH fields of view (Mosaic01–Mosaic25), with raw FLIM decays, cell masks, and an IRF. It is the same example used in the [IRF calibration and time-gate walkthrough](#).

1. **Download the inputs.** Download [the Mosaic example ZIP](#) from the FLIM Playground repository and unzip it. Open the extracted 3d_decay_1channel folder. It contains:
 - 25 .sdt decay files, such as Mosaic01.sdt.
 - 25 matching cell masks, such as Mosaic01_Ch2_summed_cellpose.tiff.
 - nadh_irf.txt, shared by all 25 fields of view.
2. **Configure the example.** In **Home**, create a profile such as Mosaic NADH example and enter the settings below. Click **Update Configuration** to save.

Setting	Value for this example
Number of channels / Channel 1 name /	1 / NADH / FLIM
Imaging modality	
FLIM input format	Decay (3/4D)
Extract feature types from NADH	Lifetime fit and Lifetime fit free
Number of components	2
Laser rate	0.08 GHz
Fit free calibration method	IRF
Decay suffix	.sdt
Mask suffix	_Ch2_summed_cellpose.tiff
IRF suffix	nadh_irf.txt
Unique cell identifier / FOV column name	cell_id / image_name

The app reads the image dimensions and timing from the SDT files: 512 × 512 pixels, 256 time bins, and a 12.5 ns decay period. See [configuration](#) when adapting this setup to your own inputs.

3. **Check the folder.** Open **Data Extraction** → **FOV Metadata Extraction** and enter the full path to the extracted 3d_decay_1channel folder. Confirm that Mosaic01–Mosaic25 report **All files found**, inspect a decay and mask preview, then click **Export FOV Metadata as CSV**. See the [metadata walkthrough](#) if a file is missing or duplicated.
4. **Calibrate and extract.** Switch to **Numeric Feature Extraction** to use the cached metadata. Keep **MLE**, **Hybrid**, two components, **Fix the Shift** selected, and the initial **Start (T1) = 0**, **End (T2) = 256** gates. Click **Optimize for Shifts**, then select Mosaic16 in the shift plot to inspect its decay and fitted curve.

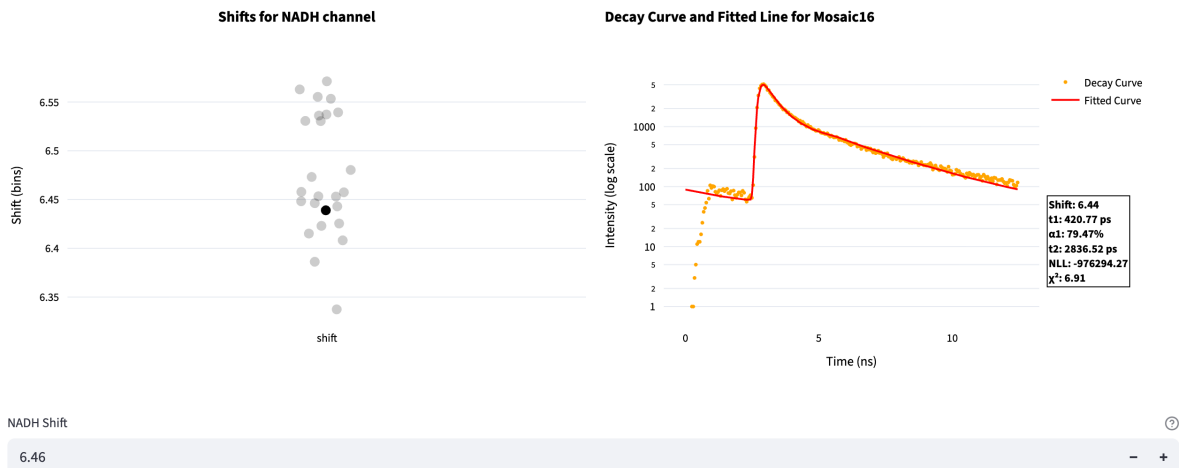


Figure 1.3: Inspect Mosaic16 in the 25-FOV shift distribution before starting cell-level extraction.

Follow [IRF Calibration](#) to review the shifts; the [time-gate example](#) shows how to adjust the fitting interval. Click **Confirm Time Gates (if applicable) and Shift for each channel**, choose **Local** for faster cell-level fitting or keep **Hybrid**, then click **Confirm and Start**. Use **Download updated metadata** to save the calibration for another run. The app saves the cell-feature CSV beside the metadata; see [Save Results](#).

5. **Analyze the cells.** Upload the saved cell-feature CSV in **Data Analysis** with **Use Dataset from Data Extraction** checked. Select **tm** under **Lifetime fit_NADH** in **Feature Comparison**, or open **Bivariate** → **Phasor Plot** to view the NADH coordinates. Use **Color by** → **image_name** to compare fields of view. For experiments whose FOV names encode labels such as treatment, follow [Categorical Feature Extraction](#) before analysis.

1.3 Installation

FLIM Playground is built entirely in Python and is open-source.

For the complete, always-current instructions — per-platform download links, install steps, **upgrading**, and building from source — see the project README, kept as the single source of truth:

👉 [Installation & upgrade guide](#)

1.4 Introduction

Fluorescence lifetime imaging microscopy (FLIM) measures the time it takes for a fluorescent molecule to emit light (return to the ground state) after being excited by a pulse of light (enter the excited state). It is sensitive to changes in fluorophore microenvironment including, pH, temperature, and conformational changes due to protein-binding and the presence of quenchers¹. Coupled with modern automated cell-segmentation methods², FLIM enables single-cell analyses that can reveal biological heterogeneity.

Instrumentation

To acquire FLIM data, a light source—typically a pulsed laser for time-domain methods or a modulated continuous-wave source for frequency-domain methods—is used to excite the fluorophore of interest. The emission is detected using instrumentation capable of resolving fluorescence decay, such as time-correlated single-photon counting (TCSPC), time-gated, or phase/modulation-based detection. In time-domain FLIM, the delay between excitation and photon arrival is measured, and often a histogram is built, with the x-axis representing the delay time and the y-axis representing the number of photons falling into each time bin. Compared to intensity images, FLIM has an additional dimension of time (e.g., 256 time bins per 12.5 nanoseconds).

In frequency-domain FLIM, the phase shift and modulation depth of the emission relative to the excitation are determined.

1.5 Challenges

A diverse set of **tools** — both open-source and commercial, ranging from libraries to code-free graphical user interfaces (GUIs) — is available to extract and analyze FLIM data, providing alternative methods and therefore flexibility to FLIM researchers. Examples include **PhasorPy**³, an open-source library for analyzing fluorescence lifetime using the phasor approach; **FLUTE**⁴, an open-source GUI for interactive phasor analysis; **FLIMPA**⁵, an open-source phasor analysis GUI enabling batch processing, ROI-based quantification, and experiment-level comparison through manual assignment; **FLIMLib**, an open-source generic curve fitting library that can be used to fit fluorescence lifetime decay data; **SPCImage**⁶, a commercial software for fitting and phasor features.

However, while some tools offer code-free interfaces, users still need to write custom code—either to prepare data in the proper format as input, or to further process their outputs for downstream analysis. The fragmentation between tools arises because each focuses on only a subset of **data levels**: pixel, cell ROI, channel, field of view, and experiment.

i Data Levels

- **Pixel**:
 - A single decay curve encoded in vendor-specific file formats (e.g., Becker & Hickl, PicoQuant, etc.)
- **Region of Interest (ROI)**
 - Mask with cell labels
- **Channel**
 - Different fluorophores
 - Fluorophore-specific calibration files
 - Masks focusing on different parts of the cell (e.g., whole cell, cytoplasm, nucleus, stain, etc.)
 - Different feature extraction methods (fitting, phasor, morphology, texture)
- **Field of View (FOV)**
 - Input decay types (e.g., **2D**, **3/4D**, **pixel-level pre-fit lifetime features**)
- **Experiment**
 - Different treatments, time points, cell lines, etc., and combinations thereof

An integrated framework should take into account all data levels      (•) while maintaining the flexibility to handle various input types (°). **It should provide a level of abstraction to address fragmentation from data levels and input types.**

Additionally, the use of FLIM is rapidly evolving, and new methods are being developed all the time. An integrated framework should allow users to choose among alternative methods seamlessly, be the backbone of iterative explorations integral to research, and be ready to incorporate new methods: **The provided level of abstraction should address fragmentation from extraction and analysis methods.**

Finally, many of the existing tools, including GUI-based software, are not cross-platform, which limits their accessibility. The [Installation](#) addresses this last challenge.

A closer parallel to FLIM Playground's integrated approach is the combination of [CellProfiler](#)⁷, which extracts per-object morphological and texture features, and [CellProfiler Analyst](#)⁸, which ingests these outputs or other feature tables for visualization and statistical analysis. However, FLIM Playground stands apart in that it can extract lifetime features from time-resolved data, alongside morphological and texture features from intensity images. Its general Data Analysis section incorporates FLIM-specific methods (e.g., [phasor analysis](#)) and provides statistical models that address additional analysis aspects such as data heterogeneity, in addition to training machine learning classifiers.

1.6 Method

1.6.1 Feature Classes

Tabular data columns can be categorized into three feature classes:

- **Identifiers**: an optional row identifier for tracking observations; the app supplies row numbers for user tables without one
- **Categorical features**: conceptually help us group the rows (e.g., treatment will group the rows into different treatment groups)
- **Numerical features**: quantify the differences/similarities between data groups

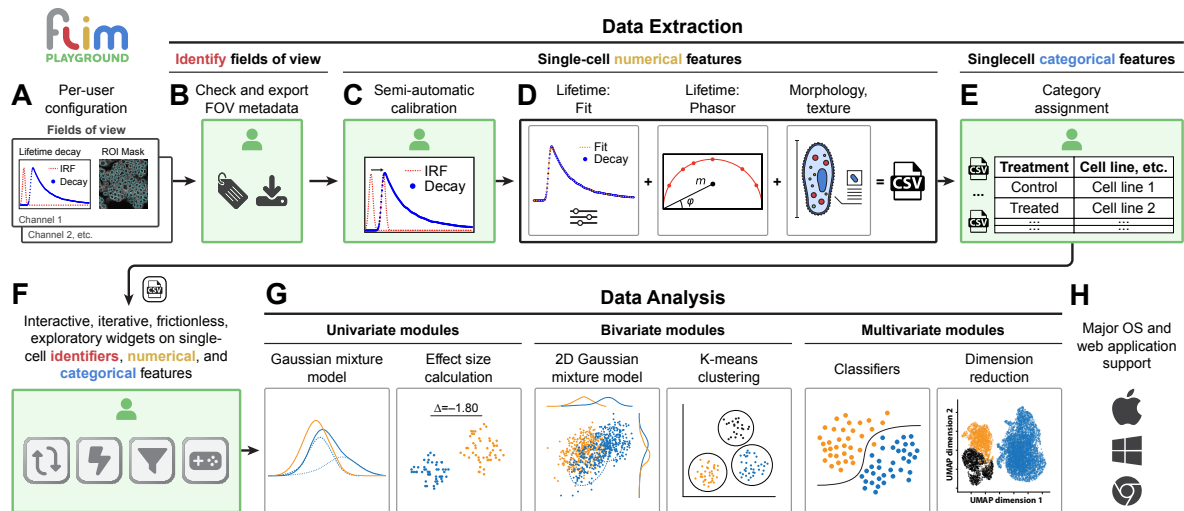
Science, from the data perspective, is about closing the conceptual categorical gaps with quantitative measurements.

At data levels, the data are processed to extract the feature classes in [Data Extraction](#), and the classes are used in the [Data Analysis](#) section. A user table can also mark columns **Ignore**; field-of-view labels use the **Categorical** role. See [column roles](#).

	Data Levels	Data Processing	Feature Classes
Increasing complexity ↑	Experiment	Assign category labels (e.g., treatment, cell line, etc.) to each cell; perform data analysis on labeled cell groups.	Categorical features
	Field of View	Assign FOV identifier to each cell, obtain unique cell identifiers	Identifiers
	Acquisition Channel	Apply per-channel ROI mask, calibration and extraction methods to cell-level signals	Numerical features
	Cell ROI	Aggregate pixel-level decays to obtain cell-level signals	N/A
	Pixel	Read pixel-level decays encoded in different formats	N/A

1.6.2 Design

FLIM Playground has two independent sections:



1.6.2.1 Data Extraction

Data Extraction extracts single-cell features from the raw data. It adopts a **framework** that offers channel-level flexibility in input types and extraction methods without incurring too much overhead for users. Following the above **categorization**, it is divided into the following steps:

- [Data Extraction Configuration \(A\)](#): allows users to choose among alternative input types and extraction methods (extractors).
- [Metadata organization \(B\)](#): extracts the field of view [identifiers](#) and their configurations
- [Numerical Feature Extraction](#): extracts single-cell [numerical features](#) based on user-selected extractors. More extractors can be integrated in the future.
 - [IRF Calibration \(C\)](#): estimate shifts for raw lifetime fitting or IRF-based phasor extraction; [dye-based calibration](#) is applied during phasor extraction
 - Lifetime extractors (D):
 - * [Fitting](#)
 - * [Phasor](#)
 - Intensity-based extractors (D):
 - * [Morphology](#)
 - * [Texture](#)
 - [Derived features](#)
- [Categorical feature extraction \(E\)](#): extracts single-cell [categorical features](#) and combines experiment-level datasets.

1.6.2.2 Data Analysis

[Data Analysis](#) analyzes features—whether extracted through Data Extraction or by other methods—using visualizations and statistical modeling. It deploys a [shared framework \(F\)](#) built to handle the [feature classes](#) across all analysis methods, enabling the same interactive and frictionless exploration experience and allowing new methods to be integrated in the future easily.

- [Data Analysis \(G\)](#) goes in-depth into how FLIM Playground handles the feature classes and [Data Analysis Config](#) goes through how users can configure FLIM Playground to analyze datasets that are not extracted by [Data Extraction](#).

Here is the list of analysis methods incorporated into FLIM Playground, grouped by the number of numerical features they take as inputs:

- [Univariate analysis](#)
 - [Feature Comparison](#)
 - [Feature Histogram](#)
- [Bivariate analysis](#)
 - [2D Feature Distribution](#)
 - [Phasor Plot](#)
- [Multivariate analysis](#)

- [Dimension Reduction](#)
- [Classification](#)

Both sections are built in Python and as self-contained apps ready to run on major operating systems and in browsers (**H**).

1.6.3 Summary

FLIM Playground resolves these [challenges](#) with an interactive code-free graphical user interface (GUI) that spans the full pipeline. It integrates validation checks that guide users at every step, and has a built-in repertoire of analytical methods with interactive widgets that encourage hypothesis-driven, iterative exploration of large datasets. It is built on a modular architecture that enables incorporation of new algorithms in the future.

1.6.4 Citation

FLIM Playground is published in [Cell Reports Methods](#). If it contributed to your research—whether through Data Extraction for single-cell feature generation or through Data Analysis for data exploration, visualization, selection of analysis methods, or hyperparameter tuning (UMAP, clustering, classification, etc.)—please cite this work in your publication. Your citation directly supports us in maintaining and improving it ✨🎈🍷.

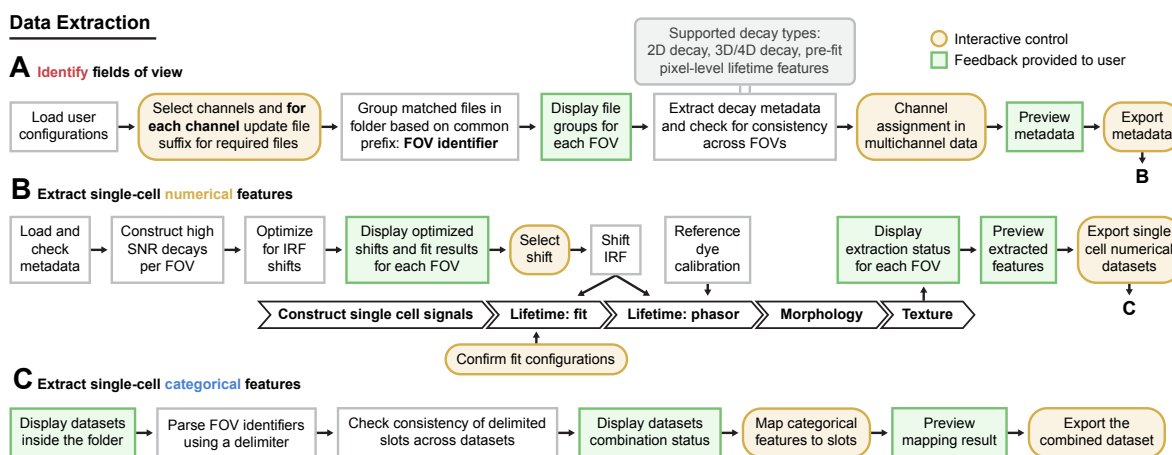
Part I

Data Extraction

2 Overview

FLIM Playground transforms raw FLIM data into single-cell numerical and categorical features, ensuring smooth integration with [downstream analysis](#). All steps are seamlessly connected through interactive widgets, with built-in error checking and reporting to ensure correctness.

A high-level overview of the data extraction process is shown below.



2.1 Stages

Users can [configure](#) the system once and apply it to future data. After configuration, users can extract features from raw data using the following steps, inspired by the [categorization](#):

Select a step to perform (?)

- ☒ FOV Metadata Extraction
- ☐ Numeric Feature Extraction (fitting, phasor, etc.)
- ☐ Categorical Feature Extraction (e.g. treatment)

Figure 2.1: Choose the extraction stage from the workflow selector.

1. [FOV Metadata Extraction \(A\)](#): each experiment session consists of multiple fields of view (FOVs). This step helps users organize the metadata of the FOVs.
2. [Numeric Feature Extraction \(B\)](#): extracts single-cell numerical features, with IRF shift calibration first for the channels that require it.
 - [IRF Calibration](#)
 - [Fit Calibration](#)
 - [Fit-Free IRF Calibration](#)
 - [Fluorescence lifetime standard](#): calibrates phasors with a reference dye during extraction.
 - Per-channel feature extraction includes:
 - [Lifetime fitting features](#): fit an exponential decay curve to the measured decay and extract lifetime features per cell ROI.
 - [Phasor features](#): calculate the phasor features such as the phasor coordinates. It also provides [phasor analysis](#).
 - [Morphology features](#): calculate morphology features based on the ROI mask.
 - [Texture features](#): calculate texture features based on the intensity image and the ROI mask.
3. [Categorical Feature Extraction \(C\)](#): extract the categorical features such as the treatment, time point, etc. based on the FOV name.
 - Users can then upload the extracted datasets to perform [Data Analysis](#).

2.2 Input File Types

2.2.1 Decay

See [decay types](#) for more details.

- 2D decay:
 - [.csv](#): a headerless numeric table with one cell per row and one time bin per column. Do not include identifier or annotation columns.
- 3D/4D decay: besides the spatial dimensions and the time dimension, the additional dimension in the 4D array represents acquisition channels.
 - [.sdt](#): Becker & Hickl
 - [.ptu](#): PicoQuant

2.2.2 Mask

Image-based inputs need a cell-level mask for each channel. Channels can share a mask or use different masks; when masks differ, the integer label for the same cell must match across them. A 2D decay table does not need a mask: its rows identify the cells.

The cell-level mask can focus on different parts of the cell region, such as the whole cell, the cytoplasm, the nucleus, or the stain part.

For an example, see [here](#).

- `.tiff/.tif`: a 2D array with background labeled as 0 and each cell ROI region labeled as a unique positive integer. That integer will be part of the [unique cell identifier](#).

2.2.3 IRF

The IRF is used as a 1D reference curve, with one value per decay time bin. Supported formats are:

- `.txt`: it uses `np.loadtxt` to parse a txt file. It returns a 1D array if the IRF numbers are in one row or in one column.
- `.csv`: comma-separated IRF values. The resulting IRF must contain one value per decay time bin.
- `.ptu`: single-channel T3 imaging data. All pixels and repeated frames are summed into one reference curve. The acquisition timing must match the sample; see [calibration reference validation](#).

2.2.4 SPCImage t1

If the decay type is [3D/4D pixel pre-fit](#), users can provide pixel pre-fit lifetime feature files.

- `.asc`: a 2D array in spatial dimensions outputted by [SPCImage](#), with each (row, column) containing the value of the lifetime feature (e.g. `t1`: the first component's lifetime) of that pixel.
- Depending on the chosen number of lifetime components, the other pixel pre-fit feature files are included and their names are inferred from the `t1` file, by replacing `t1` in the file name with `t2`, `t3`, `a1 [%]`, etc.

2.2.5 Fluorescence Lifetime Standard

If the fluorescence lifetime standard-based calibration is chosen, users are expected to provide the fluorescence lifetime standard file.

- `.tiff/.tif`: a 3D array, with one dimension representing the time axis.
- `.ptu`: single-channel T3 imaging data. All pixels and repeated frames are summed into one reference curve. The acquisition timing must match the sample; see [calibration reference validation](#).

2.2.6 Intensity (2D)

If the channel's [imaging modality](#) is Intensity-only, users are expected to provide a 2D intensity image in:

- `.tiff/.tif`: a 2D array.

2.3 Limitations

- To simplify the workflow and due to the framework chosen, FLIM Playground does not support pixel-level fitting and fit-free analysis, and there are many other open-source tools that can do so (for a comprehensive list, see [here](#)). Instead, it does cell-level fitting and fit-free (phasor) feature extraction by summing up all the decays belonging to the same cell ROI to get one cell-level decay as a preprocessing step⁹. It also accepts pre-fit pixel-level lifetime features from [SPCImage](#) and [aggregates pixel-level lifetime features to cell-level lifetime features](#).

3 Configuration

Note

Configure FLIM Playground once and it is applied to all future data. You can also keep multiple named **configuration profiles** and switch between them — see [Extraction Profiles](#).

Constructing a framework that balances flexibility and usability is about choosing the right abstraction level. FLIM Playground chooses acquisition *channel* from data levels as the focus and adopts the **channel-centric framework**. Settings are divided into two categories: [cross-channel configurations](#) and [per-channel configurations](#).

The page begins with the active profile, channel count, and identifier names. `Channels` contains channel names and imaging modalities. `Shared FLIM settings` appears when at least one channel uses FLIM, followed by per-channel `Extraction settings`. Categorical columns and the derived-feature builder are near the bottom.

Deploy

[Home](#)
[Data Extraction](#)
[Data Analysis](#)
v1.13.0-7-gfb49ba-dirty



Welcome 🎉 to **FLIM Playground**! 🐞 📄
[Data Extraction](#) pulls single-object features from raw microscopy data.
[Data Analysis](#) transforms tabular datasets into insights through visualization and statistical modeling.

Configuration

Profile ? New profile
 Documentation ▼ e.g. experiment-B
Delete + Create

Number of channels ? 2 ▼
 Unique cell identifier column name ? cell_id
 FOV column name ? image_name

Channels

Channel 1 name nadh
 Channel 2 name fad
 Imaging modality FLIM ▼
 Imaging modality FLIM ▼

Shared FLIM settings

FLIM input format ? Decay (3/4D) ▼
 Laser rate (GHz) for Decay (3/4D) ? 0.08 - +
 Fit free calibration method ?
☒ IRF ☐ Fluorescence Lifetime Standard

Extraction settings

Extract feature types from nadh ? Lifetime fit × Lifetime fit free × Intensity morph... × Intensity texture × ▼
 Extract feature types from fad ? Lifetime fit × Lifetime fit free × Intensity morph... × Intensity texture × ▼
 Number of components for nadh ? 2 ▼
 Number of components for fad ? 2 ▼

Figure 3.1: Configuration starts with the active profile, identifiers, channels, and shared FLIM settings.

3.1 Extraction Profiles

Keep up to **10 extraction profiles**, each storing the settings for an experiment or dataset.

On the **Configuration** page:

- Select an existing profile from the **Profile** dropdown to load it.
- Enter a name in **New profile**, then click **Create**.
- Delete the active profile with **Delete** (disabled when only one profile remains).

Configuration

Profile ? New profile
 Documentation ▼ e.g. experiment-B
Delete + Create

Figure 3.2: Switch, create, or delete extraction profiles from Configuration.

A newly created profile starts blank. Fill in the settings described below and click **Update Configuration** to save it. Everything described below pertains to the current profile.

3.2 Cross-Channel Configurations

3.2.1 Categorical Features

Categorical features organize observations into groups in [Data Analysis](#). Use `Categorical` columns (type to add more) to choose names for the [Categorical Feature Extraction](#) step; type a new name to add it.

3.2.2 Decay Types

Choose FLIM input format under Shared FLIM settings: 2D decay, 3D/4D decay, or 3D/4D pixel pre-fit input.

3.2.2.1 2D Decay

In the system developed by Samimi et al.¹⁰, single cells flow through and a decay curve is acquired for each cell by aggregating all the photon arrival time delays deemed to be from the same cell. In exchange for spatial distribution, the acquisition speed is increased enormously. The output of the system is a tabular data sheet with each row representing a cell and each column representing a time bin. **FLIM Playground currently supports 2D decay in the CSV format.**

Note

A 2D decay CSV must be headerless, with only numeric time bins. Do not include a header, cell-label column, or annotation column. Non-numeric columns are rejected because every column must represent a time bin.

3.2.2.2 3D/4D Decay

In TCSPC-based FLIM, the data is stored in a 3D/4D array and in special format (e.g. `.sdt`, `.ptu`). In addition to the spatial dimensions and the time dimension (XYT), the channel dimension may also be present, making it 4-dimensional (CXYT). **FLIM Playground currently supports both `.sdt` and `.ptu` files as 3D/4D decay images.** Here, T is the photon-arrival time axis within a laser pulse interval.

It reads the `duration` and the number of `time` bins from the decay file's acquisition metadata. For PTU files, the bin count is the laser period divided by the TCSPC resolution, rounded down to a whole

number of bins. The duration is **bin count** $\times$ **TCSPC resolution**, expressed in nanoseconds. This preserves the recorded bin width and can make the duration slightly shorter than the nominal laser period. The bin count may not necessary be the last time bin that happened to receive a photon.

Repeated frames in a PTU file are summed into one decay image per FOV, and the [FOV metadata step](#) warns when frames have been combined. For actual time-lapse acquisitions, split the data into one file per time point before extraction so that the time points remain separate.

3.2.2.3 3D/4D pixel pre-fit

Since FLIM Playground currently does **not** support pixel-fitting, users have the option to provide pixel pre-fit outputs from [SPCImage](#) in the format of .asc. See [here](#) for how FLIM Playground processes SPCImage outputs to obtain cell-level lifetime fitting features.

3.2.3 Identifiers

3.2.3.1 Cell Identifier

Users can specify the column name for the unique identifier of the cell to be stored in the output dataset. The cell identifier is constructed as {FOV Identifier}_{cell_label}, where the `cell_label` is the unique integer label in the mask or the row number if the decay type is [2D decay](#).

3.2.3.2 FOV Identifier

Users can specify the column name for the field of view identifier. If the decay type is [2D decay](#), users may want to specify the column name as `exp_name`, because the FOV is not an image.

3.2.4 Decay Info

3.2.4.1 Laser frequency

Enter the laser frequency in GHz, up to 1 GHz; for example, 0.08 GHz equals 80 MHz. This control appears when at least one FLIM channel selects `Lifetime fit free`, and the value is used for phasor extraction and [analysis](#).

3.2.4.2 2D Decay-specific

Since the [2D decay](#) file does not provide the duration and the number of time bins per laser pulse interval, users need to specify them.



The screenshot shows the 'Shared FLIM settings' panel. Under 'FLIM input format', 'Decay (2D)' is selected. The 'Laser rate (GHz) for Decay (2D)' is set to 0.08. The 'Fit free calibration method' has 'IRF' selected. The 'Decay (2D) duration (ns)' is set to 12.50 and the 'Decay (2D) time bins' is set to 200.

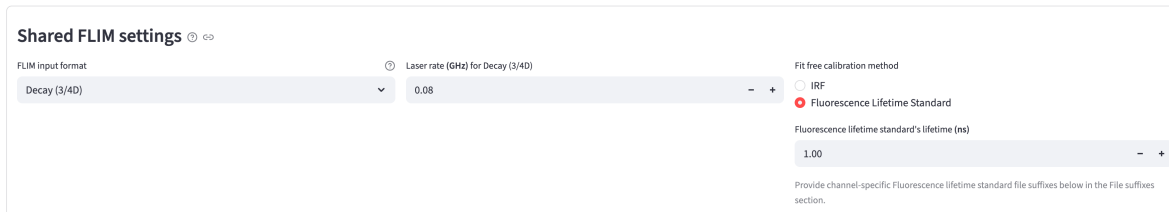
Figure 3.3: For 2D decay input, enter the acquisition duration and time-bin count.

3.2.5 Calibration Method

When `Lifetime fit free` is selected, choose between two calibration methods:

- IRF: [shift-based calibration](#).
- Fluorescence Lifetime Standard: [reference-based calibration](#).

Since the instrumental response function (IRF) or the fluorescence lifetime standard is measured for each channel, they are set in the [per-channel configurations](#). The fluorescence lifetime standard's lifetime is channel-agnostic.



The screenshot shows the 'Shared FLIM settings' panel. Under 'FLIM input format', 'Decay (3/4D)' is selected. The 'Laser rate (GHz) for Decay (3/4D)' is set to 0.08. The 'Fit free calibration method' has 'Fluorescence Lifetime Standard' selected. The 'Fluorescence lifetime standard's lifetime (ns)' is set to 1.00. A note at the bottom states: 'Provide channel-specific Fluorescence lifetime standard file suffixes below in the File suffixes section.'

Figure 3.4: Standard-based phasor calibration uses a shared reference lifetime.

3.2.6 Number of Channels

Each FOV is assumed to have the same number of channels. The configuration will dynamically adjust the [per-channel configurations](#) based on the specified number of channels. Currently, the maximum number of channels is 8. When more than four channels are configured, the first four fold into a collapsed Channels 1-4 group while the newer channels (5 and up) stay expanded, so the ones you just added remain in view.

3.3 Per-Channel Configurations

3.3.1 Channel Name

Users can customize the channel name based on, for example, the name of the fluorophore.

3.3.2 Imaging Modality

Users can specify the imaging modality for each channel. Currently, two modalities are supported:

- FLIM: The signal is time-resolved, i.e., has a time dimension, and is expected to be from one of the [decay types](#).
- Intensity-only: The signal is a spatial map of intensity values (2 spatial dimensions), i.e., has no time dimension. **Currently, the only supported signal type is a 2D intensity image in the format of .tiff/.tif.**

Tip

This setup allows users to image some channels in FLIM mode, and others in non-FLIM mode. It also allows users to extract and analyze their data if they image in non-FLIM microscopic systems such as epifluorescence, confocal, brightfield, etc. by selecting Intensity-only for all channels.

Note

A configuration cannot mix [2D decay](#) FLIM channels with intensity-only channels. Select a 3D/4D format or change the channel modalities.

3.3.3 Feature Extractor

Four types of feature extractors are available if the [imaging modality](#) is FLIM:

- [Lifetime fit](#)
- [Lifetime fit free](#)
- [Intensity Morphology](#)
- [Intensity Texture](#)

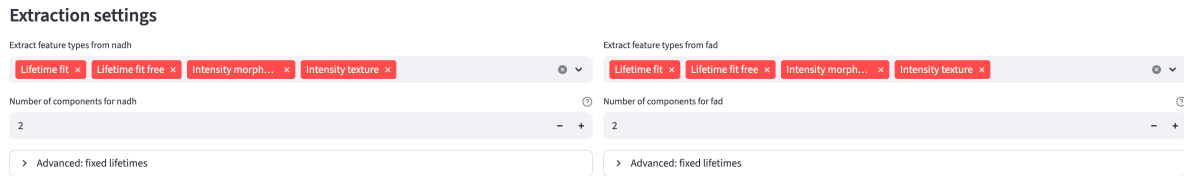


Figure 3.5: Choose extractors and lifetime component counts independently for each channel.

i Note

If the decay type is [2D decay](#), the [Intensity Morphology](#) feature extractor is **not** applicable, because there is no ROI mask to measure. The [Intensity Texture](#) extractor *is* available, but a 2D decay CSV has no intensity image, so it produces only the single `intensity_sum` feature (the sum of each cell's decay curve).

The [Intensity Morphology](#) and [Intensity Texture](#) feature extractors are available if the [imaging modality](#) is [Intensity-only](#).

Each channel can be assigned its own set of feature extractors. For example, if users do not intend to do lifetime extraction, they can de-select the `Lifetime fit` and `Lifetime fit free` options.

3.3.4 Number of Components

If users select the `Lifetime fit` under this channel, they can specify the number of components to fit.

3.3.5 Fixed Lifetimes

When the number of components is greater than one, open `Advanced: fixed lifetimes` to set a known component lifetime in nanoseconds, using controls such as `Fix 1 (ns)` and `Fix 2 (ns)`. Set a component to 0 to leave it free for optimization. Positive values become defaults for the [interactive fitting phase](#).

i Note

Fixed lifetimes apply only to channels that FLIM Playground fits. For [pre-fit](#) inputs (SPCImage .asc), whose lifetimes are read rather than fit, the fixed-lifetime controls are not shown.

3.3.6 File Suffix

The [FOV Metadata Extraction](#) step finds inputs using file suffixes. The selected [decay type](#), [calibration method](#), and [extractors](#) determine the required file types. Enter a filename suffix for each. Additional SPCImage component suffixes are inferred from the τ_1 suffix.

File suffixes: nadh	File suffixes: fad
Decay _n.sdt	Decay _f.sdt
IRF nadh_irf.txt	IRF fad_irf.txt
Mask _n_photonscell_mask.tiff	Mask _n_photonscell_mask.tiff

Figure 3.6: File suffixes identify each channel's decay, calibration, and mask inputs.

The two-channel example uses separate NAD(P)H and FAD decay and IRF suffixes. Both channels use the same mask suffix to measure the same cell ROIs. Choose suffixes that unambiguously identify each input in the folder you will scan.

For detailed information on what file formats are supported for each file type (Mask, IRF, etc.), see [input file types](#).

For detailed information on how the file suffixes are used, and what each file type (e.g. IRF, Mask, etc.) is, see the [FOV Metadata Extraction](#) page.

3.4 Derived Features

The Configuration page also hosts the `Derived features` builder, which composes new single-cell features by arithmetic over the features you already extract — optionally combining operands from different channels (for example, an optical redox ratio). Definitions are saved with the [current profile](#) and computed later, during [Numerical Feature Extraction](#). See [Derived Features](#) for the full walkthrough.

3.5 Save

Finally, users can save the configuration into the **current profile** for future use by clicking the `Update Configuration` button.

If you keep FLIM Playground open in more than one browser tab, saving here does not reach the others on its own. Any tab already showing Data Extraction will notice and post a small notice asking you to reload it, so you can finish what you are in the middle of first and pick up the new settings when it suits you.

4 Field of View Metadata

It is the first step in the [Data Extraction](#) workflow. It organizes the metadata of each field of view (FOV)—including file paths for all required inputs and decay information—into a single metadata file. This file is then used for [numerical feature extraction](#), as well as for users’ own bookkeeping and troubleshooting.

The screenshot shows the 'Data Extraction' interface. On the left, there's a configuration panel with options for 'FOV Metadata Extraction' (selected), 'Numeric Feature Extraction', and 'Categorical Feature Extraction'. Below these are checkboxes for 'has nadh' and 'has fad', both checked. There are also buttons for 'Feature extractors for nadh' and 'Feature extractors for fad'. A 'Laser rate (GHz)' input is set to 0.08. Under 'File suffixes: nadh', there are input fields for Decay (_n_sdt), IRF (nadh_irf.txt), and Mask (_n_photonscell_ma). Under 'File suffixes: fad', there are input fields for Decay (_f_sdt), IRF (fad_irf.txt), and Mask (_n_photonscell_ma). A note states: 'Channels ["nadh", "fad"] share the same mask suffix: _n_photonscell_mask.tif'. At the bottom, there's a 'Copy the folder path here' section with a text input containing '/private/tmp/fp-docs-resume-20260909/extracton-example' and a 'Rescan folder' button.

On the right, the 'Fields of view:' section shows 'MCF7_2DG_dish2_04' with a green checkmark and the text 'All files found.' Below this, a green box contains the text: 'Fields of View with [checkmark] are loaded successfully [smiley face]. FOVs with [X] (if any) will not be recorded. Here is the preview of the FOVs and metadata recorded:'. Below this is a table with the following data:

	image_name	nadh_Decay	nadh_IRF	nadh_Mask
0	MCF7_2DG_dish2_04	/private/tmp/fp-docs-resume-20260909/extracton-example/MCF7_2DG_dish2_04_n	/private/tmp/fp-docs-resume-20260909/extracton-example/nadh_irf.txt	/private/tmp/fp-do

Below the table is a button labeled 'Export FOV Metadata as CSV'.

Figure 4.1: Metadata settings appear beside the file checks and metadata preview.

It is divided into two sections:

- The left 1/3 of the screen is the metadata configuration panel, inherited from the [Data Extraction Configuration](#) step.
- The right 2/3 shows the extraction status for all FOVs.

4.1 Metadata

Metadata are configured earlier in the [Data Extraction Configuration](#) step and copied here. Some fields remain editable for flexibility, while others are fixed to ensure usability. Non-editable fields can be modified in the Data Extraction Configuration step. After saving your modifications, return to this page and refresh it to see the changes.

4.1.1 Decay Type

See [decay types](#) for more details. The decay type is not editable here.

4.1.2 Channel Names

See [channel name config](#) for more details. The names are not editable here, though users can choose which channel to include.

4.1.3 Feature Extractors

See [feature extractor config](#) for more details. The feature extractors for each channel are shown but not editable here.

4.1.4 Decay Info

The [decay settings](#) start from the active profile. Laser rate is editable when `Lifetime fit free` is selected; 2D inputs also show duration and time-bin controls.

4.1.4.1 2D Decay-specific

Since the [2D decay](#) file does not provide the duration and the number of time bins per laser pulse interval, users need to specify them.

Decay input type: Decay (2D)

☒ has nadh

> Feature extractors for nadh

Duration (ns)

12.50

–

+

Time bins

200

–

+

Laser rate (GHz)

0.08

–

+

Figure 4.2: Review the timing values used to interpret a 2D decay table.

4.1.5 File Suffix

Copied from the [file suffix config](#). Users can edit the file suffixes here.

4.1.6 Fluorescence Lifetime Standard

When [Fluorescence Lifetime Standard](#) calibration is selected, specify the standard's filename suffix for each fit-free channel under `File suffixes`, and its shared lifetime in nanoseconds. These values start from the active configuration. The folder scan finds each standard by suffix, independently of the FOV name.

4.1.7 Folder Path

Finally, users can specify the folder path that contains all the required input files.

4.2 Metadata Extraction

4.2.1 FOV File Paths

The first step is to find the fields of view:

- It finds files recursively in the folder path using the first per-FOV file suffix, skipping calibration suffixes.
- The prefix of all the matched files is considered to be the field of view name.

- FOV name = file name - 1st file suffix

Then, it uses the found FOV names to find all other files:

- The file name to search = FOV name + other file suffixes for each file suffix

Calibration files are shared across FOVs and are searched for by suffix alone, independently of the FOV name. For example, the same `standard.ptu` or `irf.ptu` will be used for every FOV in the folder. Files matching configured IRF or fluorescence lifetime standard suffixes are excluded from FOV discovery, even when they share the sample's `.ptu` extension. This also applies to configured references for an inactive calibration method; those files are excluded if present but are not required.

Each FOV card reports whether the required inputs were found:

- **Missing:** no file matches the expected FOV name plus suffix, or no calibration file matches its suffix.
- **Duplicate:** multiple files match a calibration suffix, or the same per-FOV filename appears in different subfolders. Keep one unambiguous match for each required input.

FOVs marked  are included in the metadata; FOVs marked  are excluded.

Tip

After correcting missing or duplicate filenames, click `Rescan folder` to repeat the scan.

4.2.2 Decay Info

If the decay type is [3D/4D decay](#), FLIM Playground will try to infer the `duration` and the number of `time bins` per laser pulse interval from the decay file metadata. It will also check for inconsistencies across all decay files.

4.2.2.1 Channel Assignment

If the decay file associated with the channel found by the previous step is a 4D array, then FLIM Playground will try to infer the channel number for that channel. For each channel name, it will list all non-empty channels as potential channels to be assigned. If there is only one, the assignment is automatic. If there are multiple, users need to select the channel intended. It will also check for inconsistencies across all decay files about their dimensions and complain if there are any.

Picking the right channel number is easy to get wrong when one file holds several non-empty channels, so when there is a choice to make, a preview of the first field of view appears under the dropdown: one thumbnail per candidate channel, with a  on the one currently selected. Glance at it to confirm the channel you picked looks like the fluorophore you expect.

4.2.2.2 IRF

For channels that require IRF calibration, the metadata records the IRF file path. IRFs are checked when numerical extraction begins. The requirements depend on the [IRF format](#):

- **TXT/CSV:** the loaded curve must contain one value per sample time bin.
- **PTU:** the file must contain non-empty, single-channel T3 imaging data. All pixels and repeated frames are summed into one IRF curve. Its time-bin count and bin width must match the sample's; its laser frequency must also match when a sample laser frequency is specified. The sample bin width is $\text{duration} / \text{time_bins}$. Small numerical rounding differences are tolerated, but the IRF is **not resampled** to fit a different acquisition grid.

4.2.2.3 Fluorescence Lifetime Standard

When fluorescence lifetime standard calibration is selected, the standard is checked before metadata export. The requirements depend on the [standard format](#):

- **TIFF:** the image must be 3D, with exactly one dimension matching the sample's time-bin count so that the time axis can be identified. A missing or ambiguous time axis prevents metadata export.
- **PTU:** the file must contain non-empty, single-channel T3 imaging data. All pixels and repeated frames are summed into one standard reference curve. Its time-bin count and bin width must match the sample's; its laser frequency must also match when a sample laser frequency is specified. The sample bin width is $\text{duration} / \text{time_bins}$. Small numerical rounding differences are tolerated, but the standard is **not resampled** to fit a different acquisition grid.

The metadata records the standard's file path, known lifetime, and time axis. For a PTU standard, the app supplies the time axis automatically from the integrated reference curve.

4.2.3 Result Preview and Export

If there is a non-zero number of FOVs with  and channel assignment and fluorescence lifetime standard checks (if applicable) are passed, a preview of the metadata data sheet will be shown for users to check.

Fields of view:

MCF7_2DG_dish2_04

✓ All files found.

Fields of View with ✓ are loaded successfully 🗑️. FOVs with ✗ (if any) will not be recorded. Here is the preview of the FOVs and metadata recorded:

	image_name	nadh_Decay	nadh_IRF	nadh_Mask
0	MCF7_2DG_dish2_04	/private/tmp/fp-docs-resume-20260909/extraction-example/MCF7_2DG_dish2_04_n	/private/tmp/fp-docs-resume-20260909/extraction-example/nadh_irf.txt	/private/tmp/fp-do

Export FOV Metadata as CSV

Figure 4.3: Check the recorded file paths before exporting the FOV metadata.

As an example, the following metadata are extracted:

- **image_name**: The **image_name** column is the name of each FOV, which is specified in the [fov identifier config](#).
- **nadh_Mask**, **nadh_Decay**, **nadh_IRF**, **fad_Mask**, **fad_Decay**, **fad_IRF**: the [file paths](#) of the mask, decay, and IRF files for the nadh and fad channels.
- **nadh_input_type**, **nadh_imaging_modality**, **fad_input_type**, **fad_imaging_modality**: the [imaging modality](#) and the corresponding input type for each channel. These are two distinct values. For a channel whose modality is FLIM, the input type is one of the [decay types](#); for a channel whose modality is Intensity-only, the input type is Intensity (2D). Channels that share the same imaging modality share the same input type.
- **nadh_Lifetime fit free**, **nadh_Intensity morphology**, **nadh_Lifetime fit**, **fad_Intensity morphology**, **fad_Intensity texture**, **fad_Lifetime fit**: [feature extractors](#) for each channel.
- **nadh_channel**, **duration**, **fad_channel**, **time_bins**, **laser_rate**: the [decay info](#) for each channel and info shared by all channels.

Export FOV Metadata as CSV writes `fov_metadata_<timestamp>.csv` into the selected folder and caches it for the numerical extraction step.

5 Numerical Feature Extraction

FLIM Playground extracts *single-cell* numerical features from **multiple channels and imaging modalities in the same workflow**. Configure each channel independently: give it a name, choose the imaging modality, and select the [feature extractors](#) needed for that channel. For example, a FLIM channel can provide lifetime and phasor features while an intensity-only channel provides morphology and texture.

It is divided into two steps:

1. The optional calibration step. For example, **an IRF calibration for shift is required for raw FLIM channels**.
2. The feature extraction step that applies the selected feature extractors to each channel and saves the results to a CSV file.

The available feature extractors are:

- [Lifetime fit](#): fit exponential models to raw decay curves to estimate lifetimes and their fractional contributions, or read existing fitted values from pre-fitted FLIM input.
- [Lifetime fit free](#): transform decay curves to phasor space using Fourier transform to obtain phasor coordinates.
- [Intensity Morphology](#): morphological features of single-cell ROI masks
- [Intensity Texture](#): texture features of single-cell ROI intensity images (or, for 2D decay input, just the per-cell decay-curve sum, `intensity_sum`)

5.1 Input

A CSV file extracted in the [fov metadata extraction](#) step that contains the metadata of the FOVs including:

- [required file paths](#)
- [decay type](#)
- [decay info](#)
- [selected feature extractors](#) for each channel
- [fit free calibration method](#) (if fit free calibration is required)

Use the cached metadata just exported in [FOV Metadata Extraction](#), or upload a previously saved metadata CSV. Cached metadata takes precedence; refresh the page to clear it and upload a different file.

5.2 Feature Extraction

Click **Confirm and Start** to extract the selected features. For channels that need an IRF shift, first [confirm calibration](#) or load metadata with saved shifts. For raw FLIM channels using `Lifetime fit`, choose the fitting mode before starting.

The right side of the screen lists the fields of view and their extraction status. Each FOV has a progress bar while it is being processed; completed FOVs show their result badges. Keep this view open to follow the batch.

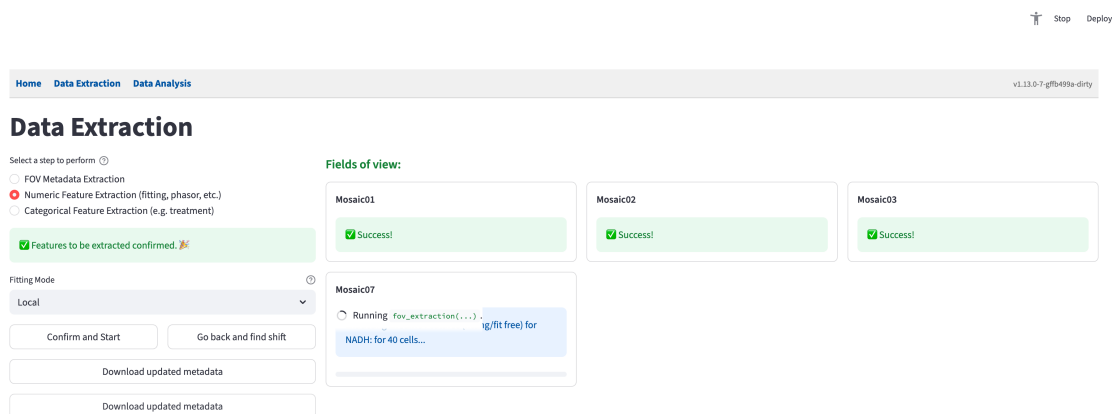


Figure 5.1: Full extraction screen: Mosaic01–03 are complete while Mosaic07 is processing.

Fields of view:

MCF7_2DG_dish2_04

✓ Success!

Fields of view features with ✓ are extracted successfully 🚨 FOVs with error messages are excluded. The first few rows of the features are shown below.

cell_id	fad_amp1	Lifetime fit_fad: t1	fad_offset	fad_reduced_chi_square	fad_amp2	Lifetime fit_fad: t2	Lifetime fit_fad: a1	fad_a2	Lifetime fit_fad: tm	Lifetime fit_fad:
MCF7_2DG_dish2_04_1	331.6151	186.3865	0.00000000000008	1.9071	126.1287	2640.0726	72.4456	27.5544	862.4858	225
MCF7_2DG_dish2_04_3	561.8295	202.2525	0.5753	2.418	191.6643	2551.6221	74.5633	25.4367	799.8557	210
MCF7_2DG_dish2_04_4	1068.7796	216.8308	0.00000000000007	3.7044	380.2513	2627.3213	73.7582	26.2418	849.386	21
MCF7_2DG_dish2_04_6	380.9845	215.6965	0.00000000000003	2.0441	125.8547	2656.9949	75.1687	24.8313	821.9023	217
MCF7_2DG_dish2_04_7	505.7736	199.5202	0.00000000000008	2.146	161.7739	2600.7303	75.7659	24.2341	781.4311	213

Download single cell features as CSV

Figure 5.2: This numerical extraction example produces 185 cell rows and 70 columns from one FOV.

The completed result above includes a preview of the extracted cell table.

Warnings are displayed if cells have NaN values for some features. They are *not* excluded from the final CSV file.

5.3 Save Results

Once the feature extraction is finished, the results are saved to a CSV file. How this happens depends on how the metadata reached this step. If the cached metadata file from the [fov metadata extraction](#) step was used, the CSV is written automatically into the same folder as that metadata file, and no button is shown. If the metadata file was uploaded manually, FLIM Playground cannot know where to write it, so a `Download single cell features as CSV` button appears at the bottom of the page instead.

6 IRF Calibration

The instrument response function (IRF) describes the detection system's response to an ultrashort light pulse. Its timing relative to a measured decay affects reconvolution fitting and IRF-based phasor calibration. This chapter covers estimating and confirming that shift in **Data Extraction** → **Numeric Feature Extraction**.

6.1 When this stage is needed

An IRF shift is required when at least one channel meets either of these conditions:

Channel input and selected extractor	How its shift is determined
Raw FLIM with Lifetime fit selected	Fit calibration optimizes the shift during reconvolution fitting. This is required even when phasor features use a calibration dye.
Raw or pre-fitted FLIM with Lifetime fit free selected and calibration method set to IRF	Fit-free IRF calibration estimates the shift by cross-correlation. If raw lifetime fitting is also selected, its fitted shift is reused.

The app evaluates these conditions separately for each channel. Estimate shifts for the channels that meet them, or reuse shifts already saved in the metadata. Then continue to [Numerical Feature Extraction](#).

A calibration dye is a **Fluorescence Lifetime Standard**: its reference-based phasor calibration runs during [fit-free feature extraction](#). It does not trigger this IRF shift stage. A workflow with only intensity-only channels also proceeds directly to extraction.

6.2 Input

Open **Data Extraction** → **Numeric Feature Extraction**. Use the cached metadata exported by [FOV Metadata Extraction](#), or upload a saved metadata CSV. The metadata records each channel's input files, modality, selected extractors, and calibration method. Cached metadata takes precedence; refresh the page to clear it before uploading a different file.

6.3 Fit Calibration

The IRF can shift relative to the measured decay between experiments. For a **raw FLIM channel with Lifetime fit selected**, estimate its IRF shift before reconvolution fitting, or reuse a shift already saved in the metadata. Pre-fitted lifetime values are read directly and do not enter this fitting-based shift search.

It is performed in two steps:

1. Gather the high signal-to-noise ratio (SNR) decay curves
2. For each curve, perform reconvolution fitting and set the shift value as the free parameter to be optimized/fitted. The shift search range is centered on the initial shift guess and extends $\pm 2 \times \text{FWHM}$ of the IRF in both directions, where FWHM is the full width at half maximum of the IRF measured in time bins. This focused search window prevents the optimizer from getting stuck in suboptimal basins when [time gates](#) are restrictive.

The distribution of the shift values is displayed as an interactive scatter plot. When clicking on a point, the corresponding decay curve with the fitted curve and the key statistics are displayed for diagnostics. Based on the distribution or using certain prior knowledge, users can specify the shift value applied to all fields of view (if `Fix the Shift` is selected, see [fitting options](#)), or use the fov-specific shift value found by the fitting (if `Fix the Shift` is deselected).

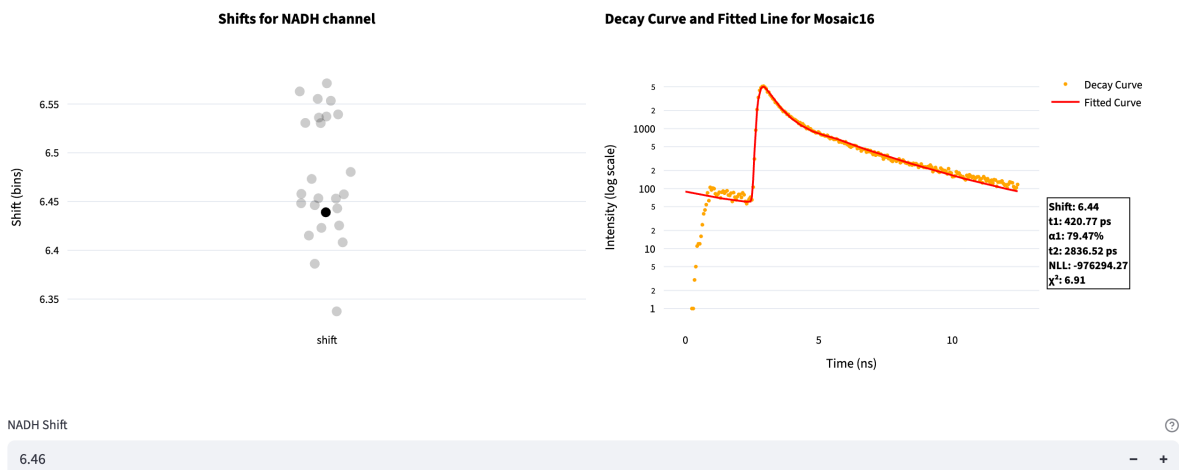


Figure 6.1: Shift calibration using the 3d_decay_1channel example: 25 NADH fields of view, with Mosaic16 selected and the full 0–256 time gate.

6.3.1 Gather High SNR Decay Curves

The high-SNR curves are gathered according to the [decay type](#). For [2D input](#), the selection targets approximately 30 curves, with at least one per FOV (one CSV file). It prefers the brightest curves below 100000 photons; if a FOV has none below that threshold, it uses its brightest curves without the cap. For [3D/4D input](#), one curve per image sums the pixels inside its [ROI mask](#).

6.3.2 Reconvolution Fitting

Fitting is essentially an optimization problem: it minimizes the difference between the fitted curve and the measured curve. The fitted curve is modeled by an n^{th} component exponential function convolved with the shifted IRF, and the difference is modeled as an objective metric. Therefore, users are provided with controls over two parts of the fitting process through the [fitting options panel](#):

1. How to construct the objective metric
2. How to perform the optimization

Implementation-wise, FLIM Playground uses the `lmfit` package that takes generic objectives to perform the optimization process that is flexible enough to handle the reconvolution fitting.

Each decay curve is fitted independently. **Fit calibration** (one optimization per high-SNR curve in the IRF shift step) and **Lifetime fit** ROI-level extraction (one optimization per ROI per channel) both schedule those independent fits **in parallel across CPU cores**, so on a multi-core machine many fits run at once.

The fitting-options panel appears when a channel uses raw `Lifetime fit` and shifts still need estimation.

Metric ?
Fitting Mode ?

MLE ▼
Hybrid ▼

Number of components:

fad

2 - +

nadh

2 - +

fad Start (T1)

0 - +

fad End (T2)

256 - +

> Advanced: fixed lifetimes (fad)

nadh Start (T1)

0 - +

nadh End (T2)

256 - +

> Advanced: fixed lifetimes (nadh)

☒ Fix the Shift ?

Optimize for Shifts

Figure 6.2: Fitting controls combine the objective, optimization mode, and per-channel model settings.

Let's break down the fitting options one by one.

6.3.2.1 Number of Components

The number n in the n^{th} component exponential function:

$$I(\mathbf{t}) = \sum_{i=1}^n A_i e^{-\mathbf{t}/\tau_i}, \quad \tau_i > 0$$

Therefore, n determines the parameters to be fitted: the amplitudes A_i and the lifetimes τ_i . $\mathbf{t}$ is the time axis of the decay curve calculated by the `duration` and `time bins` from the [decay info](#):

$$\mathbf{t} = [0, \Delta t, 2\Delta t, \dots, (N-1)\Delta t], \quad \text{where } \Delta t = \frac{T}{N}.$$

It supports $n = 1, 2, 3$ components.

Additionally, the constant offset parameter (Z) that represents a time-independent background (i.e., room light) is also optimized¹¹.

$$\hat{y}(t) = [I(t) + Z] \circledast \text{IRF}(t)$$

where $\circledast$ denotes circular (periodic) convolution, implemented by multiplying the FFTs of the decay and IRF, then applying the inverse FFT. Circular convolution correctly accounts for the wrap-around effects inherent in periodic TCSPC decays, where photons arriving after the end of the measurement window appear at the beginning of the next period.

i Previous Versions (< 1.8.0)

In versions prior to 1.8.0, the fitting process utilized linear convolution with a cut off:

$$\hat{y}(t) = (I(t) * \text{IRF}(t))_{0:N} + Z$$

where $*$ denotes linear convolution and the subscript $(0 : N)$ denotes truncating the result to the length of the original decay curve, effectively ignoring the part of the convolution that extends beyond the measurement window. The difference between this approach and circular convolution is minimal for shorter lifetimes. However, the correct way to fit incomplete decays is formulated above using circular convolution, which is now the standard used in the latest version.

6.3.2.2 Fixed Lifetimes

While finding the optimal amplitudes A_i and lifetimes τ_i (when [number of components](#) is greater than 1), users can optionally assign fixed values to specific τ_i components during the interactive fitting phase. These explicit constraints remain constant (are not treated as free parameters) while minimizing the cost metric, making the optimization more robust when the true lifetime state is known. The default fixed lifetime values can be initialized in the [Data Extraction Configuration](#).

i Note

Fixed lifetimes apply only to channels that FLIM Playground fits. For [pre-fit](#) inputs (SPCImage .asc), whose lifetimes are read rather than fit, the fixed-lifetime controls are not shown.

6.3.2.3 Time Gates

Detector dead time can make some bins at the beginning or end of a decay unreliable. Set **Start (T1)** and **End (T2)** independently for each fitted channel to choose the bins included in its fitting objective. T1 is included and T2 is excluded: the default 0–256 uses bins 0–255 of a 256-bin decay.

To adjust the gates in the `3d_decay_1channel` example:

1. Select a point in the [shift distribution](#) to inspect its measured and fitted decay. Expand the decay plot if needed, then hover over measured points to read their bin numbers.
2. Change **NADH Start (T1)** from 0 to 20 and **NADH End (T2)** from 256 to 240. This example fits bins 20–239, trimming both ends while retaining the decay peak. Choose the limits from the decay in your own dataset.
3. Click **Optimize for Shifts** and select `Mosaic16` again to inspect the updated fit. The plot still displays the full decay; only bins inside the gates contribute to the fitting objective and reported fit statistics.
4. When satisfied, click **Confirm Time Gates (if applicable) and Shift for each channel**. Use **Download updated metadata** after confirmation to retain these settings for later extraction.

The screenshot shows a software interface for fitting NADH data. At the top, there are two dropdown menus: 'Metric' set to 'MLE' and 'Fitting Mode' set to 'Hybrid'. Below these, the 'Number of components' is set to 2. The 'NADH' section shows 'NADH Start (T1)' set to 20 and 'NADH End (T2)' set to 240. A button labeled 'Advanced: fixed lifetimes (NADH)' is visible. At the bottom, there is a checkbox labeled 'Fix the Shift' which is checked, and a button labeled 'Optimize for Shifts'.

Figure 6.3: Change the NADH gates to $T1 = 20$ and $T2 = 240$, then optimize the shifts.

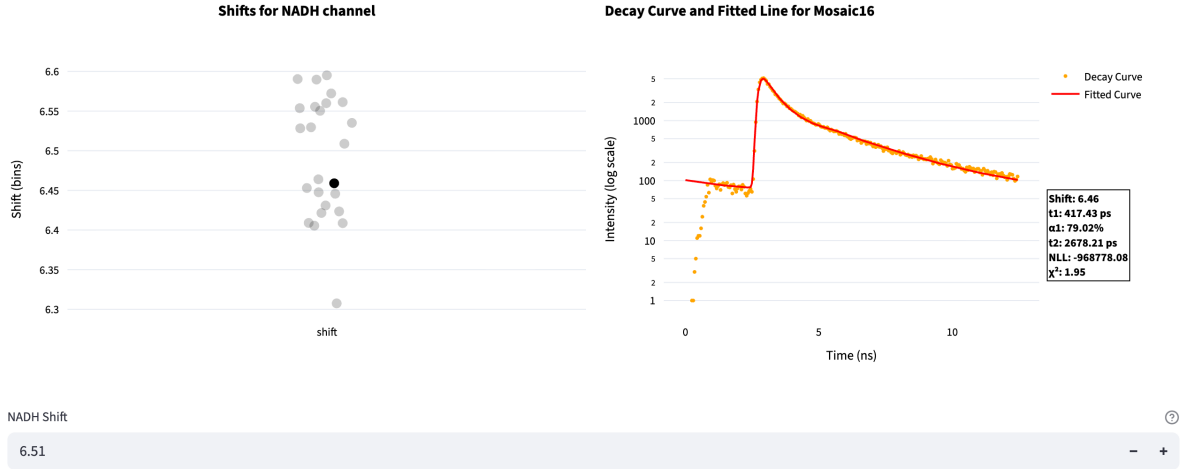


Figure 6.4: The same Mosaic16 decay after fitting with the 20–240 time gate. The full curve remains visible for inspection.

6.3.2.4 Metric

Maximum Likelihood Estimation (MLE) estimates the parameters by maximizing the likelihood function, which is the probability of the measured data given the model parameters. A mathematically convenient way to do this is to minimize the negative log-likelihood function:

$$\text{NLL}(\theta; t_s, t_e) = - \sum_{n=0}^{N-1} \mathbf{1}_{[t_s, t_e)}(t_n) [y_n \log \hat{y}(t_n) - \hat{y}(t_n)].$$

$\mathbf{1}_{[t_s, t_e)}$ is the indicator function that is 1 if t_n is within the [time gates](#) $[t_s, t_e)$, and 0 otherwise. y_n is the observed count at time t_n , and $\hat{y}(t_n)$ is the model prediction at t_n .

Weighted Least Squares (WLS) minimizes the squared residuals between the measured curve and the model prediction, with each time bin weighted by the inverse of its expected variance. Because photon counts follow Poisson statistics (variance $\approx$ mean), the per-bin variance is estimated by the model prediction $\hat{y}(t_n)$, so each residual is scaled by $1/\sqrt{\max(\hat{y}(t_n), 1)}$, and the cost minimized is

$$\text{WLS}(\theta; t_s, t_e) = \sum_{n=0}^{N-1} \mathbf{1}_{[t_s, t_e)}(t_n) \frac{(y_n - \hat{y}(t_n))^2}{\max(\hat{y}(t_n), 1)}.$$

This weighting is Pearson's χ^2 — the per-bin variance is taken from the model $\hat{y}(t_n)$ rather than the noisy measured count — which keeps the high-count bins near the decay peak from dominating the fit; the $\max(\cdot, 1)$ floor avoids division by zero where the model is near zero. The reported [reduced](#)

[chi-square](#) also uses model-based variance, but divides by positive model values without the WLS floor and adjusts for the number of free parameters.

6.3.2.5 Fitting Mode

The fitting mode selects the optimization strategy. It is a trade-off between speed and the effort to avoid local minima.

During shift estimation, `Fitting Mode` offers **Hybrid** only. After all required shifts have been confirmed or loaded from saved metadata, the fitting workflow offers both **Hybrid** and **Local** for the subsequent cell-level fits. Choose the mode before clicking `Confirm` and `Start`.

- **Local**: Each cell is fit with a single local optimizer — `leastsq` (least squares with Levenberg-Marquardt) if the chosen metric is WLS, otherwise `nelder` (Nelder-Mead). To give those per-cell fits a good starting point, a one-time **warm start** is performed first: the differential evolution global optimizer is run once on the *mean* decay of all ROIs in the batch, and the resulting lifetimes (and the constant offset) seed the initial values for every per-cell fit. The mean is used rather than the sum so the seed stays at single-cell scale, keeping a cell's result independent of how many cells share its batch. This warm start runs once, up front, before the per-cell fits are [parallelized across CPU cores](#); because the global search runs once for the whole batch instead of once per cell, `Local` is substantially faster than `Hybrid`.
- **Hybrid**: The slower but more accurate option. For **each cell**, it runs the differential evolution algorithm to obtain a good initial guess for all parameters, followed by the `Local` optimizer to refine the solution.

6.4 Fit-Free IRF Calibration

This shift calibration applies to channels with `Lifetime fit free` selected and [calibration method](#) set to IRF, including pre-fitted FLIM channels when phasor features are requested. The [fluorescence lifetime standard](#) method instead calibrates phasors during extraction.

Before calculating phasors, the app subtracts the mean signal in the final 10% of each cell's decay and clips negative values to zero. See [fit-free preprocessing](#).

6.4.1 Shift IRF

It is performed similarly to the [fit calibration](#) steps, but in the second step, the shift is not optimized (fitted). It shares the [same interface](#) as the [fit calibration](#) step, where a scatter plot of the shift values is displayed for each channel, only that the plot is not interactive to show the fit. Instead, it outputs the shift that maximizes the cross-correlation between the IRF and each decay. If `Fix the Shift` is selected, users can specify the shift value that will be applied to all fields of view. With `Fix the`

Shift cleared, image inputs use their individual FOV shifts; 2D inputs use the median sampled shift within each FOV.

i Note

If both `Lifetime fit` and `Lifetime fit free` are selected for a **raw FLIM channel**, the optimized IRF shift from [fit calibration](#) is reused for IRF-based phasor extraction. For pre-fitted FLIM, the existing fitted values do not trigger reconvolution fitting; IRF-based phasor extraction uses the cross-correlation shift described above.

The signals of the shifted IRF are deconvolved from the offset-subtracted decay curves using `phasor.phasor_divide` from the `phasorpy` package.

6.5 Confirm Calibration

Once users are satisfied with the calibration settings ([fitting options](#) and shift values), they can click the Confirm Time Gates (if applicable) and Shift for each channel button at the bottom of the [page](#).

After confirmation, choose the fitting mode for raw FLIM channels using `Lifetime fit`, then click Confirm and Start, or use Go back and find shift to repeat calibration. Download updated metadata preserves the shifts, time gates, fitting options, component counts, and fixed lifetimes. For metadata exported in the current session, this updates its existing file; for uploaded metadata, it downloads a new CSV. Loading that calibrated CSV later lets you reuse its shifts.

✔ Features to be extracted confirmed. 🎉

Fitting Mode ?

Local ▼

Confirm and Start

Go back and find shift

Download updated metadata

Figure 6.5: After calibration, choose the fitting mode and start extraction or save updated metadata.

7 Lifetime Features (Fit)

Biological samples often contain mixtures of fluorescent species, each with its own characteristic lifetime. By modeling the decay as a sum of exponentials, the analysis can separate and quantify these different contributions. For example, NADH in cells exists in both free (short lifetime, ~0.4 ns) and protein-bound (longer lifetime, ~2–3 ns) states; by fitting the fluorescence decay with a bi-exponential model, one can estimate the fraction of each state, which provides direct insight into cellular metabolism and energy production pathways.

FLIM Playground extracts cell-level lifetime fitting features in this feature extractor, including the fraction of each lifetime component (α_i), their lifetimes (τ_i), mean lifetime (τ_{mean}), intensity-weighted mean lifetime ($\tau_{m,iw}$), and the reduced chi-square (χ_r^2) goodness-of-fit metric. (See [here](#) for the definition of the equation below)

$$I(t) = \sum_{i=1}^n A_i e^{-t/\tau_i}$$

7.1 Fitting

If the decays have not been pre-fit, FLIM Playground applies the confirmed fitting options ([number of components](#), [fixed lifetimes](#), [time gates](#), [metric](#), and [fitting mode](#)) to the [reconvolution fitting process](#). The IRF shift is fixed at the confirmed value for the channel or FOV. Shift calibration uses **Hybrid**; subsequent cell-level fits can use **Hybrid** or **Local**, selected after calibration or when reusing saved metadata. These per-cell fits are [parallelized across CPU cores](#). Reconvolution fitting minimizes the selected cost metric between the measured decay and a multi-exponential model convolved with the IRF.

[Otherwise](#), FLIM Playground uses the [pre-fit values](#) to calculate the fitting features.

In the final dataset, the primary fitting features are prefixed by the combination of the feature extractor name (i.e. `Lifetime fit`) and the channel name, allowing [Data Analysis](#) to group the features. For example, `Lifetime fit_nadh: a1` means the fraction of the first lifetime component for the NAD(P)H channel.

Not every column produced by this extractor carries that prefix. The lifetimes (τ_i , τ_{mean} , $\tau_{m,iw}$) and the leading fractions do, but the raw amplitudes (A_i), the constant offset, the reduced chi-square, and the final fraction (which is derived as the remainder of the others) are written with only the channel

name, e.g. `nadh_reduced_chi_square`. These are treated as bookkeeping values, so Data Analysis places them in the **Uncategorized Features** group rather than under `Lifetime fit_nadh`.

! Units in the exported dataset

The extracted CSV mixes three units, so check the column before using a value in a formula:

- **Lifetimes** (`t1`, `t2`, `t3`, `tm`, `tm_iw`) are exported in **picoseconds**, even though this chapter writes the formulas in nanoseconds.
- **Fractions** (`a1`, `a2`, `a3`) are exported as **percentages** in the range 0–100, not as the 0–1 fraction α_i used in the formulas below. Divide by 100 before substituting them into τ_{mean} or $\tau_{m,iw}$.
- Phasor-derived lifetimes from the [Lifetime fit free](#) extractor (`Tau_phase`, `Tau_mod`) are in **nanoseconds**.

7.1.1 Preprocessing

It sums up all the pixel decays belonging to the same cell ROI labeled by the ROI [mask](#) as one decay curve⁹. The ROI summing reduces variability and bias and allows for short integration times at acquisition in exchange for sub-cellular resolution (i.e., pixel-level). Users do not need to specify the bin factor (each pixel sums up the surrounding pixels' decay curves) to account for insufficient photon counts. ROI-level aggregation also keeps the workload manageable; independent ROI fits are then [run in parallel across CPU cores](#).

It also shifts the IRF using the shift values from the [IRF shift calibration](#) step. To shift an IRF, FLIM Playground upsamples the IRF 10 times using linear interpolation to fill the gaps. Then it shifts the IRF by the shift values $\times 10$ and downsamples the IRF back to the original size.

7.1.2 Fraction of components

In addition to the absolute amplitudes of each component directly from the fitting result, FLIM Playground calculates the fraction of each component, α_i , as the amplitude of the component divided by the sum of all amplitudes. This normalization allows lifetime to be independent of the absolute intensity of the signal.

α_i is a value between 0 and 1, and it is used in that form throughout the formulas in this chapter. In the exported dataset, however, the corresponding `a1/a2/a3` columns are stored as **percentages** ($\alpha_i \times 100$, ranging from 0 to 100). The final fraction is not fitted independently; it is derived as the remainder that makes the fractions sum to 100.

7.1.3 Mean lifetime

The mean lifetime, τ_{mean} , is calculated as the weighted average of the lifetimes of all components, where the weights are the fractions of each component.

$$\tau_{mean} = \sum_{i=1}^n \alpha_i \tau_i$$

7.1.4 Intensity-weighted mean lifetime

The [mean lifetime](#) above is amplitude-weighted: each component is weighted by its fraction α_i . The intensity-weighted mean lifetime, $\tau_{m,iw}$, instead weights each component by its relative intensity contribution ($\alpha_i \tau_i$), so longer-lived components — which emit more photons per molecule — count more.

$$\tau_{m,iw} = \frac{\sum_{i=1}^n \alpha_i \tau_i^2}{\sum_{i=1}^n \alpha_i \tau_i} = \frac{\sum_{i=1}^n \alpha_i \tau_i^2}{\tau_{mean}}$$

For a single component, $\tau_{m,iw} = \tau_1$.

Both fitting and the [pixel pre-fit](#) workflow export τ_{mean} (`tm`) and $\tau_{m,iw}$ (`tm_iw`) only for two or three components. For a single component, both mean lifetimes equal τ_1 : use the exported `t1` column, as no separate `tm` or `tm_iw` columns are written.

7.1.5 Goodness of fit

For each fitted decay, FLIM Playground also reports the reduced chi-square (χ_r^2) as a goodness-of-fit diagnostic:

$$\chi_r^2 = \frac{1}{N - p} \sum_n \frac{(y_n - \hat{y}_n)^2}{\hat{y}_n}$$

where the sum runs over the time bins within the [time gates](#) that have a positive model value ($\hat{y}_n > 0$), y_n is the measured count, $\hat{y}_n$ is the fitted value, N is the number of those bins, and p is the number of free parameters (amplitudes, lifetimes, and the offset, minus any [fixed lifetimes](#)). Values near 1 indicate a good fit.

7.2 Pixel pre-fit

Currently, FLIM Playground supports pixel pre-fit lifetime features from [SPCImage](#). The pixel-level lifetime fitting features are expected to be stored in [2D arrays](#) in spatial dimensions, with each row and column having the value of a lifetime feature outputted from SPCImage. SPCImage assigns 0 for pixels that are not fitted (e.g. thresholded out). To avoid them biasing the results, FLIM Playground uses `np.ma.masked_array` to create a masked array and disregards them when calculating the averages using `np.ma.average`.

Then it uses the ROI [mask](#) to calculate the cell-level lifetime features by averaging the pixel-level features within each ROI.

8 Lifetime Features (Fit-Free)

Phasor analysis provides a fast, model-free view of lifetimes by transforming fluorescence decays into phasor coordinates through a Fourier transform. This approach is considered “fit-free” because it does not require iterative curve fitting or predefined decay models—each decay is mapped directly to a point on the [phasor plot](#). It has the nice property that mono-exponential decays fall on the universal semicircle while mixtures lie inside as linear combinations, so clusters reveal distinct species and their fractional contributions. FLIM Playground currently extracts cell-level phasor features: `G(1st)`, `S(1st)`, `Tau_phase`, `Tau_mod`, `G(2nd)`, and `S(2nd)`. (Datasets extracted before this feature was renamed use `Tau_m`, which Data Analysis still recognizes.)

Fit-free features in the final dataset are prefixed by the combination of the feature extractor name (i.e. `Lifetime fit free`) and the channel name, allowing [Data Analysis](#) to group the features. For example, `Lifetime fit free_nadh: G(1st)` means the *g* coordinate for this cell in the NAD(P)H channel.

8.1 Preprocessing

For image inputs, the app sums the pixel decays within each cell ROI labeled by the ROI [mask](#) to obtain one cell decay⁹. For [2D decay inputs](#), each CSV row already supplies a cell decay, so no mask is needed. Phasor features are calculated from each cell decay.

Two preprocessing steps are then applied, and they are independent of each other:

- **Offset subtraction** is applied to every decay curve, regardless of the chosen [calibration method](#). For each cell, the offset is estimated as the mean of the last 10% of the time bins of that cell’s own decay curve, subtracted from the curve, and any resulting negative values are clipped to zero. Note that this per-cell baseline is unrelated to the `offset` parameter fitted during [lifetime fitting](#).
- **IRF shifting** is applied only when the chosen calibration method is IRF shift calibration. The IRF is shifted using the shift values as described in the [Shift IRF step](#). Under fluorescence lifetime standard calibration, no IRF shift is applied.

8.2 Calibration method

In [Home → Configuration](#), choose **IRF** or **Fluorescence Lifetime Standard** for `Lifetime fit free` extraction. This choice determines the reference and the correction applied in [Step 2](#). Both methods use the same [offset subtraction](#) for each cell decay.

8.2.1 IRF

Provide an IRF for each channel that uses this method, then complete [fit-free IRF calibration](#). The app shifts the IRF using the selected shift values and calculates its first- and second-harmonic phasors. It divides each cell's raw phasor by the shifted IRF phasor at the same harmonic.

8.2.2 Fluorescence lifetime standard

Provide a calibration dye with a known lifetime, enter that lifetime in **nanoseconds**, and set each channel's standard-file suffix. This method does not apply an IRF shift to the phasor calculation.

The app calculates the standard's measured phasor separately at each harmonic. PhasorPy's [lifetime.phasor_calibrate](#) uses those measurements and the known lifetime to correct each cell's phasor at the matching harmonic.

Selecting raw `Lifetime fit` as well still requires [IRF shift calibration for fitting](#); the standard only calibrates the phasor features.

8.3 Phasor features

The steps below describe **default phases**, followed by IRF or lifetime-standard calibration and lifetime conversion. FLIM Playground uses this workflow when the supplied time window matches one laser period. Its explicit-phase calculation for other windows is described in the collapsible note at the end.

8.3.1 Step 1: raw phasor coordinates

8.3.1.1 Default phases

Let T be the supplied window's duration in ns and K its number of bins. PhasorPy's [phasor_from_signal](#) assigns phases $2\pi hk/K$, where h is the harmonic and $k = 0, \dots, K - 1$ is the bin index. **The raw transform uses bin positions and count; neither duration nor laser_rate is passed to it.** The stored bins and photon counts are unchanged.

For bin times $t_k = kT/K$, these default phases correspond to an **analysis frequency of $1/T$** . In the matching-window case described here, that equals the laser frequency (`laser_rate`, in GHz), which is used in the calibration and lifetime equations below. For a decay F_k :

$$g_{\text{raw}}^{(h)} = \frac{\sum_{k=0}^{K-1} F_k \cos(2\pi h k / K)}{\sum_{k=0}^{K-1} F_k},$$

$$s_{\text{raw}}^{(h)} = \frac{\sum_{k=0}^{K-1} F_k \sin(2\pi h k / K)}{\sum_{k=0}^{K-1} F_k}.$$

In this default-phase workflow, the app calls `phasor_from_signal` with `harmonic=1` and `harmonic=2` for each cell decay to obtain `g_raw`, `s_raw`, `g_raw_2nd`, and `s_raw_2nd`.

With **IRF calibration**, it makes the same two calls for the shifted IRF, obtaining `g_irf`, `s_irf`, `g_irf_2nd`, and `s_irf_2nd`.

With **fluorescence lifetime standard calibration**, it calculates the reference values separately for each harmonic as shown below. `reference_image` is the loaded standard decay array, and `reference_time_axis` is its zero-based time-bin axis (for example, 2 for an array shaped (height, width, time_bins)).

```
from phasorpy import phasor

ref_mean, ref_real, ref_imag = phasor.phasor_from_signal(
    reference_image, axis=reference_time_axis, harmonic=1,
)
ref_mean_2nd, ref_real_2nd, ref_imag_2nd = phasor.phasor_from_signal(
    reference_image, axis=reference_time_axis, harmonic=2,
)
```

8.3.2 Step 2: calibrated phasor coordinates

Apply the selected calibration using the reference phasor at the matching harmonic.

8.3.2.1 IRF calibration

FLIM Playground divides each raw cell phasor by the shifted IRF phasor at the same harmonic to obtain `g`, `s`, `g_2nd`, and `s_2nd`:

```

from phasorpy import phasor
g, s = phasor.phasor_divide(g_raw, s_raw, g_irf, s_irf)
g_2nd, s_2nd = phasor.phasor_divide(
    g_raw_2nd, s_raw_2nd, g_irf_2nd, s_irf_2nd
)

```

8.3.2.2 Fluorescence lifetime standard calibration

FLIM Playground uses the known lifetime and the standard's measured reference values for each harmonic calculated in [Step 1](#):

```

from phasorpy import lifetime

g, s = lifetime.phasor_calibrate(
    g_raw, s_raw, ref_mean, ref_real, ref_imag,
    frequency=laser_rate * 1000,
    lifetime=reference_dye_lifetime,
)
g_2nd, s_2nd = lifetime.phasor_calibrate(
    g_raw_2nd, s_raw_2nd,
    ref_mean_2nd, ref_real_2nd, ref_imag_2nd,
    frequency=laser_rate * 1000,
    lifetime=reference_dye_lifetime,
    harmonic=2,
)

```

The `laser_rate` is entered in **GHz** in [FOV metadata](#) or [Configuration](#). PhasorPy expects **MHz**, so both calls multiply by 1000: 0.08 GHz becomes 80 MHz. The `reference_dye_lifetime` remains in **nanoseconds**. Using `harmonic=2` calibrates the second-harmonic coordinates at twice the laser frequency.

8.3.3 Step 3: phasor-derived lifetime

`Tau_phase` and `Tau_mod` are calculated using the first harmonic's phasor coordinates `g` and `s`.

```

import numpy as np
w = 2 * np.pi * laser_rate
phi = np.arctan2(s, g)
mod = np.sqrt(g**2 + s**2)
tau_phase = 1/w * np.tan(phi)
tau_mod = 1/w * np.sqrt(1/mod**2 - 1)

```

$$\phi = \text{atan2}(s, g), \text{mod} = \sqrt{g^2 + s^2}, \tau_\phi = \frac{\tan \phi}{\omega}, \tau_{\text{mod}} = \frac{1}{\omega} \sqrt{\frac{1}{\text{mod}^2} - 1}$$

Because ω is derived from the laser rate in GHz, `Tau_phase` and `Tau_mod` are expressed in **nanoseconds**. Note that this differs from the [Lifetime fit](#) extractor, whose lifetime columns are exported in picoseconds.

i Explicit sample phases

FLIM Playground uses explicit phases when the supplied time window differs from one laser period by more than a small rounding tolerance. It then calculates phasors at the **laser frequency**, using both `duration` and `laser_rate` to place each bin within the laser cycle. A shortened acquisition window or a small difference caused by PTU bin rounding can select this branch; it is not a test for whether the acquisition was complete.

For example, a 10 ns window with a 12.5 ns laser period covers 80% of a laser cycle. Let f be `laser_rate`, $P = 1/f$ the laser period, and T the supplied window's duration. The bin times are $t_k = kT/K$, with first-harmonic phases $\phi_k = 2\pi t_k/P = 2\pi f t_k$. At harmonic h :

$$g_{\text{raw}}^{(h)} = \frac{\sum_{k=0}^{K-1} F_k \cos(h\phi_k)}{\sum_{k=0}^{K-1} F_k},$$

$$s_{\text{raw}}^{(h)} = \frac{\sum_{k=0}^{K-1} F_k \sin(h\phi_k)}{\sum_{k=0}^{K-1} F_k}.$$

For **cell decays** and the **shifted IRF**, the app evaluates these sums directly. For a **fluorescence lifetime standard**, it supplies the same phases through PhasorPy's `sample_phase` argument. The standard must have the same bin count and spacing as the sample:

```

import numpy as np
from phasorpy import phasor

time_bins = reference_image.shape[reference_time_axis]
time_axis = np.arange(time_bins) * (duration / time_bins) # ns
phi = 2 * np.pi * laser_rate * time_axis

ref_mean, ref_real, ref_imag = phasor.phasor_from_signal(
    reference_image, axis=reference_time_axis,
    sample_phase=phi, harmonic=1, use_fft=False,
)
ref_mean_2nd, ref_real_2nd, ref_imag_2nd = phasor.phasor_from_signal(
    reference_image, axis=reference_time_axis,
    sample_phase=2 * phi, harmonic=1, use_fft=False,
)

```

Use `harmonic=1` and `use_fft=False` with `sample_phase`. The doubled phases select the second harmonic; `harmonic=2` with `sample_phase` would raise an error. The resulting coordinates enter the calibration and lifetime calculations in Steps 2 and 3.

Explicit phases preserve bin timing but do not recover missing fluorescence. For a substantially truncated window, the usual calibration and lifetime equations can remain biased; a standard measured over the same window does not generally remove that bias.

Using default phases over the 10 ns window would instead mean analysis at **100 MHz**, requiring that frequency in calibration and lifetime conversion. This is a different approach with its own limitations for real IRFs and mixtures. Release 1.13.1 retains the **80 MHz laser frequency** in this example and uses explicit phases. See the [analysis of truncation and phasor calibration](#) for these distinctions.

9 Morphological Features

FLIM Playground extracts single-cell morphological features based on the cell region of interest (ROI) mask of each *channel*. By inputting the ROI mask to `skimage.measure.regionprops`, the following features are extracted:

- **area**: the number of pixels in the ROI.
- **perimeter**: length of the ROI boundary in pixels.
- **solidity**: $\text{Solidity} = \frac{\# \text{ROI pixels}}{\# \text{convex hull pixels}}$, where the convex hull is the smallest convex polygon that contains the ROI. A value close to 1 means the shape has few concavities (more solid), while lower values indicate more irregular boundaries.
- **eccentricity**: equals the eccentricity of the ellipse that has the same second moments as the region. It is given by the ratio of the focal distance (the distance between the foci) to the length of the major axis: $e = \frac{c}{a}$, $e \in [0, 1)$, where a is the semi-major axis length, b is the semi-minor axis length, and $c = \sqrt{a^2 - b^2}$ is the focal distance. When $e = 0$, the ellipse reduces to a circle. It quantifies how elongated the ROI is, with 0 being a perfect circle and 1 being a line.
- **major_axis_length**: the length of the major axis of the ellipse that has the same second moments as the region: $2a$. It represents the longest dimension of the ellipse that approximates the region's shape, essentially describing its maximum elongation.
- **minor_axis_length**: the length of the minor axis of the ellipse that has the same second moments as the region: $2b$. It represents the shortest dimension of that ellipse, capturing the region's minimum elongation.
- **circularity**: describes how close the ROI is to a perfect circle. It is calculated as $\text{circularity} = \frac{4\pi A_{\text{ROI}}}{P_{\text{ROI}}^2}$, where A_{ROI} is the area of the region and P_{ROI} is its perimeter. In continuous geometry, values lie in $(0, 1]$, with 1 for a perfect circle. The pixel-based perimeter estimate used here can produce values above 1, and exported values are not clipped. If the estimated perimeter is zero, the app reports 0.

Feature names in the final dataset are prefixed by the combination of the feature extractor name (i.e. Intensity morphology) and the channel name, allowing [Data Analysis](#) to group the features. For example, `Intensity morphology_nadh: area` means the area of each cell in the NAD(P)H channel.

The extractor also exports `<channel>_centroid_x` and `<channel>_centroid_y` as uncategorized numerical columns. If several channels request morphology from the same mask file, morphology is exported once under the first such channel.

10 Textural Features

FLIM Playground extracts single-cell texture features based on the intensity image and the ROI mask from each acquisition *channel*. For an example application, refer to this [paper](#) that used CellProfiler to extract the texture features.

The first step is to create a single-cell intensity image for each cell based on the ROI mask and the intensity image. Then a list of texture features (the list may be extended in the future) is calculated:

10.1 Granularity

$Granularity_n$ is the percentage of intensity contributed by bright objects of n pixels in radius. It is computed by performing a morphological opening with a disk of radius n on the cell intensity image and measuring the intensity lost as a percentage of the total intensity of the original cell ROI.

$n = 1, 3, 5, 7$ are computed.

Semantically, higher granularity at smaller scales implies higher fragmentation.

10.2 Radial Distribution

To calculate the radial distribution, each cell intensity image is partitioned into 4 concentric rings, and the intensity fraction over the total intensity for each ring is calculated. For example, a higher fraction in ring 1 implies that signals are more concentrated at the center of the cell, whereas a higher fraction in ring 4 implies that signals are more concentrated at the cell edge.

10.3 Mass Displacement

Mass displacement is the Euclidean distance (in pixels) between the geometric and intensity-weighted centroids, both calculated using pixels with intensity greater than zero within the cell ROI. If the ROI has no such pixels, the result is a missing value (NaN).

10.4 Intensity Sum

Intensity sum is defined as the sum of pixel intensities within the cell intensity image.

Note

For a [2D decay](#) channel there is no intensity image or ROI mask, so [Granularity](#), [Radial Distribution](#), and [Mass Displacement](#) cannot be computed. Selecting Intensity Texture for such a channel therefore yields only `intensity_sum`, defined here as the sum of the cell's decay curve (its total photon count across all time bins). The output column is `Intensity texture_{channel}: intensity_sum`.

In the final dataset, feature names are prefixed by the combination of the feature extractor name (i.e. `Intensity texture`) and the channel name, allowing [Data Analysis](#) to group the features. For example, `Intensity texture_nadh: granularity_1` means the percentage of intensity removed when bright objects of 1-pixel radius are removed in the NAD(P)H channel.

Visual explanation

[This post](#) provides a visual explanation of and code to calculate some of the texture features above.

11 Derived Features

FLIM Playground lets users compose new **derived features** by combining existing features, potentially from *different channels*. An example would be the normalized optical redox ratio, composed from the `intensity_sum` feature from each of the two channels: $\text{NAD(P)H} / (\text{NAD(P)H} + \text{FAD})$.

11.1 Build Derived Features

The builder is revealed when the `Derived features` checkbox near the bottom of the [Configuration](#) page is ticked.

The screenshot shows the 'Derived features' builder interface. At the top, a checkbox labeled 'Derived features' is checked. Below it, a list of existing derived features is shown, with 'redox_ratio' selected. The builder form is divided into several sections: 'Template' (set to 'Custom...'), 'Name' (set to 'background_corrected_nadh'), 'Operands' (a list containing 'intensity_texture_nadh' and 'nadh_offset'), and 'Formula' (containing 'A - B*256'). Below the formula, a green status bar indicates the derived feature is successfully created: '✓ Derived: background_corrected_nadh = Intensity_texture_nadh - intensity_sum - nadh_offset*256'. At the bottom, there is a button labeled '+ Add derived feature'.

Figure 11.1: The builder previews a custom formula using the selected operands before adding it.

Any derived features already defined for the current profile are listed at the top, each showing its name, its expanded formula, and a button to delete it. Below the list, a new feature is composed from:

- a **Template** (a common formula, or `Custom...` to write your own),
- a **Name** (e.g. `redox_ratio`) — the output column becomes `Derived: <Name>`, and
- the **operands** that fill the formula.

Not finding the features you expect?

The operands/features available for selection should span all channels. If no channel has a [feature extractor](#) selected yet, the builder shows “Select feature extractors for your channels first — there are no features to derive from yet.” — configure the extractors first, then come back.

The categorized numerical features come first. The raw bookkeeping columns — `amp1`, `offset`, `reduced_chi_square`, the centroid coordinates, and the `a2 / a3` fit remainders — carry no extractor prefix, so they are listed **after** them, at the bottom of each operand dropdown. They are ordinary per-cell numbers and can be used like any other operand: a background-corrected intensity, for instance, is `Intensity texture_<channel>: intensity_sum minus <channel>_offset times the number of time bins` (typed as a plain number in Custom... mode, e.g. `A - B*256`).

A pixel-prefitted (SPCImage) channel does no fit of its own, so it contributes only its `a2 / a3` remainder — no `amp`, `offset` or `reduced_chi_square`.

Derived features are stored **per profile**, so each [extraction profile](#) keeps its own set.

When the preview is valid, click `Add derived feature`. Adding and deleting definitions saves them immediately to the active profile.

11.1.1 Templates

Choosing a Template other than Custom... fills in the formula for you and renders one searchable dropdown per operand slot (A, B, ...). Four ready-made templates are provided: normalized $A / (A + B)$, ratio A / B , difference $A - B$, and sum $A + B$.

Each takes two operands: pick a feature for A and a feature for B, give the result a Name, and the live preview confirms it. The fifth option, Custom..., unlocks the free formula editor described next.

11.1.2 Custom Formulas

Custom... mode lets you compose an arbitrary formula. First pick the operands from the Operands (each becomes A, B, C... in the order picked) multiselect — the features you choose are aliased A, B, C, ... in pick order, and a caption echoes the mapping (e.g. `A = Intensity texture_fad: intensity_sum`, `B = ...`). Then build the formula in the Formula box, typing directly or using the append buttons for the operators (+ - * / ()) and for each alias.

The formula language supports:

- the four arithmetic operators +, -, *, /,
- a leading sign (unary + / -),
- numeric constants, and
- parentheses for grouping.

Division by zero yields NaN rather than raising an error or producing infinity. Anything outside this language — function calls, exponentiation (**), modulo (%), comparisons, or a bare operand with no operator — is rejected.

11.2 Compute Derived Features

Derived features are computed during [Numerical Feature Extraction](#), after the successful FOV results have been concatenated. The definitions are baked into the FOV metadata, so re-running extraction from a saved metadata file reproduces the same derived columns. Each definition adds one `Derived: <name>` column to the output dataset.

If any cell ends up with a NaN in a derived column (for instance from a divide-by-zero or a NaN operand), a single dataset-wide warning is shown for that feature — “Derived feature **name** has **N** NaN value(s) out of *M* cell(s) (e.g. from divide-by-zero or a NaN operand).” As with other features, cells with NaN are **not** dropped from the output.

11.3 Derived Feature In Data Analysis

On the [Data Analysis](#) page, all derived features are collected into one `Derived Features` group in the [numerical selection widgets](#), alongside the per channel-extractor groups. A derived feature is labeled by its plain name (e.g. `redox_ratio`); unlike extracted lifetime or intensity features, it carries no unit or symbol, since its meaning is entirely user-defined.

12 Categorical Feature Extraction

Based on the datasets extracted in the [numerical feature extraction](#) step, FLIM Playground:

- [merges the datasets](#) into a single dataset
- [assigns categorical features](#) to each cell in the merged dataset

Deploy

HomeData ExtractionData Analysis

v1.13.0-7-gf9e489a-dirty

Data Extraction

Select a step to perform

☐ FOV Metadata Extraction

☐ Numeric Feature Extraction (fitting, phasor, etc.)

☒ Categorical Feature Extraction (e.g. treatment)

Copy the folder path here

/private/tmp/fp-docs-resume-20260909/merge-example

Field of View Name Delimiter

-

Found 1 available csv files ready to be assigned categories

```
[
  {
    "image_name": "/private/tmp/fp-docs-resume-20260909/merge-example/single_cell_features.csv"
  }
]
```

Merging datasets...

Finished combining all datasets and got 1572 cells!

Now your task is to map the categories to (combination of) slots.

Example fov_name: MCF7_2DG_dish2_04 has slots: ['MCF7', '2DG', 'dish2', '04']

Choose Categorical features (specified in Configuration @ Home tab) to populate

cell_line ×

treatment ×

dish ×

Slots for cell_line

Slots for treatment

Slots for dish

MCF7 ×

2DG ×

dish2 ×

Preview of category mapping:

	image_name	cell_line	treatment	dish
0	MCF7_2DG_dish2_04	MCF7	2DG	dish2
185	MCF7_2DG_dish1_05	MCF7	2DG	dish1
300	MCF7_2DG_dish1_01	MCF7	2DG	dish1
410	MCF7_2DG_dish2_03	MCF7	2DG	dish2
515	MCF7_2DG_dish1_06	MCF7	2DG	dish1

Confirm category mapping & export the combined dataset

Figure 12.1: This category-mapping example uses a table of 1,572 cells from 12 FOVs.

12.1 Merging Datasets

Enter the full path to the folder containing your saved [numerical feature CSV files](#). The folder is scanned recursively for CSV files. Each usable dataset must contain the configured [cell and FOV identifiers](#). Cell identifiers must be nonmissing and unique within the file and across the accepted files; a file containing overlapping identifiers is skipped.

! Important

The FOV identifier uniquely identifies a field of view and is assumed to contain all information necessary to extract the categorical features (e.g., treatment, time point, etc.), by including segments separated by a certain delimiter (e.g., _). Categorical features are assigned from individual segments or their combinations. Example: The FOV Panc1_Cyanide_dish1 denotes that it comes from the Panc1 cell line, under the Cyanide treatment, in the 1st dish.

Therefore, it checks if all field-of-view names include the same number of segments to determine the eligibility of [categorical feature assignment](#).

Before merging, all-empty columns are removed. The remaining datasets are stacked using only their common columns, and the page reports columns discarded during merging.

12.2 Assigning Categorical Features

Available categories are the [configured categorical names](#) that are not already columns in the merged dataset.

Set Field of View Name Delimiter to the separator used in the FOV names. Leaving it empty treats each complete name as one slot. For each chosen category, select one or more slots; their values are joined in the selected order.

The live preview shows one row from each of the first five FOVs. Check the assigned values before exporting.

Now your task is to map the categories to (combination of) slots.

Example fov_name: MCF7_2DG_dish2_04 has slots: ['MCF7', '2DG', 'dish2', '04']

Choose Categorical features (specified in Configuration @ Home tab) to populate

cell_line × treatment × dish ×

Slots for cell_line

MCF7 ×

Slots for treatment

2DG ×

Slots for dish

dish2 ×

Preview of category mapping:

	image_name	cell_line	treatment	dish
0	MCF7_2DG_dish2_04	MCF7	2DG	dish2
185	MCF7_2DG_dish1_05	MCF7	2DG	dish1
300	MCF7_2DG_dish1_01	MCF7	2DG	dish1
410	MCF7_2DG_dish2_03	MCF7	2DG	dish2
515	MCF7_2DG_dish1_06	MCF7	2DG	dish1

Confirm category mapping & export the combined dataset

Figure 12.2: Map FOV-name slots to categories and check the preview before exporting.

Clicking Confirm category mapping & export the combined dataset then writes the merged dataset with the categorical features as a timestamped CSV (`{timestamp}_combined.csv`) into the same folder as the input dataset. Note that this is a direct file write rather than a browser download, so the file is saved on the machine running FLIM Playground. When using the [hosted demo](#), the file is written to the server and does not reach your computer, which is one reason Data Extraction is intended to be run on a local installation.

Part II

Data Analysis

13 Overview

Data Analysis lets you explore a table of measurements with interactive plots, statistical comparisons, and machine learning. Use a table from [Data Extraction](#) or upload your own measurements and [review their column roles](#).

13.1 General Workflow

1. [Upload your table](#). For a user-provided table, review and save its column roles and feature groups before analysis.
2. Choose an analysis type and method on the left, then select its [numerical features](#).
3. Use the [categorical and numerical filters](#) to select the observations to analyze. Later controls operate on this filtered dataset.
4. Choose [groups and visual encodings](#). In Feature Comparison and 2D Feature Distribution, **Collapse by** can replace individual observations with one mean per category within each analysis group.
5. Adjust the method-specific controls, [plot style](#), and [hover over points](#) to inspect observations.
6. [Export the analysis as a Python script](#) to reproduce or customize it. GMM analyses also offer labeled CSV downloads.

The plots and results update as you change the controls.

13.2 Methods

Analysis type	Method	Numerical input
Univariate	Feature Comparison	One feature compared across groups
Univariate	Feature Histogram	One feature's histogram or Gaussian mixture model
Bivariate	2D Feature Distribution	Two features on a scatter plot
Bivariate	Phasor Plot	A matching pair of phasor G and S coordinates

Analysis type	Method	Numerical input
Multivariate	Dimension Reduction	At least two features projected into two dimensions
Multivariate	Classification	One or more features used to predict categorical labels

Phasor Plot is available after upload only when the table contains a complete G/S pair with the [expected phasor column names](#). All methods use the shared controls below, with additional controls described in their chapters.

13.3 Input

Use **Use Dataset from Data Extraction** for a table produced by [Data Extraction](#). For your own table, clear that checkbox and **upload first**, then review its column roles and feature groups in the [interactive configuration walkthrough](#). The uploader accepts CSV, TSV/TXT with tab, semicolon, or pipe separators, Excel .xlsx/.xlsm, and OpenDocument .ods. Spreadsheet uploads use the first sheet.

13.3.1 Requirements

Use a plain table with column headers on the first row, one observation per row, and at least one usable numerical measurement. For a user-provided table, categorical columns, a unique row identifier, and a field-of-view column are optional. With **Use Dataset from Data Extraction** checked, the table must contain the identifier named in its extraction configuration.

13.3.2 Column roles

Each column in a user-provided table has one role. The app suggests roles after upload; review them according to what the values mean.

- **Row ID** identifies an individual observation in [hover text](#), such as a cell or flower. This role is optional and can belong to at most one column. Its values must be nonmissing and unique after conversion to text. Without an assigned Row ID, the app creates row numbers from 1 through the number of input rows. See [automatic row numbering and changing identifiers](#).
- **Categorical** supplies labels for [filters](#) and visual grouping, such as species, treatment, day, or patient. Numeric codes can be categorical when they represent labels rather than quantities. For a user table, a field-of-view label also uses this role. Categories are stored as text. If every nonmissing value in a numeric category column is a whole number, labels use 1 rather than 1.0; missing values become N/A.

- **Numerical** supplies measurements for plotting and analysis, such as length, intensity, or lifetime. At least one column with this role must contain usable numbers. Numerical columns can be organized into [feature groups](#), each providing a dropdown in the feature pickers.
- **Ignore** excludes a column from analysis while recording its header in the profile, so future uploads can still be matched to the complete table structure.

Use the column's **Role** dropdown to correct a suggestion or change how it is used, then save the profile. For example, assigning Categorical to a numeric treatment code makes it available as a category filter. Changing a Numerical column to another role removes its feature-group assignment. The [column-review walkthrough](#) explains these changes and the checks before saving.

Tip

If a **numerical feature** is missing from the feature pickers, including Uncategorized Features, first check that its Role is Numerical in a user-table review. Then inspect its values for stray text such as --. If non-numeric values account for **1% or less** of the column's non-empty values, the app converts the column to numeric, replaces those cells with NaN, and reports their count. Above that threshold, the column remains text and cannot be used as a numerical feature until those values are cleaned, even if most values are numbers.

FLIM Playground searches for **categorical features** in the uploaded dataset based on the [user-specified configuration](#) if the dataset is not extracted by [Data Extraction](#). Otherwise, it searches for categorical features specified in the [Data Extraction configuration](#).

13.3.3 Warning Messages

Warnings report cleanup such as dropping empty columns or converting a small number of stray non-numeric values to NaN (see feature recognition). Missing measurements can remain in the table. Plot methods use rows with valid values for their [selected numerical features](#); [Classification](#) passes the selected rows to the classifier and reports an error if it cannot accept their missing measurements.

13.3.4 Error Messages

Errors block saving or analysis until the input or roles are corrected. Examples include a malformed table, an assigned Row ID containing missing or duplicate values, or no usable Numerical column. Duplicate row identifiers are not silently removed. In a user-table review, correct the identifier, choose another column, or leave Row ID unassigned to use generated row numbers. For an extraction table, check the identifier and its extraction configuration.

13.4 Shared Interactive Widgets

Feature selection is on the left; filters, grouping controls, plots, and exports are on the right. Only controls supported by the current method are shown.

13.4.1 Numerical Selection Widgets

For extraction tables, numerical features are grouped by [extractor](#) and channel. For user-provided tables, the [saved column review](#) defines the numerical features and their groups. Ungrouped measurements appear under Uncategorized Features. Each group has its own picker.

13.4.1.1 Univariate Analysis

One single-select widget is rendered per group; choosing a feature in any group clears the selections and resets the others to Select.

13.4.1.2 Bivariate Analysis

For 2D Feature Distribution, choose an X feature and a different Y feature from the same or different feature groups. Each axis uses the single-feature selection behavior above, and the X feature is excluded from the Y options. Once selected, an axis's pickers collapse into an expander labeled with the full feature name; reopen it to change the selection. [Phasor Plot](#) instead selects a channel and a complete G/S harmonic pair.

13.4.1.3 Multivariate Analysis

One set of selection widgets, each can select multiple features from a feature group, is rendered. A special value All is introduced so that users can conveniently select all features under the feature group. If users select All, all the other options will be cleared, and vice versa.

Except: selects every feature in a group except the ones you name. For example, select Except: together with `intensity_sum` to exclude that measurement while keeping the other features. Selecting All clears everything else, while selecting a feature clears All but keeps Except:, so you can add exclusions as you click.

Figure 13.1 displays a series of multivariate feature pickers. Each picker consists of a title, a help icon (?), and a selection area. The selection area contains red buttons with 'x' to remove items. The features are:

- Lifetime fit_nadh: All
- Lifetime fit free_nadh: All
- Intensity morphology_nadh: All
- Intensity texture_nadh: Except: intensity_sum
- Lifetime fit_fad: All
- Lifetime fit free_fad: All
- Intensity texture_fad: Except: intensity_sum
- Derived Features: redox_ratio
- Uncategorized Features: Choose options

Figure 13.1: Multivariate feature pickers use Except: to exclude intensity_sum from both the NADH and FAD texture groups.

13.4.2 Filter Widgets

13.4.2.1 Categorical Filters

Figure 13.2 displays four categorical filters. Each filter consists of a title, a help icon (?), and a selection area. The selection area contains red buttons with 'x' to remove items. The filters are:

- Select cell_line(s): All
- Select treatment(s): All
- Select dish(s): All
- Select image_name(s): All

Figure 13.2: Categorical filters select the observations included in the analysis.

For complex datasets that are collected over multiple days, experiments, treatments, etc., it is useful to filter the data to focus on a subset of the data (data of interest). One filter widget is rendered for each categorical feature so that users have the flexibility to filter the data based on combinations of

categorical features. All is a special option that include all categories of the selected categorical feature. Once it is selected, all the other options are cleared, and vice versa.

Use **Except**: when it is easier to name the categories to leave out. For example, select **Except**: and one value in an `image_name` filter to keep every other field of view. The exclusion is evaluated against the current data, so categories added by widening another filter are included unless you excluded them explicitly.

The filters work as a set rather than in sequence: each one offers only the categories that still have data under everything else you have selected. That means you can approach a subset from whichever direction is natural — narrowing `treatment` first and then `day` leaves you the same options as doing it the other way around — and no combination is out of reach because of the order the categorical features happen to be listed in your configuration.

One consequence is worth knowing about. Tightening one filter can deselect a choice you made elsewhere. Say you have fields of view picked out under `image_name` and then narrow `cell_line` to `Panc1`: every MCF7 field of view among them is left with no rows. Rather than hand you an empty plot, FLIM Playground deselects those fields of view and tells you which selection ruled them out.

A filter therefore never quietly stops matching what you asked for.

13.4.2.2 Numerical Filters

Users can further filter the data based on numerical features. Each row specifies a condition to filter the data:

- **Feature**: the numerical feature to filter on
- **Operator**: `>` or `<=`
- **Threshold**: the range of values to filter on, dynamically updated based on previous filters, is displayed for reference. The entered value should be within the range.
- **Add another**: show the next condition. Clear it to stop applying later rows; choose `None` in a feature picker to stop at that row.

After the two sets of filters are applied, the final filtered data is used for visualization and analysis.

! Important

Make sure you clearly state all numerical filters used.

13.4.3 Visual Channels Widgets

Visual encodings map categories to color, opacity, or marker shape, making patterns easier to compare¹². Grouping controls also determine which observations are compared or modeled together.

Method	Grouping and layout	Point encoding
Feature Comparison	Color by / Group by, Separate by, Collapse by	Choose one of Opacity, Subcolor, or Shape, then a column
Feature Histogram	Color by, Separate by	—
2D Feature Distribution	Color by, Separate by, Collapse by	Choose Opacity or Shape, then a column
Phasor Plot	Color by, Separate by	Independent Opacity by and Shape by pickers
Dimension Reduction	Color by, up to two columns in Separate by	Independent Opacity by and Shape by pickers
Classification	Classify by forms the target classes	—

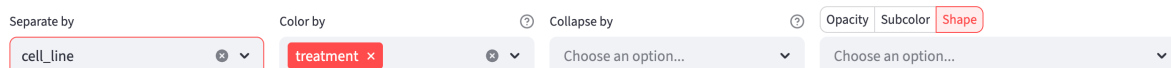


Figure 13.3: Feature Comparison controls for grouping, averaging by category, and point encoding.

13.4.3.1 Color by

Choose one or more categorical features in **Color by**. Each distinct combination receives a color; combined labels use `::` between categories. In Feature Comparison, these groups also define the x-axis positions. When its **Subcolor** mode is selected, **Color by** becomes **Group by**: it still defines the comparisons, while the selected subcolor column supplies point colors.

Leave the grouping picker empty to analyze all filtered observations as one group. In Classification, **Classify by** defines the labels the classifier learns instead.

i Note 1: How to order the groups

The order of selected categorical features determines the grouping hierarchy. For example, choose `cell_line` and then `treatment` to keep each cell line's treatments together; reverse the selection order to keep treatments together.

Within each feature, groups use numeric-alphabetical sorting: labels containing a number are ordered by that number, then by their text. Thus `Panc1` precedes `MCF7`, and `Day 30` precedes `Day`

100. Feature Comparison also provides [custom x-axis ordering](#).

13.4.3.2 Opacity and Shape by

Opacity by gives ordered categories different transparencies, and **Shape by** gives them different marker shapes. Each uses one categorical column, with categories ordered [numeric-alphabetically](#). These encodings change the appearance of points while preserving the groups used for modeling.

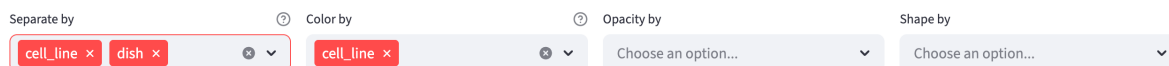


Figure 13.4: Dimension Reduction keeps independent opacity and shape controls beside its category-map and color selections.

Feature Comparison shares one column picker between **Opacity**, **Subcolor**, and **Shape**. 2D Feature Distribution shares one between **Opacity** and **Shape**. Select the mode above the picker, then choose a column; switching modes keeps that column when it is available. Clear the column to turn the encoding off. Only the active mode is applied.

13.4.3.3 Separate by

Separate by organizes the current data into category views:

- [Feature Comparison](#) places categories in sections on one x-axis.
- [Feature Histogram](#) stacks one histogram or GMM panel per category.
- [2D Feature Distribution](#) and [Phasor Plot](#) show one category at a time with the others in gray for context.
- [Dimension Reduction](#) adds category highlight maps beside a shared overview.

For the first four methods, the separation column cannot also be used for **Color by**. Dimension Reduction permits the same column for both because separation highlights parts of one shared embedding.

13.4.3.4 Collapse by

In [Feature Comparison](#) and [2D Feature Distribution](#), **Collapse by** averages observations that share a value in a selected categorical column, separately within each analysis group. The category determines what each mean represents. For example, choose `dish` to plot one mean per dish within each treatment and cell line, or `patient` to average observations from the same patient within each group.

The resulting category means become the input to the method’s summaries, statistics, and models. The method chapters explain which measurements enter each mean and how [SuperPlot](#) can show the original observations alongside them. Choose the grouping columns first: the same column cannot also be selected for collapse.

13.4.4 Unique ID Hover

Hover over a point to see its measurement values and identifier. Extraction tables label the identifier `Cell ID` and include the configured field-of-view column when present. User tables show the assigned Row ID column’s name, or ID when the app generated row numbers. User-table field-of-view columns remain ordinary categories.

With `Collapse by`, the hover identifies the category and the number of observations averaged into that point, such as `dish1 (n=1874)`. [SuperPlot](#)’s smaller observation points retain their individual identifiers.

13.4.5 Plotting Configuration Widgets



Figure 13.5: Plot Styling controls point size, text sizes, the color map, and optional legend counts.

Under **Plot Styling**, adjust `Point Size` for point plots, `Axis Label Font Size`, `Legend Font Size`, and `Color Map` when color groups are present. Available palettes include `tab10`, `tab20`, `colorblind`, `Set1–Set3`, `Pastel1`, `Pastel2`, `Accent`, `viridis`, `plasma`, `inferno`, `magma`, and `cividis`.

`Show group counts (n) in legend` adds counts where supported. Counts refer to the points used in the analysis: individual observations, or category means when `Collapse by` is active. When `Separate by` is active, `Phasor` and `2D` views count only points from the selected separation category. Histogram panels report their own group counts; `Dimension Reduction`’s shared legend reports counts for the full embedding.

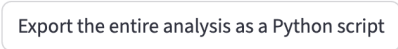
`Classification` places its font-size controls between the metric tables and the ROC/confusion-matrix plots.

13.4.6 Feature Labels

Numerical features extracted by [Data Extraction](#) are automatically rendered in proper FLIM notation on plot axes, legends, and hover text. For example, the column `Lifetime fit_nadh: a1` is shown as “ $\text{nadh } \alpha_1 (\%)$ ”, and other extracted features map to symbols such as “ τ_1 (ps)”, “ τ_m ”, “ $\tau_{m,i}$ ”, “ χ_r^2 ”, and “ τ_{mod} ”. The mapping (channel + Greek/scientific symbol + unit) follows the feature naming convention introduced in [Lifetime Features \(Fit\)](#). Columns that are not recognized — identifiers, categorical features, or arbitrary user-provided columns — are displayed unchanged. The same labels are used in the [exported script](#), so its figures match what you see in the app.

13.4.7 Export Workflow as Python Script

Click **Export the entire analysis as a Python script** below the results and styling controls. The script records the selected features, filters, grouping, point encodings, plot style, and method settings, including collapse, category separation, and GMM export names where applicable.



Export the entire analysis as a Python script

Figure 13.6: Download a Python script that records the current analysis settings.

Keep the original uploaded table beside the downloaded `.py` file and run the script from that directory, or edit its `DATA_PATH` to point to the table. The dataset is not embedded in the script. The script reproduces the analysis with matplotlib and saves SVG figures that you can edit or use in a publication.

For [1D](#) and [2D GMM](#) analyses, the app also offers labeled CSV downloads. Those downloads contain the analyzed data, including any collapse or log transformation. In the Python script, set `SAVE_DERIVED_DATA = True` to write the corresponding labeled CSV alongside the figures.

14 Configuration

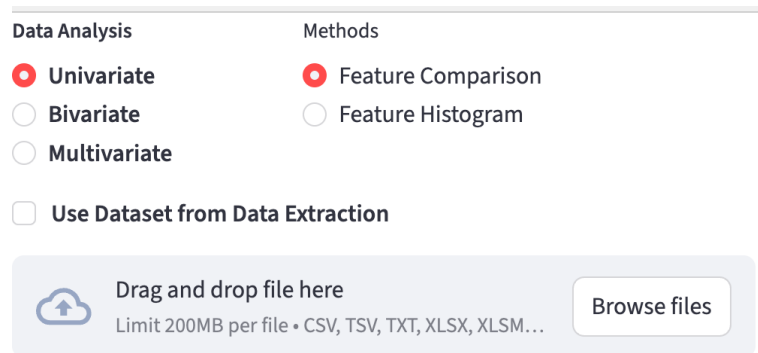
For a table you prepared yourself, configuration starts after upload on **Data Analysis**. Review how each column will be used, organize measurements into feature groups, and save those choices as an analysis profile. The app can reuse a profile for later tables with matching headers.

This walkthrough uses [iris.csv](#), a table of **150 flowers and six columns**. Download it to follow the screenshots. Each step includes a detail view; the web version also links to each full screenshot.

1. Upload Iris

Open **Data Analysis** and select **Univariate** → **Feature Comparison** for the view shown here. Clear **Use Dataset from Data Extraction**, then upload `iris.csv`.

If **Which profile describes this file?** appears, choose **Auto-detect — start a new profile**. This fills the column review with suggested roles and groups. If a saved profile already matches this table, analysis may open directly; use the summary pencil described in Step 5 to inspect it.



Data Analysis	Methods
☒ Univariate	☒ Feature Comparison
☐ Bivariate	☐ Feature Histogram
☐ Multivariate	
☐ Use Dataset from Data Extraction	

 Drag and drop file here
Limit 200MB per file • CSV, TSV, TXT, XLSX, XLSM...
Browse files

Figure 14.1: Clear the extraction checkbox before uploading your own table.

For your own data, use a plain grid with column names on the first row and one observation per row. The uploader also accepts TSV/TXT with tab, semicolon, or pipe separators, Excel `.xlsx/.xlsm`, and OpenDocument `.ods`. Put a spreadsheet's intended table on its **first sheet**.

2. Review column roles

The review reports **6 columns** × **150 rows**. Each column has one **Role**: **Row ID** identifies an observation, **Categorical** supplies labels for filtering or grouping observations, **Numerical** supplies measurements, and **Ignore** excludes a column. See [Column roles in the Data Analysis overview](#) for their uses and requirements.

For Iris, keep the four length and width columns **Numerical**, species **Categorical**, and flower_id **Row ID**:

Column	Role
sepal_length	Numerical ▼
sepal_width	Numerical ▼
petal_length	Numerical ▼
petal_width	Numerical ▼
species	Categorical ▼
flower_id	Row ID ▼

Figure 14.2: The six Iris columns and their suggested roles.

To change a role, open that column’s **Role** dropdown and choose its intended use. For example, change a numeric treatment code from Numerical to Categorical so it appears as a category filter. Check the preview values before deciding: a unique integer measurement can be guessed as Row ID. A change away from Numerical removes the column’s feature-group assignment; changing it back enables its **Group** dropdown so you can assign it again. Save the profile to apply role changes to analysis.

A Row ID is optional. Auto-detection first tries the leftmost column containing distinct, nonmissing whole numbers or text labels. If no column has the Row ID role, the app automatically adds **Row number** to the analysis table, populated with 1 through 150 for Iris (or 1 through the number of input rows). These labels follow the uploaded row order and identify points in hover text. They are regenerated for each upload, so use your own identifier when identity must remain stable across reordered files. If Row number already exists, the generated column uses Row number . 1, then . 2, and so on as needed.

To use generated numbers for Iris, change `flower_id` from Row ID to **Ignore**; choose Categorical or Numerical instead if you need that column for another purpose. The review then says **Rows will be identified by row number**. Only one column can be Row ID. Assigning it to another column automatically changes the previous holder to Numerical if it contains numbers, or Categorical otherwise; check that reassignment. Missing or repeated values in an assigned ID block saving, so correct the values, choose another identifier, or remove the role.

At least one **Numerical** column must contain usable numbers. Categorical labels are stored as text. If every nonmissing value in a numeric category column is a whole number, labels use 1 rather than 1.0; missing values become N/A.

Profile matching uses **exact headers**. Case, spaces, and punctuation matter; IL-18 and IL_18 are different names. Reordering columns does not change the match.

3. Feature group management

Feature group management, above **Column preview**, organizes Numerical columns into shorter lists. Each group becomes a dropdown in the analysis feature pickers, making related measurements easier to find in a wide table. Groups are optional: unassigned measurements remain available under **Uncategorized Features**. Feature groups organize columns; categorical features such as species group or filter observations. Changing a feature group does not change measurement values.

How group names are inferred

For a new profile, the app takes the prefix before the earliest `_`, `.`, or `:` (a colon followed by a space) in each Numerical column's header. At least two new Numerical columns must share that prefix to create a suggested group:

Column headers	Suggested group
sepal_length, sepal_width	sepal
petal_length, petal_width	petal
NADH.mean, NADH.std	NADH
Texture: contrast, Texture: entropy	Texture

Hyphens remain part of a name: `anti-PD1_mean` and `anti-PD1_std` form **anti-PD1**. A header without a supported prefix, or a lone prefix in a new profile, stays ungrouped. Saved assignments are preserved when reusing a profile. A new column can follow existing columns with the same prefix into their saved group, including a renamed group; otherwise it can join an existing group whose name matches its prefix.

Change groups in bulk

Keep the suggested **sepal** and **petal** groups, or combine their four measurements into **Flower size** as shown here:

1. In **Feature group management**, type **Flower size** in the **Group name** box and click **+ Add**. This creates and selects an empty destination group.
2. Tick the checkboxes beside the Numerical columns you want to move. For Iris, click **Select all** to tick all four measurements; this button becomes **Clear** to deselect them.
3. Select **Flower size** in the group dropdown above the table, then click **Apply to 4**. The number on **Apply to N** counts the ticked columns. Their **Group** values change together.

Feature group management ?

+ Add

Flower size ▼

Apply to 4

Column preview ?

	Column	Role	Group
☒	sepal_length	Numerical ▼	sepal ▼
☒	sepal_width	Numerical ▼	sepal ▼
☒	petal_length	Numerical ▼	petal ▼
☒	petal_width	Numerical ▼	petal ▼

Figure 14.3: Four Numerical columns selected for bulk assignment to Flower size.

Only Numerical columns have enabled selection checkboxes and **Group** dropdowns. For a single feature, change its row's Group directly. To remove several assignments together, tick those columns, choose **Uncategorized** as the destination, and click **Apply to N**.

To **rename an entire group**, select it in Feature group management, type the replacement name in **Group name**, and click . Every member follows the new name; entering another existing group's name **merges** the groups. To **delete a group**, select it and click ; its columns remain available as Uncategorized Features. Save the profile to retain any group changes.

4. Save as Iris

For a new profile, type **Iris** in the profile-name box beside  **Save profile as**, then click the button. Initial review requires a save before analysis can begin. Saving applies the reviewed roles and groups to this upload and stores them for future uploads; editing the review alone does not update a saved profile. An existing profile instead offers  **Save to {profile} & use**.

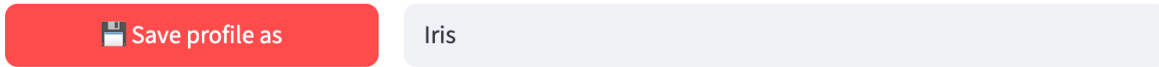


Figure 14.4: Save the reviewed roles and groups as Iris.

If **Iris** already exists, the first click requests confirmation.  **Replace profile** overwrites that profile's roles and groups; enter a different name to keep both configurations.

5. Reopen from the Iris summary

After saving, select **sepal_length** from **Flower size** to begin analysis. If you kept the automatic groups, select it from **sepal** instead.

The applied-profile summary reads **Iris: 1 Categorical · 4 Numerical**. Its counts omit the optional Row ID. Click the small  beside this summary to review the columns again.

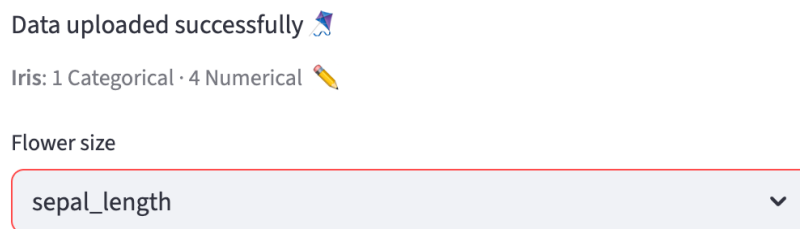


Figure 14.5: Use the pencil beside the Iris summary to reopen review.

Reopening hides the analysis controls and plots while preserving their settings.  **Save to Iris & use** saves your changes and resumes analysis. **Cancel** discards the current column and group edits, reloads the saved decisions, and resumes analysis. Cancel is available when reopening a saved decision.

6. Manage saved profiles

While review is open, expand **Manage saved profiles** below the save controls. It lists every saved profile. To rename one, click its , enter a name in **Rename to**, and click **Rename**. To delete one, click its , then choose **Delete** or **Keep**.

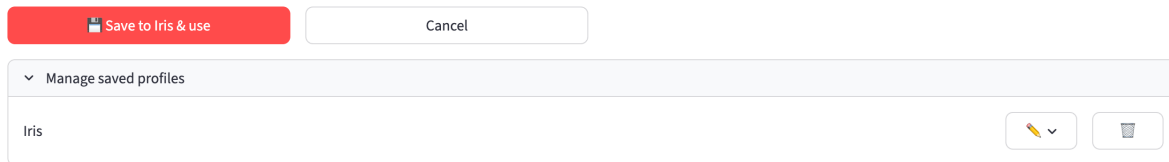


Figure 14.6: Save or cancel review edits; manage stored profiles below.

Renames and confirmed deletions take effect immediately; Cancel only discards column and group edits. Deleting the profile supplying this review clears its decisions and returns to the chooser. Deleting the last profile leaves no saved profiles. You can store **20 profiles**; at the limit, replace an existing profile or delete one before saving another.

On later uploads, the app compares the **complete header set**, including columns marked Ignore. One exact match with valid stored roles applies automatically. Otherwise, the chooser offers profiles sharing at least one column and reports **shared**, **missing**, and **new** counts, followed by Auto-detect.

Selecting a partial match keeps saved roles and groups for present columns and guesses new ones. Changing the chooser selection discards your current edits. **Save to {profile} & use** replaces that profile with this file's decisions: new columns are recorded and absent columns are removed. Review the match counts before updating a profile used by other tables.

Part III

Univariate Analysis

15 Feature Comparison

This method allows users to compare the distributions of a single numerical feature across groups. It can help users spot overarching trends between distributions, identify within-distribution shape-related characteristics such as skewness or bimodality that might signal sub-populations, and diagnose potential outliers.

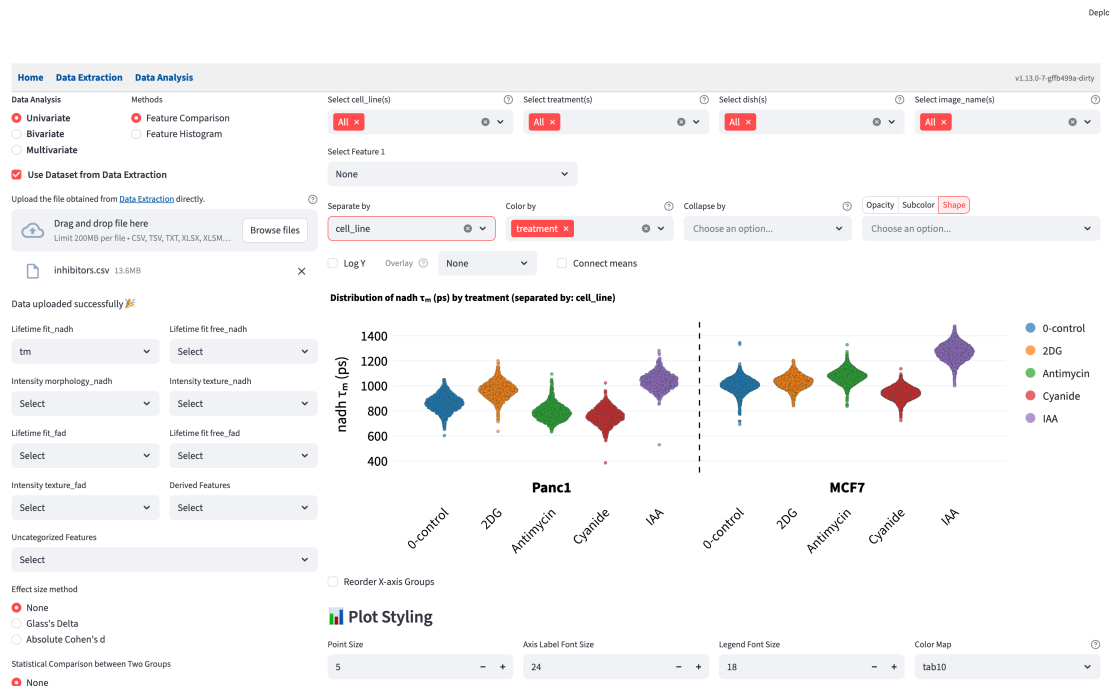


Figure 15.1: Feature Comparison brings feature selection, filtering, grouping, and the comparison plot into one view.

Note

To keep the point-based visualization style that supports [interactive hover](#) while showing the violin-plot-like distribution, [Sina plot](#) is used: it uses `gaussian_kde` from `scipy.stats` to make sure the width of the point distribution is proportional to the kernel density.

The spread is computed per group on the x-axis, so **Shape by**, **Opacity by** and **Subcolor by** change how a point looks, not where it sits.

15.1 Shared Interface Components

- On the left, users can use the **selection widgets** to select the numerical feature to be compared. Exactly one feature can be selected from all feature groups.
- On the top right, users can use the **filters** to subset the data to find the groups of interest.
- Below the filters, choose **Separate by**, **Color by** (or **Group by** in **Subcolor mode**), and **Collapse by**. The **Opacity | Subcolor | Shape** selector shares one column picker: choose a mode, then its categorical column. Only that point encoding is applied.
- On the bottom right, users can change the plot style using the **plot styling widgets**.

15.2 Plot Options

Above the plot, use **Log Y**, **Overlay**, and **Connect means** to control the display:

- **Log Y**: transforms the selected feature with $\log_{10}(x + 10^{-6})$ before plotting and calculating summaries and comparisons. The small offset allows zero values; the axis label becomes $\log_{10}(\dots)$. If negative values are present, the app reports an error and keeps the original scale. With **Collapse by**, it averages first and then transforms the category means.
- **Overlay** → **Boxplot**: overlays a box-and-whisker summary on each group. The box spans the interquartile range ($Q_1 - Q_3$) with the median as a solid line and the mean as a dashed line; the whiskers follow the **Tukey** definition at $Q_1 - 1.5 \times \text{IQR}$ and $Q_3 + 1.5 \times \text{IQR}$, bounded by the data range. Choose **None** to show the points without an overlay. Selecting **Collapse by** also makes **SuperPlot** available.
- **Connect means**: draws a line connecting the mean of each group (within each section when **Separate by** is used), which can help reveal trends across ordered groups.

15.3 Collapse by

Choose a categorical column in **Collapse by**, such as **dish**, **patient**, or **image_name**, to replace individual observations with one mean per category within each displayed group. The app first applies the filters and removes rows missing the selected measurement. It then averages the measurement separately for every combination of the selected collapse category, **Color by** / **Group by**, and **Separate by**. A dish represented in two treatments therefore contributes one point to each treatment.

The boxplot, connected means, effect sizes, and statistical tests now use these category means. Each point has equal weight in its displayed group's mean, regardless of how many observations were averaged into it. Selecting a collapse column keeps the existing x-axis groups and sections. A column already used for either grouping control is unavailable in Collapse by.

Collapse changes the analysis unit, but the supported t-tests still assume independent samples. The app does not pair observations across groups, even when the same dish or patient appears in several groups.

Hover over a category mean to see its label and contributing observation count, such as `dish1` (n=1874). **Show group counts (n) in legend** counts category means after collapse. If a point-encoding column varies within any group being averaged, the app turns that encoding off and explains why; using the collapse column itself for Subcolor, Opacity, or Shape keeps it available.

15.3.1 SuperPlot

After selecting Collapse by, choose **Overlay** → **SuperPlot** to show the original observations as smaller, fainter points behind the category means. The larger points remain the category means, and each displayed group receives a mean bar with $\pm$ one standard error of the mean (SEM), calculated across those category means. A group with only one category mean still has a mean bar, but its SEM is undefined.

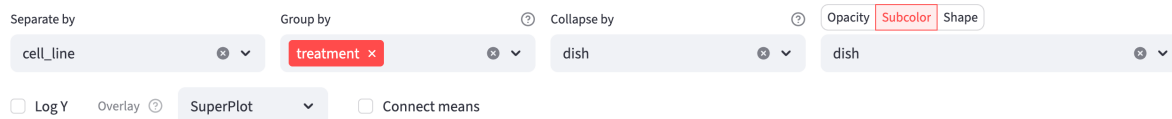


Figure 15.2: Use the same categorical column for Collapse by and Subcolor, then select the SuperPlot overlay.

The original observations provide context while all comparisons continue to use category means. Hovering over a small point shows its individual identifier; hovering over a large point shows the category and its observation count. Legend counts still count category means. With **Log Y**, the small points are transformed individually and the larger points are transformed after averaging, so a large point need not equal the mean of the small points on the transformed scale.

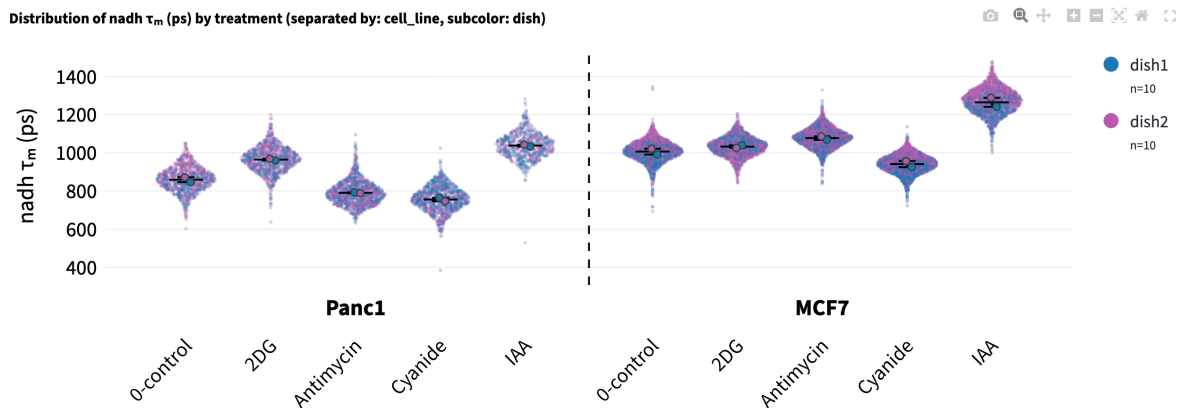


Figure 15.3: SuperPlot shows original cells behind dish means and mean $\pm$ SEM summaries.

15.4 Separate by

`Separate by` takes effect before `Color by`. `Separate by` divides the plot into sections separated by a dashed line, with each section labeled by one of the available categories of the selected categorical feature after `filtering`. The selected feature of `Separate by` is automatically removed from available features in `Color by`. The section order is determined by the order of the categories which are sorted `numeric-alphabetically`.

Point encodings and comparisons are then applied within each section. For example, select `cell_line` in `Separate by` and `treatment` in `Color by` to compare treatments separately for each cell line. Treatment colors stay consistent across sections. `Effect sizes` are calculated within each section, with annotations limited by the selected pairs and `effect size threshold`.

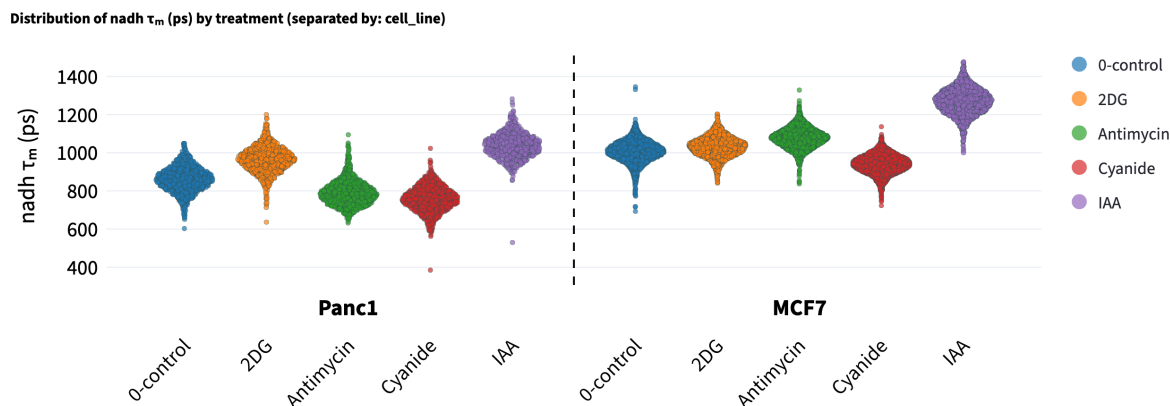


Figure 15.4: Separate by divides the x-axis into cell-line sections while treatment colors remain consistent.

If `Separate by` is left blank and both `cell_line` and `treatment` are selected in `Color by`, then colors are assigned to each cell line and treatment combination, and effect sizes are calculated across all combinations of group pairs (again, only pairs at or above the [effect size threshold](#) are shown).

15.5 Subcolor by

`Color by` gives one color per x-axis group, which leaves no way to ask where the points *within* a group came from. If each treatment pools three donors, a shift between treatments may be a real effect, or it may be one donor sitting away from the other two. `Subcolor by` answers that.

Select **Subcolor** in the **Opacity | Subcolor | Shape** selector, then choose a categorical column in the picker below it. `Color by` is relabeled `Group by`: it continues to define the x-axis groups, while the subcolor column supplies point colors. Switching to `Opacity` or `Shape` keeps the selected column when it remains available, but applies only the new encoding. Clear the column to turn the encoding off.

Only the paint changes: x positions, tick labels, the [boxplot and mean connector](#), [effect sizes](#), [statistical tests](#), and [comparison pairs](#) all stay on the `Group by` groups. The color map is **global**, one color per value across the whole figure, so a donor appearing under three treatments is the same color in all three, gets a single legend entry, and toggles everywhere at once. With the [group count toggle](#) on, its `n` is likewise the figure-wide count.

The picker offers categorical features with more than one category after [filtering](#), except the [Separate by](#) feature, and is disabled while `Group by` is empty. A grouping column can also supply the point encoding. To give each collapsed category a consistent color across treatments, choose the same categorical column in `Collapse by` and `Subcolor`. Missing values form an N/A level with a color of its own rather than being dropped.

How the subcolors are chosen

Subcolors are generated rather than sliced out of the [colormap](#): cyclic palettes put their first and last entries at neighboring hues, and qualitative ones such as `tab10` repeat once you ask for more colors than they hold. The colormap's first color seeds a palette of exactly the size needed, built in [OKLCH](#) so the members read as a family — hue confined to an arc around the seed, lightness stepping outward for separation. Candidates are scored composited over the background at the opacity points are drawn with, since two semi-transparent colors that differ in RGB can look the same on screen. The result is deterministic, so the [exported script](#) matches the app.

Important

`Subcolor by` is for a **small** nested feature, roughly 2–5 values. Past about 7, no palette lets a reader assign a point to a value; use [Separate by](#) or a [filter](#) instead.

15.6 Reorder X-axis Groups

Groups are ordered [numeric-alphabetically](#) by default, which puts the control wherever its name happens to sort. The `Reorder X-axis Groups` checkbox below the plot opens a drag-and-drop list; rearrange it and press `Confirm Reordering`.

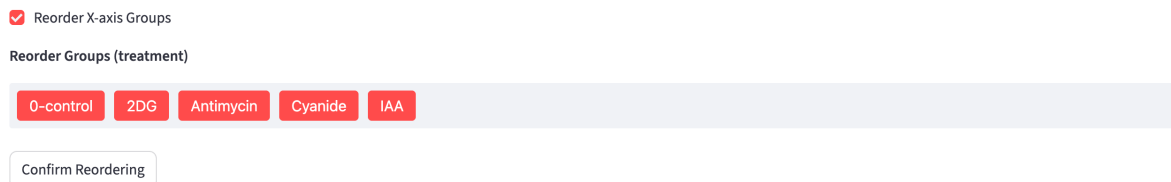


Figure 15.5: Drag the groups into the desired x-axis order, then confirm the change.

The order is applied **before** [comparison pairs](#) are formed, and each pair runs left to right, so the leftmost member is the one [Glass's Delta](#) treats as the control. Reordering is how you tell FLIM Playground which group that is.

- It reorders the `Color by / Group by` groups only. [Separate by](#) sections keep their default order, and one group order is shared by every section.
- It is stored per (numerical feature, `Color by`, `Separate by`) combination. Groups you did not move keep their relative order, and groups that turn up later are appended rather than resetting your arrangement.
- It is carried into the [exported script](#) as `CUSTOM_ORDER`.

15.7 Comparison Widget

Statistical tests and effect size calculations are performed on the selected comparison pairs. FLIM Playground uses the groups on the x-axis populated by **Color by** — labeled Group by when **Subcolor by** is on — to create comparison groups, starting from the leftmost group. Pairs are formed after any **custom x-axis order** is applied. For example, if there are 5 groups, then $\binom{5}{2} = 10$ comparisons will be created. The number of groups can get combinatorially large and many of the comparisons are not meaningful. Therefore, two widgets are provided to filter the comparison groups:

- A selection widget that shows all possible comparisons by default and users can deselect groups they do not need.
- A threshold widget that filters out comparisons whose effect size falls below the threshold.

15.7.1 Effect Size Threshold

The threshold widget is rendered per effect size method, and the two do not share a default:

Effect size method	Default	Step	Range
Glass's Delta	0.7	0.05	0–3
Absolute Cohen's d	0.5	0.1	0–3

A pair is kept when the **absolute** effect size is greater than or equal to the threshold, so a large negative **Glass's Delta** is not dropped for being negative; 0 keeps every selected pair. Each threshold is remembered per numerical feature. Under **Separate by**, one threshold and one pair selection govern every section.

15.8 Effect Size

Complementary to statistical tests, which address “is there a difference?”, effect size metrics answer “how large is the difference?”, enabling cross-study comparison. FLIM Playground offers two effect size calculation methods, Glass's Delta and Absolute Cohen's d, each can be in mean or median form.

15.8.1 Glass's Delta

Mean ($\bar{X}$) form:

$$\Delta_G = \frac{\bar{X}_{\text{treatment}} - \bar{X}_{\text{control}}}{s_{\text{control}}}, \quad s_{\text{control}} = \sqrt{\frac{1}{n_c - 1} \sum_{i=1}^{n_c} (X_{\text{control}_i} - \bar{X}_{\text{control}})^2}$$

Median ($\tilde{X}$) form:

$$\tilde{\Delta}_G = \frac{\tilde{X}_{\text{treatment}} - \tilde{X}_{\text{control}}}{\text{MAD}_{\text{control}}}, \quad \text{MAD}_{\text{control}} = 1.4826 \times \text{median} |X_{\text{control}_i} - \tilde{X}_{\text{control}}|, \quad i = 1, \dots, n_c$$

i Note

MAD stands for [median absolute deviation](#). In order to make MAD asymptotically consistent with the standard deviation under normality, it is scaled by $C = 1/\Phi^{-1}(0.75) \approx 1.4826$. Implementation-wise, it uses `median_abs_deviation` from `scipy.stats`, and `scale` option is set to "normal" to account for the constant multiplier C . THE MEDIAN FORM of both Glass's Delta and Absolute Cohen's d IS NOT REPORTED IN THE LITERATURE.

! Important

Which group is treatment and which is control matters! From the formula of Δ_G , switching the order not only flips the sign of the result, but also changes the denominator, which is s_{control} , the standard deviation of the *control*. FLIM Playground treats the group on the left as control and the group on the right as treatment. Use the [custom x-axis order](#) to place the control on the left. [Absolute Cohen's d](#) does not assume a treatment-control structure.

15.8.2 Absolute Cohen's d

Mean ($\bar{X}$) form:

$$|d| = \left| \frac{\bar{X}_1 - \bar{X}_2}{s_p} \right|, \quad s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Median ($\tilde{X}$) form:

$$|\tilde{d}| = \left| \frac{\tilde{X}_1 - \tilde{X}_2}{s_{\tilde{p}}} \right|, \quad s_{\tilde{p}} = \sqrt{\frac{(n_1 - 1)\text{MAD}_1^2 + (n_2 - 1)\text{MAD}_2^2}{n_1 + n_2 - 2}}$$

15.9 Statistical Test

Statistical tests are performed on the selected comparison pairs that meet the effect size threshold (if users choose to calculate effect sizes). The statistical tests supported are:

- Independent t-test (student's t-test, for equal variances)
- Welch's t-test (for unequal variances)

FLIM Playground uses `scipy.stats.ttest_ind` to perform the independent t-test and `scipy.stats.ttest_ind` with `equal_var=False` to perform the Welch's t-test.

With [Collapse by](#), sample sizes count the category means in each comparison group. The app reports when a selected comparison has too few points for a statistic; adding observations to an existing category can change its mean but does not add another point.

P-values are summarized with significance asterisks: if $p \leq 0.0001$, four asterisks (****) are shown; otherwise if $p \leq 0.001$, three (***) ; otherwise if $p \leq 0.01$, two (**); otherwise if $p \leq 0.05$, one (*). Values with $p > 0.05$ are not given a significance asterisk marker in this convention.

15.10 Comparison Annotation

Eligible comparisons are annotated with brackets above the points and existing annotations. The annotation combines the numeric effect size with significance asterisks when both are available; it does not display a numeric p-value. For example:

- 1.23** shows an effect size of 1.23 with $p \leq 0.01$.
- ** shows significance when only a statistical test is selected.
- $\Delta=1.23$ shows an effect size without significance asterisks, including when the selected test has $p > 0.05$.

16 Feature Histogram

Feature histograms give an instant readout of each group's center, spread, skew and any multi-peaked shape that raw tables hide. FLIM Playground plots a histogram for each group based on the selected numerical feature. Additionally, the [Gaussian Mixture Model](#) mode can help find subcomponents of each histogram in an unsupervised way. To quantify the distribution heterogeneity, normalized [H-index](#) is calculated for each group.

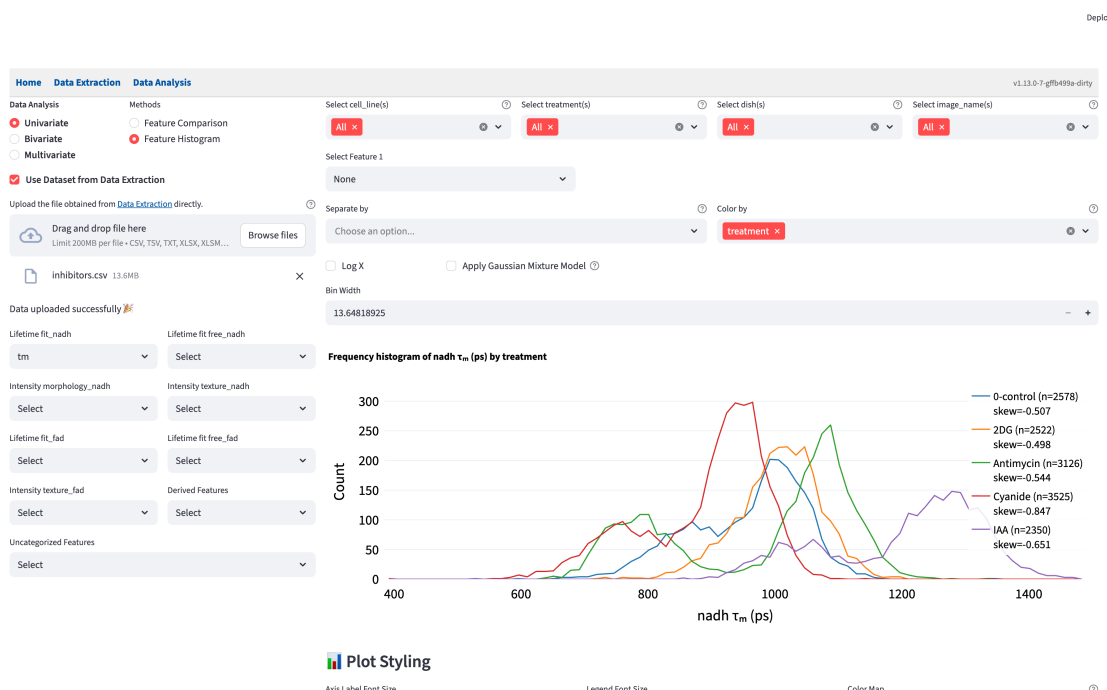


Figure 16.1: Histogram mode shows group counts and skewness alongside shared bin-width and GMM controls.

16.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select the numerical feature to see the histogram/GMM distribution. Exactly one feature can be selected from all feature groups.

- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, use [Color by](#) to define groups and [Separate by](#) to place categories in separate panels. Both histogram and GMM modes analyze individual observations within each category and color group.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

16.2 Histogram Mode

Select **Apply Gaussian Mixture Model** to switch from histogram mode to [GMM mode](#). Clear it to return to the histogram.

A **Log X** checkbox next to the mode switch transforms the selected feature with $\log_{10}(x + 10^{-6})$ before computing either analysis. Histogram bins and skewness then describe transformed values; the GMM components, model selection, [classification](#) thresholds, and [H-index](#) are also computed in log space. The small offset allows zero values. If negative values are present, the app reports an error and keeps the original scale.

Histogram mode plots observation counts for each group. Use **Bin Width** to adjust the shared bins. The default uses NumPy's automatic choice between the Sturges and Freedman–Diaconis estimators, capped at one-third of the data range (see [NumPy's bin estimators](#)). The maximum width is one-third of the range and the step is 1/50 of the range. The app keeps separate width choices for each feature's raw and logarithmic scales. A constant or very small dataset may have a single bin and no width control.

The legend reports each group's bias-corrected skewness. It displays *undefined* for fewer than three observations or a constant group. Enable **Show group counts (n) in legend** under **Plot Styling** to include the number of observations in each group.

A rule of thumb categorization of skewness is as follows:

- Strongly left-skewed: < -1
- Moderately left-skewed: -1 to -0.5
- Approximately symmetric: -0.5 to -0.25 and 0.25 to 0.5
- Almost symmetric: -0.25 to 0.25
- Moderately right-skewed: 0.5 to 1
- Strongly right-skewed: > 1

Skewness alone cannot describe every distribution shape: a symmetric distribution can still have several peaks. When these peaks might represent subpopulations, such as cell types, states, or cell-cycle phases, use [Gaussian Mixture Model mode](#) to explore possible components.

16.3 Separate by

Choose one categorical column in **Separate by** to stack one full-width histogram or GMM panel per category. The same color represents the same **Color by** group in every panel. The separation column is removed from the available **Color by** choices.

For the [inhibitor dataset](#), select **a1** under **Lifetime fit_nadh**, set **Separate by** → **cell_line**, and set **Color by** → **treatment**. Keep the categorical filters at **All** and **Apply Gaussian Mixture Model** cleared. The result is exactly two stacked histograms: **Panc1** above **MCF7**.



Figure 16.2: Separate by **cell_line** produces two stacked histograms for the inhibitor dataset, with the same treatment colors in both panels.

All panels share axis ranges and, in histogram mode, the same bin edges. Histogram legends report local skewness; both modes can display local counts. GMM fits are calculated separately within each category and color group; their component tables and any fitting notices appear below the plots. Changing

Separate by therefore changes the populations being modeled. This method always uses individual observations; averaging by category is available in [Feature Comparison](#) and [2D Feature Distribution](#).

16.4 Gaussian Mixture Model Mode

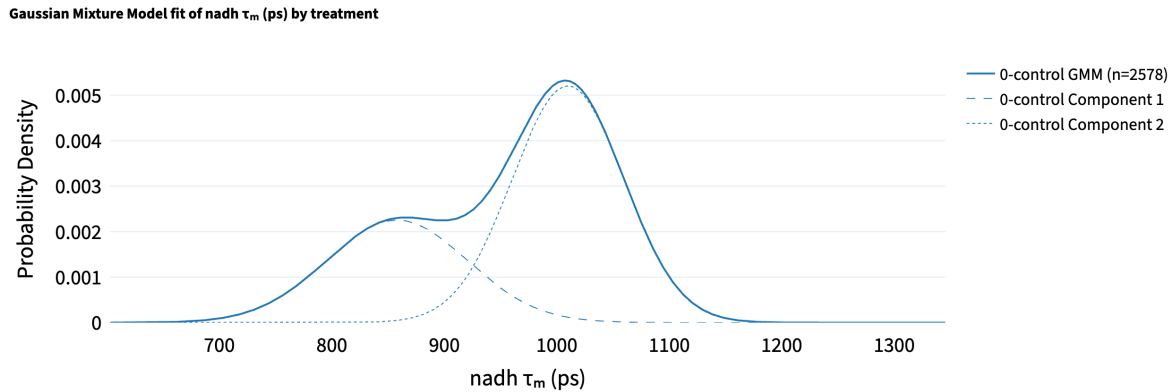


Figure 16.3: A fitted mixture density is shown with its individual Gaussian components.

A Gaussian Mixture Model (GMM) represents a dataset as a weighted sum of multiple Gaussian (normal) distributions, each defined by its own mean (μ) and (co)variance (σ^2). By estimating the component parameters and their weights (mixing coefficients) from the data—commonly using the Expectation-Maximization (EM) algorithm—GMMs can capture complex, multimodal distributions that a single Gaussian cannot.

16.4.1 Fit Gaussian Mixture Models

- Fits candidate GMMs directly to the selected numerical values, starting with one component and continuing up to `Max Components`. The maximum can be set from 2 to 5; its default is 3. Each component has a weight (w_i), mean (μ_i), and variance (σ_i^2).
- Retains only *valid* models in which every component has weight at least `Min Weight Threshold` (default 0.1, or 10%; the weights of all components add to 1).
- FLIM Playground will choose the valid model with the lowest BIC score that penalizes the model complexity to avoid overfitting.

The plot shows the fitted mixture density and, for a model with multiple components, its component curves. Tables below the plot report **Mean $\pm$ SD**, **Weight**, and the group's H-index. These are fitted component parameters based on soft membership probabilities; the mean, SD, and fraction of rows assigned to a component by a hard rule can differ. A group needs at least two distinct observations for

fitting. Groups with insufficient data or no valid model receive a notice without preventing the other groups from being analyzed.

16.4.2 Heterogeneity index

To quantify the subpopulation structure, based on the GMM fit, FLIM Playground computes an “H-index” for each group as a weighted entropy-distance: each component contributes according to its weight’s uncertainty ($-w_i \log w_i$)¹³ scaled by how far its mean μ_i sits from the overall mixture mean $\bar{\mu} = \sum_{i=1}^k w_i \mu_i$, normalized by the standard deviation of μ_i s.

$$H = \sum_{i=1}^k (-w_i \log w_i) d_i, \quad d_i = \frac{|\mu_i - \bar{\mu}|}{\sigma}$$

16.4.3 Classification

For fits with more than one component, the app assigns each observation to one subpopulation using [hard assignment](#) by default. Select **Use intersection as threshold** to use [intersection thresholding](#) instead. Components are numbered in ascending order of their fitted means under either rule. A one-component fit has H-index 0 and leaves its rows unassigned, as do groups with insufficient data or no valid fit.

16.4.3.1 Export labeled data

Double-click a subpopulation’s **Name** in its component table to set the label used in the download. **Exported column name** defaults to GMM_group; if that column already exists, the app uses an available name such as GMM_group_2. Click **Download GMM Grouped Data** to save `gmm_grouped_data.csv`.

🔗 Double-click a subpopulation name to rename it.

0-control (H-index: 0.477)

#	≡ 🔗 Name	Mean ± SD	Weight
1	0-control_group1	857.94 ± 62.42	0.35
2	0-control_group2	1009.93 ± 49.67	0.65

Figure 16.4: Edit subpopulation names beside the fitted means, standard deviations, and weights.

Exported column name

Control lifetime population

Download GMM Grouped Data

Figure 16.5: Choose the exported label-column name before downloading the grouped data.

Default values look like 0.5hr_group1 and 0.5hr_group2. With `Separate by`, labels include the category, for example `{category}::{color_group}_group1`; without a color group they use `{category}_group1`. You can give multiple components the same exported name to combine their labels. Names must be nonempty, and the exported column name cannot overwrite another column. Naming changes affect the exported copy and script, while the fitted curves and parameters stay the same.

The CSV contains the analyzed observations after filtering and removal of missing values in the selected feature. It retains their available columns, including the analyzed values when **Log X** is enabled, plus the new label column. Unassigned rows have an empty label. The download is available when at least one group has generated labels.

On reupload, the label column follows the normal [column-role review](#); its name has no special reserved role. Assign it **Categorical** to use its labels in filters and grouping. The [exported Python script](#) retains the naming choices; set `SAVE_DERIVED_DATA = True` in the script to also save the labeled CSV.

16.4.3.2 Intersection thresholding

- Once the GMM is estimated, thresholds can be obtained by locating the intersection points of adjacent component densities:

$$w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) = w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

where w_1, μ_1, σ_1 are the mixture weight, mean, and standard deviation of the first Gaussian component, respectively, and w_2, μ_2, σ_2 those of the second component.

- The app searches between adjacent component means with Brent's root-finding method. When valid, increasing intersection thresholds are found, data points are assigned to the intervals they define. Weighted Gaussian components need not intersect between their means, so this rule is not always available.

! Important

If valid, increasing intersection thresholds cannot be found for a group, the app reports this and uses hard assignment for that group.

16.4.3.3 Hard assignment

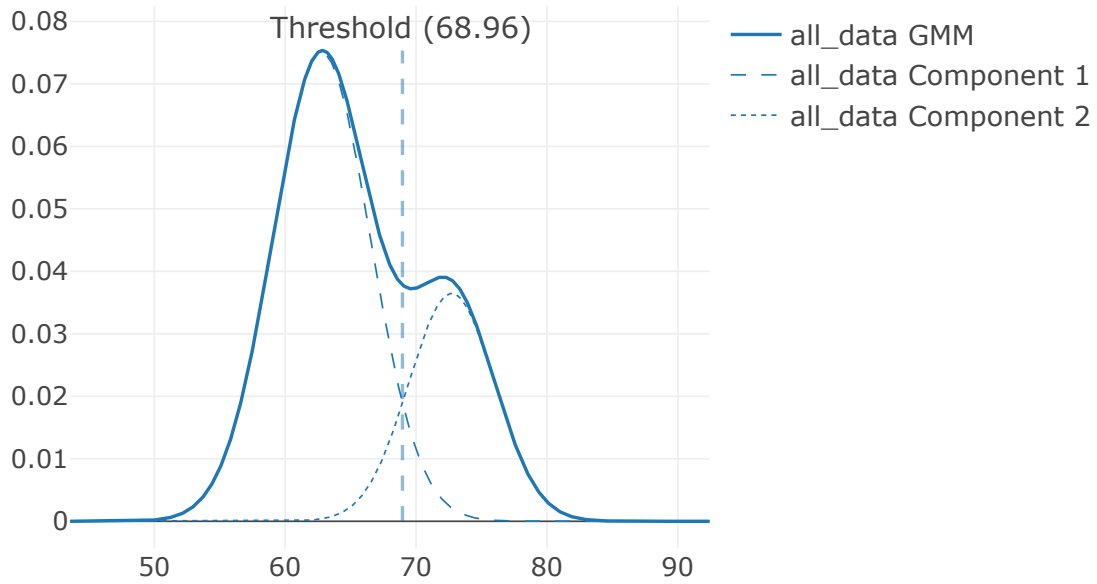
- For each data point, it computes the posterior probability (responsibility) of each component, and assigns the data point to the component with the highest responsibility.

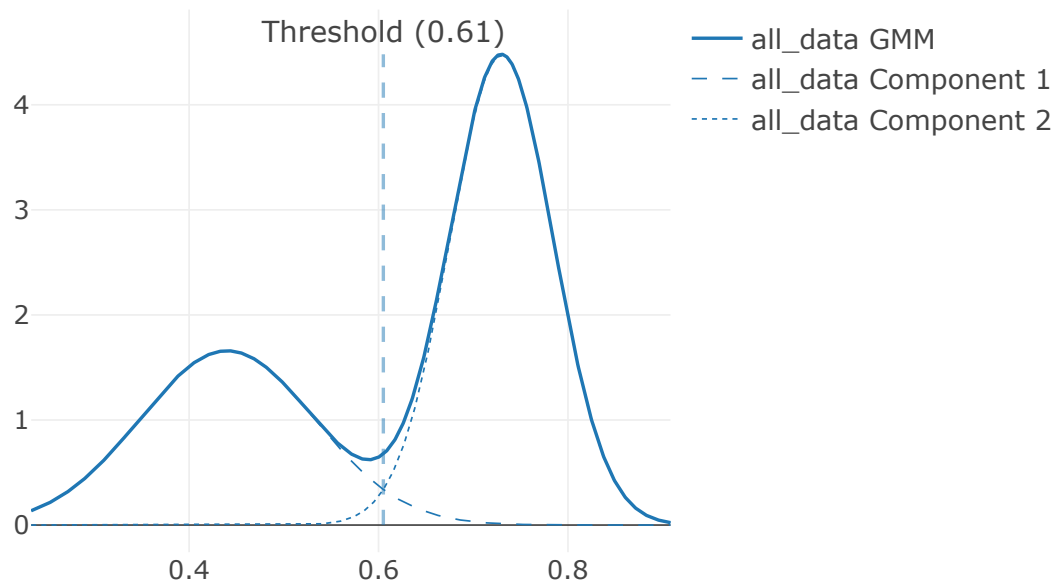
$$\text{label}(x) = \arg \max_i \gamma_i(x)$$

$$\text{where } \gamma_i(x) = \frac{w_i \mathcal{N}(x | \mu_i, \sigma_i^2)}{\sum_{j=1}^K w_j \mathcal{N}(x | \mu_j, \sigma_j^2)}$$

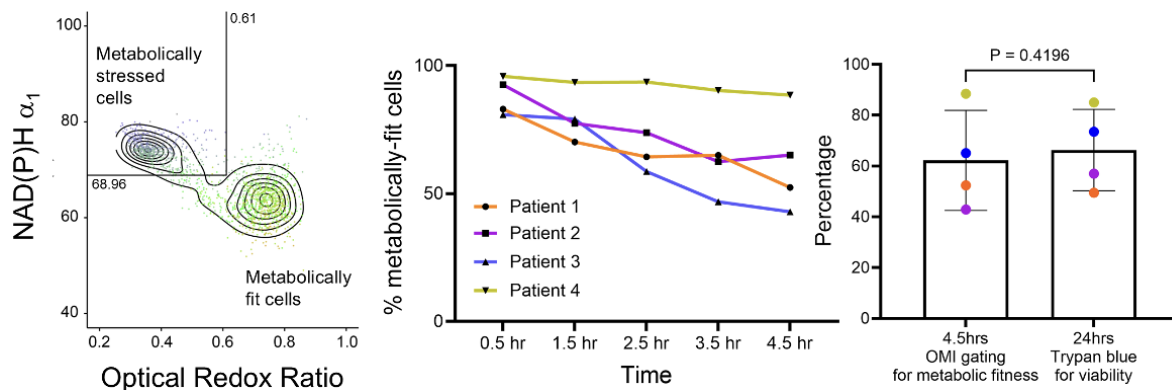
16.4.4 Example

Pham et al.¹⁴ used GMM with intersection thresholding to create gates using NAD(P)H a_1 and optical redox ratio. These gates were used to define metabolically stressed and fit cells in their study of the impact of cryopreservation on T cell metabolism.





They compared the GMM-derived gate with cell viability by Trypan blue staining:



It suggests that optical metabolic imaging (OMI) could serve as an early, non-destructive indicator of T cell viability post-thaw.

Part IV

Bivariate Analysis

17 2D Feature Distribution

This method visualizes the distribution of two numerical features on a scatter plot. It helps reveal patterns such as correlations, clusters, trends, or outliers.

17.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select the two numerical features on the x-axis and y-axis.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, choose **Color by**, [Separate by](#), and [Collapse by](#). The **Opacity | Shape** selector shares one categorical column picker; only the selected encoding is applied. These point encodings preserve the groups used for marginal distributions, correlation, regression, and GMM fitting.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

17.2 Log-scale Axes

Log X and **Log Y** transform the selected feature with $\log_{10}(x + 10^{-6})$ and relabel its axis $\log_{10}(\dots)$. The small offset allows zero values. The transformation happens before marginal distributions, Pearson correlation, regression, and 2D GMM fitting, so all of them use the transformed values. If a selected feature contains negative values, the app reports an error and keeps that feature on its original scale. With [Collapse by](#), it averages first and then transforms the category means.

17.3 Separate by

Choose one categorical column in **Separate by**, then choose a category above the plot. Its observations appear in color on a full-size scatter plot, with all other categories in gray for context. The same axes, colors, shapes, and opacities are used across category views. The separation column cannot also be used for **Color by** or **Collapse by**.

For the [inhibitor dataset](#), select **tm** under **Lifetime fit_nadh** for X and **tm** under **Lifetime fit_fad** for Y. Set **Separate by** → **cell_line** and **Color by** → **treatment**, keep the categorical filters at **All**, and leave

Collapse by blank. Select **Panc1** in **cell_line category** to highlight its cells by treatment, with MCF7 cells in gray.

Separate by Color by Collapse by Opacity

cell_line category

Marginal Plot Type ☐ Log X ☐ Log Y ☐ 2D Gaussian Mixture Model ☐ Regression line

Figure 17.1: Choose cell_line for separation, treatment for color, and Panc1 as the displayed category.

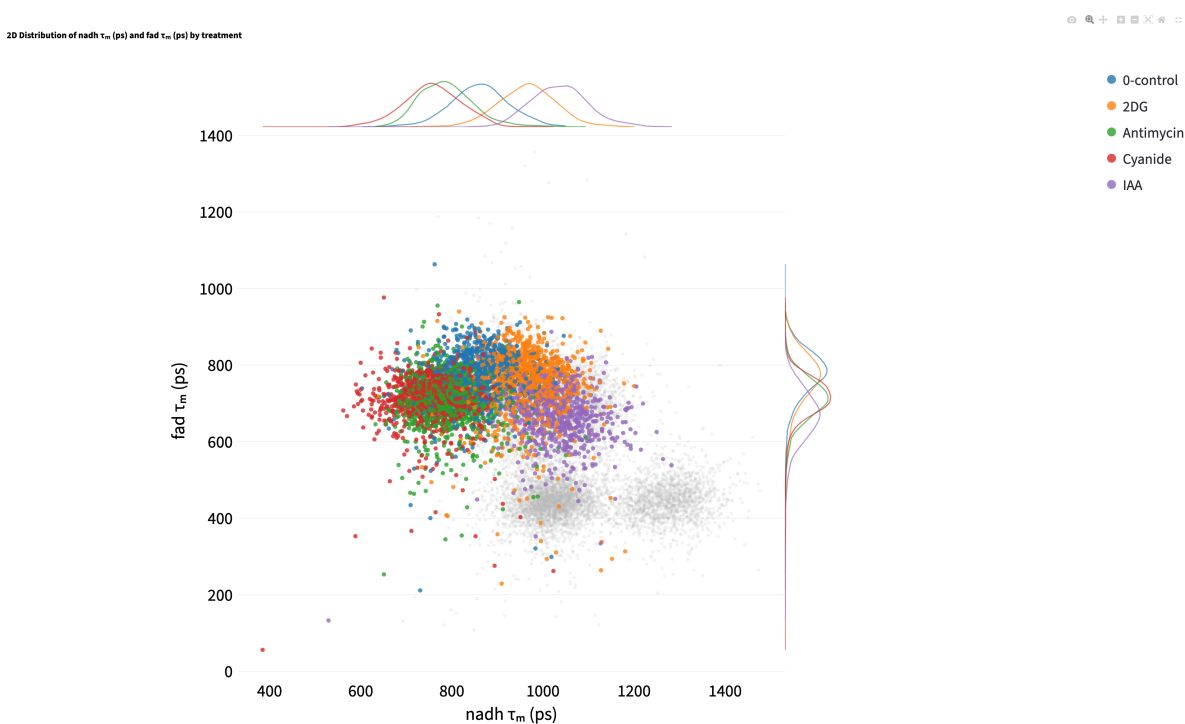


Figure 17.2: Panc1 cells are colored by treatment while MCF7 cells remain in gray; the marginal curves describe Panc1.

Select **MCF7** to switch the highlighted cells and their marginal distributions. Both category views keep the same axis ranges and treatment colors.

Marginal distributions, Pearson correlation, regression, and GMM fits are calculated independently within each separation category and color group. The displayed statistics and component tables describe the selected category. Switching the displayed category keeps those fits and the shared axis ranges; changing the separation column changes the populations being modeled. With **Show group counts (n)**

in legend, counts refer to the selected separation category's observations, or its aggregated points after collapse.

17.4 Collapse by

Choose a categorical column such as `dish`, `patient`, or `image_name` in **Collapse by** to plot one mean X/Y point per value of that column within each separation category and color group. The app removes rows missing either selected feature before averaging, so each point's X and Y means use the same observations. Hover text includes the collapse category's label and contributing observation count.

Marginal distributions, Pearson correlation, regression, and GMM fitting then use these category means with equal weight. A column is unavailable for collapse if it already defines **Color by** or **Separate by**. Other categorical columns remain available for point encoding only when their values are constant within every collapsed group; the app explains when it must disable a varying encoding.

A labeled GMM download after collapse contains one row per averaged group, with averaged numerical columns. It retains categorical metadata only when constant within every collapsed group. Clear **Collapse by** to return to individual observations.

17.5 Marginal Distribution

Marginal distributions (the distribution of a single feature) are plotted at the top and the right of the scatter plot. Users can choose the plot type from the dropdown menu, which offers three options.

- **gaussian fit**: provides a kernel-density estimate using Gaussian kernels. It uses `gaussian_kde` from `scipy.stats` and sets all parameters to default values.
- **boxplot**: summarizes each color group with a box and whiskers, which makes the median and spread easier to compare across groups than overlapping density curves.
- **violin**: shows the full distribution shape for each color group, combining the density information of the Gaussian fit with the per-group separation of the boxplot.

17.6 2D Gaussian Mixture Model

Select **2D Gaussian Mixture Model** above the plot to fit a GMM to each color group, separately within every category when **Separate by** is active. As in the [1D GMM](#), **Max Components** and **Min Weight Threshold** control candidate fits; the valid model with the lowest BIC is chosen.

Ellipses that capture approximately 95% of the points in each subcomponent are drawn. The color of the ellipses is determined by the color group that the subcomponent belongs to.

i How ellipses are drawn

Each ellipse represents a subcomponent of the GMM.

- Center of the ellipse is the mean of the subcomponent (μ_x, μ_y) .
- The ellipse is rotated by $\theta = \arctan 2(v_{1y}, v_{1x})$, where $\mathbf{v}_1 = \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix}$ is the first eigenvector of the covariance matrix, so that the major axis of the ellipse is aligned with the eigenvector corresponding to the largest eigenvalue.
- The semi-axis lengths are $r\sqrt{\lambda_1}$ and $r\sqrt{\lambda_2}$, where λ_1 and λ_2 are the covariance eigenvalues (variances along the principal axes) scaled by $r = \sqrt{\chi^2_{df=2, p=0.95}}$ (Mahalanobis distance) that captures approximately 95% of the points in the subcomponent.

The plot shows ellipses for each fitted component in the displayed category and color group.

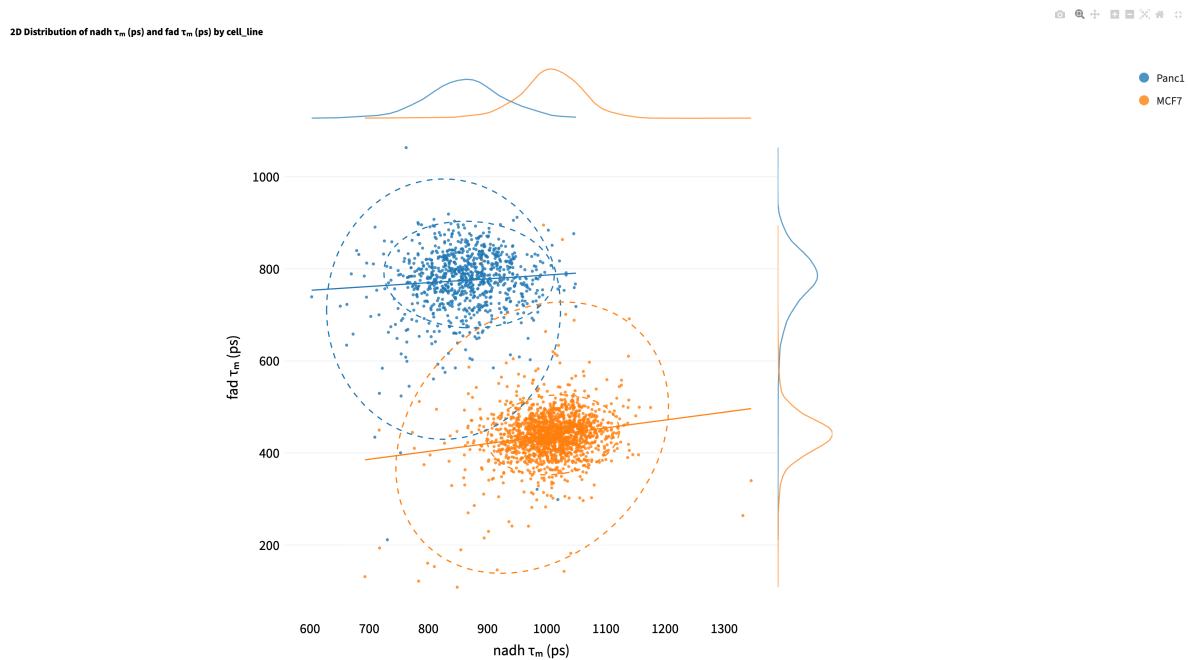


Figure 17.3: The full-screen plot displays marginal densities, fitted GMM ellipses, and regression lines together.



Tip

Click a legend group to hide or show its points and associated curves. This can make overlapping groups easier to inspect.

Use the chart's **Fullscreen** control to inspect a tall plot with both marginal distributions visible.

Tables below the plot report each component's **X (mean $\pm$ SD)**, **Y (mean $\pm$ SD)**, and **Weight**. These describe the fitted Gaussian components using soft membership probabilities. They can differ from the mean, SD, and fraction of observations assigned to each component by the hard assignment rule.

Panc1: Pearson $r = 0.08$ ($p = 0.0212$) · Regression $R^2 = 0.006$ (slope = 0.083, intercept = 703.634)

MCF7: Pearson $r = 0.17$ ($p = 7.96e-13$) · Regression $R^2 = 0.030$ (slope = 0.171, intercept = 266.880)

🔗 Double-click a subpopulation name to rename it.

Panc1

#	🔗 Name	X (mean $\pm$ SD)	Y (mean $\pm$ SD)	Weight
1	Panc1_group1	825.73 $\pm$ 80.78	712.52 $\pm$ 115.55	0.17
2	Panc1_group2	868.69 $\pm$ 58.82	787.56 $\pm$ 47.36	0.83

MCF7

#	🔗 Name	X (mean $\pm$ SD)	Y (mean $\pm$ SD)	Weight
1	MCF7_group1	975.55 $\pm$ 94.13	433.29 $\pm$ 120.43	0.13
2	MCF7_group2	1011.01 $\pm$ 45.61	439.34 $\pm$ 35.51	0.87

Figure 17.4: Results below the plot report correlation and regression summaries together with editable GMM component tables.

17.6.1 Export labeled data

The app uses [hard assignment](#) to give each analyzed point the label of its highest-probability component. A component's number identifies it within that fitted model; unlike the 1D GMM, the 2D components are not ranked by increasing mean. Groups with too little data, constant X or Y, no valid model, or only one fitted component remain unassigned.

Double-click a subpopulation's **Name** in the component table to rename it, and use **Exported column name** to name the new label column. The default is `2D_GMM_group`, with an available suffix such as `_2` when needed to preserve an existing column. Click **Download 2D GMM data** to save `2D_gmm_data.csv`.

Exported column name

2D_GMM_group

Download 2D GMM data

Figure 17.5: The download adds the named GMM assignment column to the analyzed data.

Default labels are `{color_group}_group1`, `{color_group}_group2`, and so on. With `Separate by`, they include the category as `{category}::{color_group}_group1`, or `{category}_group1` if `Color by` is empty. Giving several components the same name combines their exported labels without changing the fit. The [naming rules](#) are shared with the 1D GMM.

The CSV includes all analyzed categories, even when only one is displayed. It contains rows passing the filters with both selected measurements present, plus the label column; after collapse each row represents one averaged group. Selected measurements retain any logarithmic transformation used in the analysis. Unassigned rows have empty labels, and a download is offered when at least one group has generated labels.

On reupload, review the label column and assign it **Categorical** to use it for filtering and grouping. The [exported Python script](#) records the fit, category view, collapse, and naming settings. Set `SAVE_DERIVED_DATA = True` in the script to also write the labeled CSV.

17.7 Correlation coefficient

Pearson's correlation coefficient r and its p-value are reported below the plot for each color group in the current category, even when **Regression line** is cleared. Scroll below the scatter plot to find the results. The coefficient ranges from -1 to 1 : its sign gives the direction of a linear association, and values near zero indicate a weak linear association.

Panc1: Pearson $r = 0.08$ ($p = 0.0212$) · Regression $R^2 = 0.006$ (slope = 0.083, intercept = 703.634)

MCF7: Pearson $r = 0.17$ ($p = 7.96e-13$) · Regression $R^2 = 0.030$ (slope = 0.171, intercept = 266.880)

Figure 17.6: The output reports Pearson r and its p-value separately from the regression statistics.

Correlation and regression use the selected category and color groups, after filtering and any [log transforms](#). Both use category means when `Collapse by` is active. A group needs at least two points and variation in both X and Y ; otherwise the app reports why correlation and regression are unavailable.

17.8 Regression line

Select **Regression line** to add a linear fit and report its R^2 , slope, and intercept alongside the correlation results.

R^2 tells how much of the variability in y is captured by the model, compared to just predicting the $\bar{y}$.

i R^2

- $R^2 = 1 \rightarrow$ perfect prediction.
- $R^2 = 0 \rightarrow$ model predicts no better than the mean.

18 Phasor Plot

Phasor Plot displays a matching pair of first- or second-harmonic phasor coordinates against the universal semicircle. Use it to compare the positions and spread of groups measured with the [Lifetime fit free](#) extractor, or from a user table with the expected coordinate names.

i Phasor Plot appears only for datasets with phasor coordinates

After a dataset is loaded, **Phasor Plot** is **hidden from the Bivariate method selector** if the **dataset has no recognized, complete G/S coordinate pair**. A pair must belong to the same channel and harmonic, and both columns must be numerical. See [coordinate names and column roles](#) below.

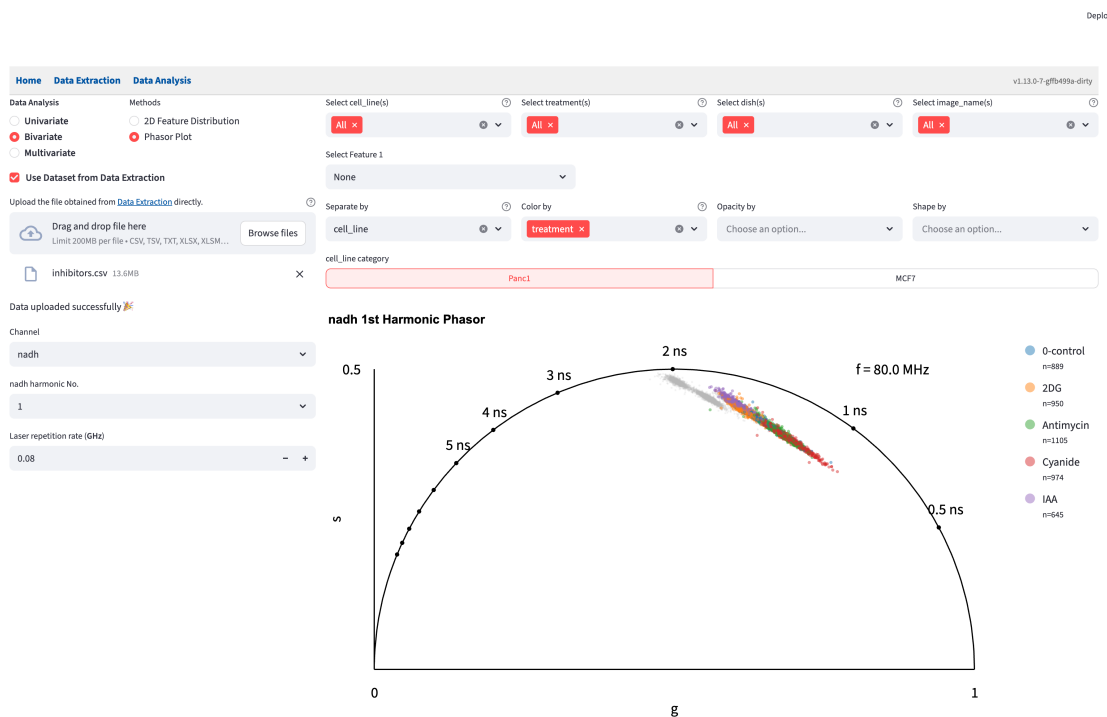


Figure 18.1: The complete Phasor Plot screen, with cell-line selection and treatment colors.

18.1 Shared Interface Components

- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, use **Color by**, **Opacity by**, and **Shape by** to encode categorical features. [Separate by](#) adds a category selector above the plot.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

18.2 Select Phasor Coordinate

On the left, select **Channel** when more than one channel is present, then choose its **harmonic No.** Only harmonics with both G and S coordinates are offered. A table containing just one complete harmonic pair can still use Phasor Plot.

The image shows a user interface for selecting parameters for a Phasor Plot. It consists of three vertically stacked controls:

- Channel:** A dropdown menu with a light blue background and rounded corners. The text "nadh" is displayed in the center, and a small downward arrow is on the right.
- nadh harmonic No.:** A dropdown menu with a light blue background and rounded corners. The text "1" is displayed in the center, and a small downward arrow is on the right.
- Laser repetition rate (GHz):** A control with a light blue background and rounded corners. It displays the value "0.08" in the center. To the right of the value are two small icons: a minus sign "-" and a plus sign "+", indicating a range or increment/decrement function.

Figure 18.2: Select the channel and harmonic for the matching G/S coordinate pair.

The method recognizes numerical columns named `Lifetime fit free_{channel}: G(1st)` and `Lifetime fit free_{channel}: S(1st)`, or their `G(2nd)` and `S(2nd)` counterparts. For example, a first-harmonic NADH pair is `Lifetime fit free_nadh: G(1st)` and `Lifetime fit free_nadh: S(1st)`. In a user-table review, assign both columns **Numerical**; they may belong to different feature groups. After upload, Phasor Plot is available only if the table contains a complete pair.

Set **Laser repetition rate (GHz)** to the acquisition's base repetition rate, for example 0.08 for 80 MHz. This controls the lifetime reference markers; it does not recalculate or calibrate the uploaded coordinates. The app removes rows missing either coordinate from the displayed plot.

The plot is drawn against the universal semicircle, with markers along it for lifetimes from 0.5 to 10 ns to read your points against. Switching to the 2nd harmonic moves those markers, because the transform

is evaluated at twice the frequency and a given lifetime lands somewhere else. The frequency printed inside the plot tells you which view you are reading — $f = 160 \text{ MHz}$ ($2 \times 80 \text{ MHz}$) for the 2nd harmonic of an 80 MHz system — so check it before comparing a point against a marker. The semicircle itself is the same for both harmonics.

18.3 Separate by

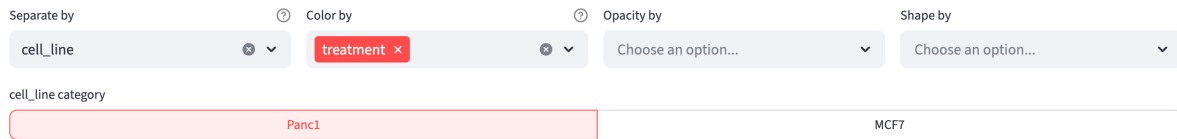


Figure 18.3: Separate by adds a category selector while color, opacity, and shape remain independent controls.

Choose `cell_line` in **Separate by** and `treatment` in **Color by**, then select a cell line above the plot. For example, select `Panc1` to highlight its treatments while keeping `MCF7` observations in gray for context. The selected category keeps its color, opacity, and shape encodings; observations from other categories remain visible as faint gray context points. The full-size axes and lifetime references remain the same as you switch categories.

The separation column cannot also be used for **Color by**. Point colors stay consistent across categories, and **Show group counts (n) in legend** counts only the selected category's complete G/S observations. Gray context points do not contribute to those counts or provide individual hover labels.

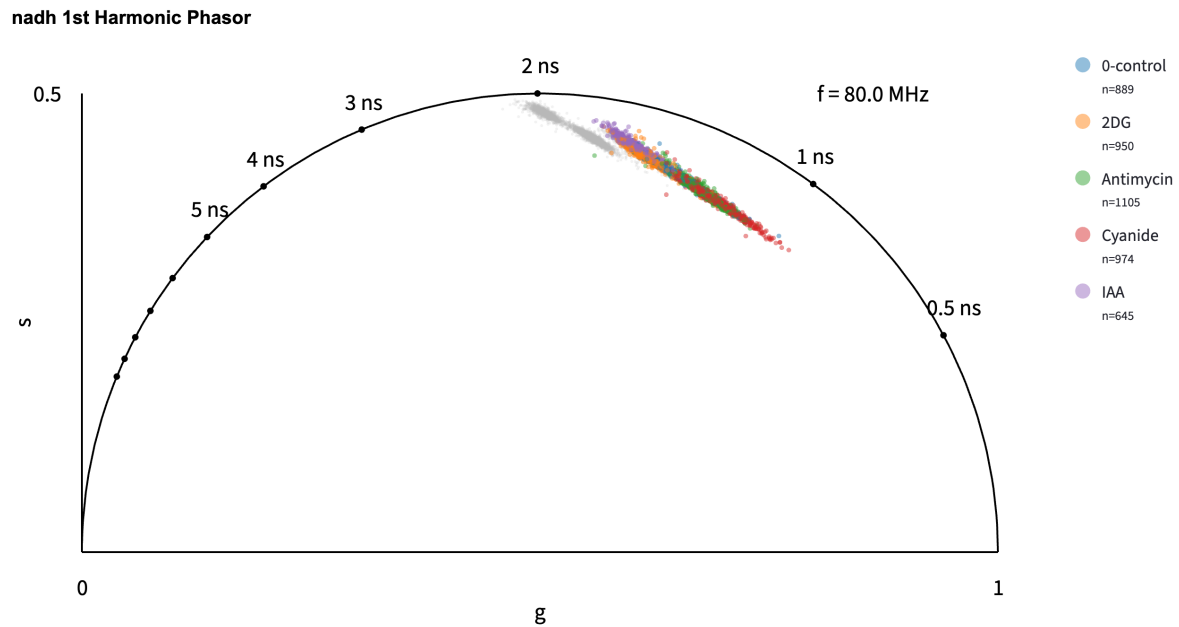


Figure 18.4: Treatments in the selected cell line appear in color against gray observations from the other cell line.

18.4 Export the plot

Click **Export the entire analysis as a Python script** below **Plot Styling** to reproduce the current channel, harmonic, repetition rate, filters, encodings, and selected category. Follow the [shared script workflow](#) to generate an SVG figure from the original table.

Part V

Multivariate Analysis

19 Dimension Reduction

To represent and visualize high-dimensional data in a meaningful way and help users interpret the data, dimension reduction methods project the data into a lower-dimensional space. Three popular dimension reduction methods are often used: UMAP, PCA, and t-SNE.

Principal Component Analysis (PCA) is a linear dimension reduction method that projects the data onto the directions of the largest variance. It is fast, deterministic, and easier to interpret (the percentage of variance explained by each principal component is reported on the axis labels), but it is limited by the linear nature of the projection. For a detailed discussion of PCA, please refer to [this post](#).

Uniform Manifold Approximation and Projection (UMAP) and t-Distributed Stochastic Neighbor Embedding (t-SNE) are non-linear dimension reduction methods that can capture more complex patterns in the data. They are more flexible and can handle non-linear relationships between features, but they are more computationally expensive, less interpretable, not deterministic (FLIM Playground uses the random seed 42 to ensure reproducibility), and dependent on the hyperparameters. Here is a [post](#) that explains how UMAP and t-SNE work at a high level and the meaning of some of the hyperparameters.

Select at least two numerical features. The app removes rows missing any selected feature, standardizes each feature to a z-score, and fits one two-dimensional embedding to the remaining filtered observations.



Figure 19.1: UMAP of the 0-control and IAA treatments, colored by cell_line, with Separate by left empty.

19.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select multiple numerical features from each feature group.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, use Color by, Opacity by, and Shape by to encode categorical features, and [Separate by](#) to add category highlight maps.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

19.2 Separate by

Choose up to two categorical columns in **Separate by** to display a shared overview beside smaller category maps:

- One column creates a vertical stack of maps, one per category.
- Two columns create a matrix: the first selected column defines rows and the second defines columns. A combination with no observations still has a panel showing the gray context.

Each small map highlights its category's observations with the same coordinates, colors, shapes, and opacities as the overview. Other observations appear in faint gray. All maps share axis ranges, so positions can be compared directly. A column may be selected in both **Separate by** and **Color by**.

Changing these display controls does not refit the embedding. Changing the numerical features, filters, reduction method, or its hyperparameters does. The shared legend controls groups across the maps, and **Show group counts (n) in legend** reports counts from the full embedding, rather than separate counts for each small map.

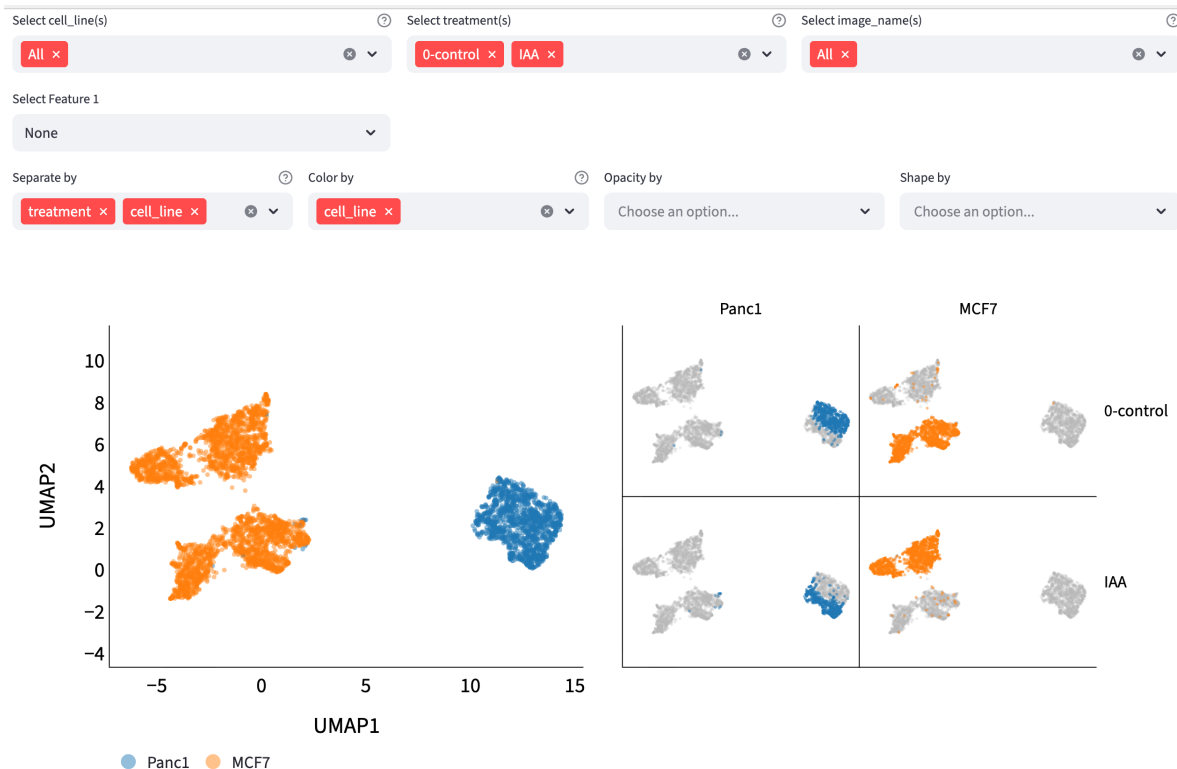


Figure 19.2: Select **treatment** and then **cell_line** in **Separate by** to show the same UMAP embedding with treatments in rows and cell lines in columns.

19.3 Hyperparameter Widget

Choose **UMAP**, **PCA**, or **t-SNE** in **Dimension Reduction Method** on the left. UMAP and t-SNE expose the controls below; PCA has no additional hyperparameter controls.

19.3.1 UMAP

FLIM Playground uses the `umap-learn` implementation of UMAP. **n_neighbors** (default 15) controls the neighborhood size used to construct the embedding, while **min_dist** (default 0.1) controls how tightly nearby points can be packed. Other hyperparameters use the implementation's [default values](#), with Euclidean distance and a fixed random seed of 42.

19.3.2 t-SNE

FLIM Playground uses the `sklearn.manifold.TSNE` implementation of t-SNE. **perplexity** (default 15) controls the effective neighborhood size and must be smaller than the number of analyzed observations. **early_exaggeration** (default 1 in the app) controls the initial emphasis on keeping neighboring observations together. Other hyperparameters use the implementation's [default values](#), with a fixed random seed of 42.

19.4 Export the analysis

Click **Export the entire analysis as a Python script** below **Plot Styling** to retain the selected features, filters, method, hyperparameters, point encodings, and category-map layout. The script fits the same shared embedding and saves an SVG figure. See the [shared script workflow](#) for running it with the original table.

20 Classification

A machine learning classifier learns patterns from categorized data and then predicts the category of new, unseen data. This module provides a set of machine learning classifiers and interactive widgets to help users perform classification.

All classifiers use `scikit-learn`, with a fixed random seed of 42 for reproducibility. The app evaluates predictions on a held-out test set and reports [performance metrics](#), a [confusion matrix](#), and [ROC curves](#).

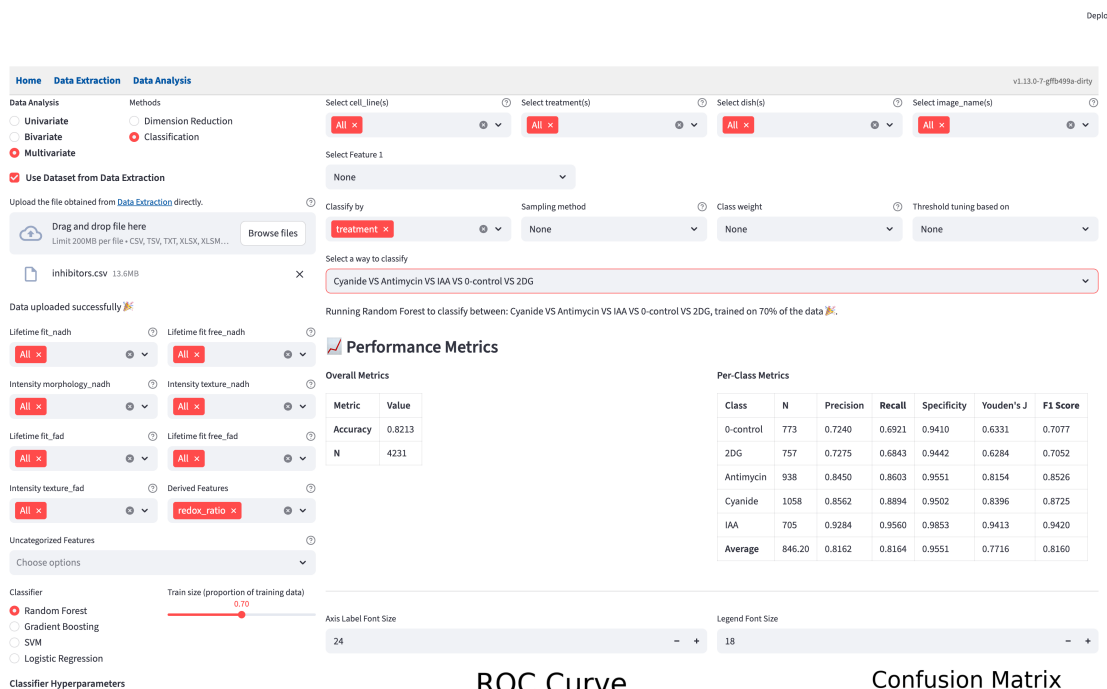


Figure 20.1: Choose features, a classifier, and target classes, then inspect results on the held-out test set.

An overview of the pipeline:

- Choose one or more numerical features on the left.
- Choose the classifier, train-test split, and [hyperparameters](#).
- Filter the dataset.
- Choose categorical features to form classes.

- Choose any of the methods to handle [class imbalance](#).
- Choose the classes to be classified.
- Look at the [performance metrics](#), [confusion matrix](#), and [ROC curve](#).
- Compare the results using the metrics that matter for your classification task, then [export the analysis](#) to reproduce it.

SVM and Logistic Regression standardize features with a z-score within their training pipeline; test observations use that training-fitted transformation. Random Forest and Gradient Boosting use the original feature scale. Classification does not automatically remove rows with missing selected measurements, so a classifier that cannot accept them reports an error. Clean those measurements or use the [numerical filters](#) to select complete observations.

20.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select numerical features for the classifier. Multiple features can be selected from each feature group.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Font-size controls appear between the performance metric tables and the ROC/confusion-matrix plots. They use the shared [plot styling settings](#).

20.2 Method-specific Components

- Choose a **Classifier** and **Train size (proportion of training data)** on the left. The slider ranges from 0.5 to 0.9 in steps of 0.1, with a default of 0.7. The remaining observations form the held-out test set. Available classifiers are:
 - [Random Forest](#)
 - [Gradient Boosting](#)
 - [Support Vector Machine \(SVM\)](#)
 - [Logistic Regression](#)

The [train-test split](#) is stratified to preserve class proportions. The app does not create a separate validation set for choosing model hyperparameters; those are set with the controls below the classifier. Optional threshold tuning uses cross-validation within the training data.

 The split is by row, not by field of view

Both the train-test split and the cross-validation used in threshold tuning operate on individual rows (i.e. single cells), stratified by class. They do not group rows by field of view, so cells segmented from the same image can end up in both the training set and the test set. Cells from one image are usually more similar to each other than to cells from a different image,

because they share a field of view, an acquisition session, and often a well. When such cells appear on both sides of the split, the reported metrics can look better than the classifier’s true performance on a genuinely new image. Keep this in mind when interpreting the [performance metrics](#), particularly when the dataset contains many cells drawn from relatively few fields of view.

- **Classify by** forms target classes from one or more categorical columns. With multiple columns, each distinct combination becomes a class, with values joined by underscores. Classification requires at least one categorical column with two distinct values after filtering. The designated field-of-view column in extraction data is excluded from this picker; a user-table field-of-view column is an ordinary categorical column.
- **Select a way to classify** chooses the classes included in a run. With two classes, the app offers their binary comparison. With more classes, it offers combinations of two or more classes, including the full multiclass problem, and a `class VS the rest` option for each class. A one-versus-rest choice combines all other classes into the `rest`. If the number of combinations is too large, the app asks you to narrow the categories or filters.

The image shows a user interface for configuring a classification task. At the top, there are four dropdown menus: 'Classify by' (with a red 'x' icon and a plus icon), 'Sampling method' (with a plus icon), 'Class weight' (with a plus icon), and 'Threshold tuning based on' (with a plus icon). All four are currently set to 'None'. Below these is a section titled 'Select a way to classify' with a dropdown menu showing 'Cyanide VS Antimycin VS IAA VS 0-control VS 2DG'.

Figure 20.2: Target and class-imbalance controls define the classification task.

20.2.1 Classifier Hyperparameters

The main hyperparameters appear below the classifier; open **Advanced settings** for additional controls. The tables below describe parameters using three roles:

- **Model-Form/Capacity**: defines the hypothesis family or expressive power.
- **Regularization**: constrains model complexity to reduce overfitting.
- **Optimization**: affects training convergence and runtime behavior.

For Random Forest, **max_depth (0=None)** uses 0 for an unrestricted depth. SVM shows **degree** only for the polynomial kernel and **coef0** for polynomial or sigmoid kernels. Logistic Regression updates its regularization choices when the solver changes and shows **l1_ratio** for saga with `elasticnet`.

20.2.1.1 Random Forest

Parameter table

Param	Role	Effect	Priority
n_estimators	Model-Form/Capacity	More trees increase ensemble capacity; gains usually show diminishing returns after enough trees.	High
max_depth	Regularization	Caps tree depth to reduce overfitting; also directly limits capacity.	High
min_samples_split	Regularization	Requires more samples before splitting, reducing fragmentation on small partitions.	Medium
min_samples_leaf	Regularization	Enforces minimum leaf size, smoothing predictions.	Medium
max_features	Regularization	Fewer features per split increase tree diversity and reduce variance/overfitting.	Medium-High

20.2.1.2 Gradient Boosting

Parameter table

Param	Role	Effect	Priority
n_estimators	Model-Form/Capacity	More stages increase additive model capacity; strongly interacts with learning_rate.	High

learning_rate	Regularization	Shrinks each stage contribution; lower values typically need more trees and often generalize better (also affects optimization dynamics).	High
max_depth	Regularization	Limits base learner complexity; also limits capacity.	High
subsample	Regularization	< 1.0 introduces stochastic boosting, reducing variance and overfitting risk.	High

20.2.1.3 Support Vector Machine (SVM)

i Parameter table			
Param	Role	Effect	Priority
C	Regularization	Higher C penalizes training errors more, which usually weakens regularization.	High
kernel	Model-Form/Capacity	Chooses hypothesis family (linear vs non-linear kernels).	High
gamma	Model-Form/Capacity	Controls influence radius of samples for rbf/poly/sigmoid; higher values create tighter boundaries.	High
degree	Model-Form/Capacity	Polynomial order for poly kernel; higher degree increases flexibility.	Medium-High

<code>coef0</code>	Model-Form/Capacity	Kernel offset for poly/sigmoid, shifting boundary behavior.	Medium
<code>tol</code>	Optimization	Convergence tolerance; mostly affects runtime/termination behavior.	Low

20.2.1.4 Logistic Regression

i Parameter table			
Param	Role	Effect	Priority
<code>regularization</code>	Regularization	Selects regularization type; available options depend on solver (see table below).	High
<code>C</code>	Regularization	Inverse regularization strength; higher C means weaker regularization.	High
<code>l1_ratio</code>	Regularization	Balances l1 vs l2 when solver='saga' and regularization='elasticnet'.	Medium
<code>fit_intercept</code>	Regularization	Disabling intercept is a mild structural constraint.	Low
<code>solver</code>	Optimization	Optimization algorithm; affects convergence speed and which regularization options are available.	Low-Medium

<code>max_iter</code>	Optimization	Iteration budget; too low can stop before convergence.	Low-Medium
<code>tol</code>	Optimization	Convergence threshold; mainly runtime-sensitive.	Low

Regularization options by solver:

Solver	Available regularization
<code>lbfgs</code>	12, none
<code>liblinear</code>	11, 12
<code>newton-cg</code>	12, none
<code>newton-cholesky</code>	12, none
<code>sag</code>	12, none
<code>saga</code>	11, 12, elasticnet, none

20.2.2 Class Imbalance Handling

We provide three options for handling class imbalance (i.e., one or more classes in a dataset have significantly fewer samples than others, which would otherwise cause a classifier to bias toward the majority class). Note: to prevent data leakage, all the methods below are applied to the training set only.

- **Sampling methods:**

- None: stratified random sampling of the data on classification classes, regardless of the class size. It is implemented using `train_test_split` from `sklearn.model_selection`.
- Undersampling: All classes are downsampled to the size of the smallest class. It is implemented using `RandomUnderSampler` from [imbalanced-learn](#).
- Oversampling: All classes are upsampled to the size of the largest class by randomly duplicating samples from minority classes. It is implemented using `RandomOverSampler` from `imbalanced-learn`.

- **Class weights:**

- None: no class weights.
- Balanced: the class weights are inversely proportional to the class size (support). Note: Gradient Boosting does not support class weights in the scikit-learn implementation, and thus this option is not available for it.

- **Threshold tuning:** Whereas the previous two options influence the probability estimation part of the classifier, this option adjusts only the decision boundary used to convert predicted probabilities into class labels.
 - None: uses the classifier's default decision rule without tuning a threshold.
 - Balanced Accuracy: the threshold is set to maximize the balanced accuracy score, which is the unweighted average of each class's recall (i.e., the minority class's recall is as important as the majority class's recall).
 - F1 Score: the threshold is set to maximize the unweighted average of each class's F1 score.

i How does threshold tuning work?

It treats threshold selection as a separate optimization problem from model training, allowing thresholds to be tuned independently of the underlying classifier's probability estimates.

K-fold cross-validation for the probability model: The procedure uses a stratified **k-fold cross-validation** scheme (default $k=5$) within the training data, after any selected resampling. That training data is divided into k mutually exclusive subsets; for each fold, one subset serves as validation while the other $k-1$ subsets train the probability model. The held-out test set is not used to choose the thresholds.

The probability models are trained separately for each cross-validation fold. This process yields k distinct probability models, each trained on a different subset of the data, along with their corresponding validation set probability predictions that collectively cover all samples exactly once.

Threshold optimization: A single threshold optimization procedure is performed once using all cross-validation folds. The threshold optimization process differs for binary and multi-class classification problems. For binary classification, a single threshold parameter $\theta \in [0, 1]$ is optimized, where class 1 is predicted when $P(\text{class 1} \mid x) \geq \theta$, and class 0 otherwise. For multi-class problems, a threshold vector $\theta = [\theta_1, \theta_2, \dots, \theta_k]$ is optimized simultaneously for all k classes, where thresholds are applied through a normalized probability scheme: predictions are made by selecting the class with the highest normalized probability $P(\text{class } i \mid x) / (\theta_i + \epsilon)$, where $\epsilon = 10^{-10}$ prevents division by zero.

The optimization objective function evaluates candidate thresholds by: (1) applying the same threshold(s) on top of the fold-specific probability models to generate prediction classes for the fold-specific validation set, (2) computing the specified evaluation metric (e.g., balanced accuracy, F1-score) between predicted and true labels for each fold, and (3) returning the negative mean cross-validated score across all folds (for minimization using the Nelder-Mead simplex algorithm). Finally, the optimized threshold(s) are applied on top of the probability model trained on the entire training set to the test set to generate the final predictions.

Interpreting class-imbalance settings

Keep the features, classifier, hyperparameters, and train size fixed when comparing class-imbalance settings. Sampling and class weights change model training, so they can change the predicted probabilities and ROC curve. Changing only the decision threshold preserves the probabilities and ROC curve, but can move the operating point and change the confusion matrix.

Lowering a binary class's probability threshold makes it easier to assign that class. Compare recall, specificity, precision, and accuracy to understand the trade-off. Thresholds are optimized within the training data; improvement on the held-out test set is not guaranteed. Choose the settings according to the metrics most important to the task.

20.2.3 Confusion Matrix

FLIM Playground evaluates the trained classifier on the held-out test set using its default decision rule or the tuned probability thresholds. The resulting predicted labels are summarized with `sklearn.metrics.confusion_matrix`. In the binary case, it is represented as:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP) – correct hits	False Negative (FN) – missed positives
Actual Negative	False Positive (FP) – false alarms	True Negative (TN) – correct rejections

20.2.4 Performance Metrics

The **Overall Metrics** table reports accuracy and **N**, the number of held-out test observations. **Per-Class Metrics** reports each class's test-set support and the measures below, with an unweighted average across classes. Recall is highlighted when threshold tuning uses Balanced Accuracy, and F1 Score is highlighted when it uses F1 Score.

Overall accuracy is the number of correct predictions divided by the number of test observations: the sum of the confusion matrix's diagonal divided by the sum of all its entries. For binary classification, this is:

- Accuracy: $\frac{TP+TN}{TP+FP+TN+FN}$

Other metrics are calculated for each class based on the confusion matrix:

- Precision: $\frac{TP}{TP+FP}$
- Recall (Sensitivity): $\frac{TP}{TP+FN}$
- Specificity: $\frac{TN}{TN+FP}$
- Youden's Index: Sensitivity + Specificity – 1
- F1 Score: $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Performance Metrics

Overall Metrics

Metric	Value
Accuracy	0.8213
N	4231

Per-Class Metrics

Class	N	Precision	Recall	Specificity	Youden's J	F1 Score
0-control	773	0.7240	0.6921	0.9410	0.6331	0.7077
2DG	757	0.7275	0.6843	0.9442	0.6284	0.7052
Antimycin	938	0.8450	0.8603	0.9551	0.8154	0.8526
Cyanide	1058	0.8562	0.8894	0.9502	0.8396	0.8725
IAA	705	0.9284	0.9560	0.9853	0.9413	0.9420
Average	846.20	0.8162	0.8164	0.9551	0.7716	0.8160

Figure 20.3: Overall and per-class metrics summarize predictions on the held-out observations.

💡 Performance Metrics Example

The reason that we need metrics in addition to accuracy is because *Accuracy is Not Enough*. For a visual explanation of metrics, see [here](#). In short, it is all about trade-off.

20.2.5 ROC Curve

Another way to look at the performance of the classifier is to plot the Receiver Operating Characteristic (ROC) curve and its Area Under the Curve (AUC).

The confusion matrix summarizes one decision rule, while the ROC curve shows how true-positive and false-positive rates change as the probability threshold varies.

At a threshold above the highest predicted probability, every observation is classified as negative, so the false-positive rate $FPR = \frac{FP}{FP+TN}$ and true-positive rate $TPR = \frac{TP}{TP+FN}$ are both 0. At a threshold of 0, every observation is classified as positive, giving the endpoint (1, 1).

AUC summarizes the area under the ROC curve. It is the probability that a randomly chosen positive observation receives a higher score than a randomly chosen negative observation, with tied scores contributing one-half.

The ROC curve displays the operating point (FPR, TPR) associated with the confusion matrix on the right.

In a binary problem, the curve's legend names the class treated as positive. Read the displayed threshold in terms of that class's probability; the app converts it to match the class shown on the curve.

For a visual explanation, see [here](#).

20.2.5.1 Multi-class ROC Curve

In multi-class classification, each class's ROC curve is computed as a one-vs-rest binary classification problem. The operating point (FPR, TPR) associated with the confusion matrix is displayed for each class.

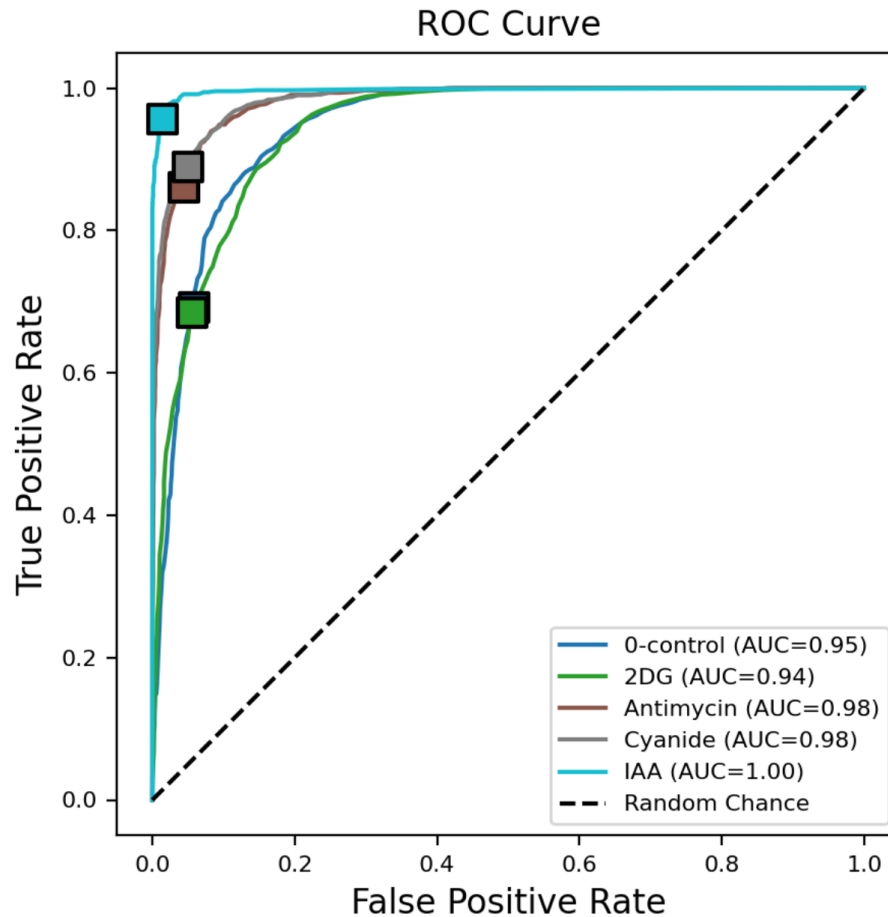


Figure 20.4: One-versus-rest ROC curves show each treatment's discrimination and operating point.

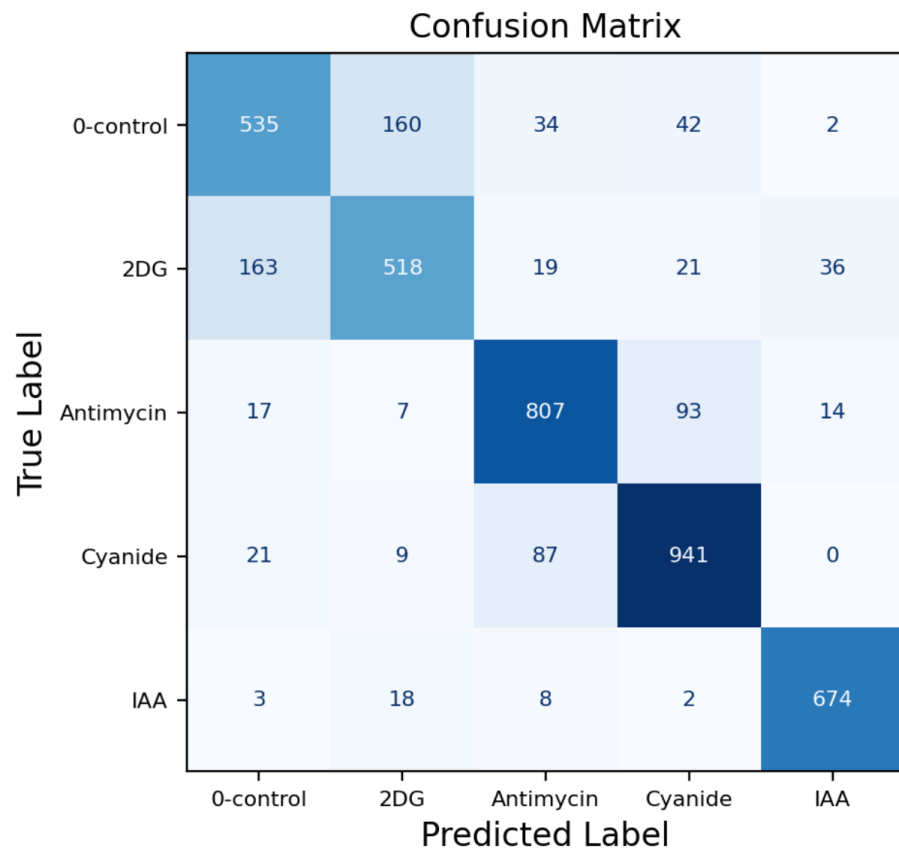


Figure 20.5: The confusion matrix compares true and predicted labels for the same test observations.

20.2.6 Feature Importance

Random Forest and Gradient Boosting classifiers provide feature importance scores.

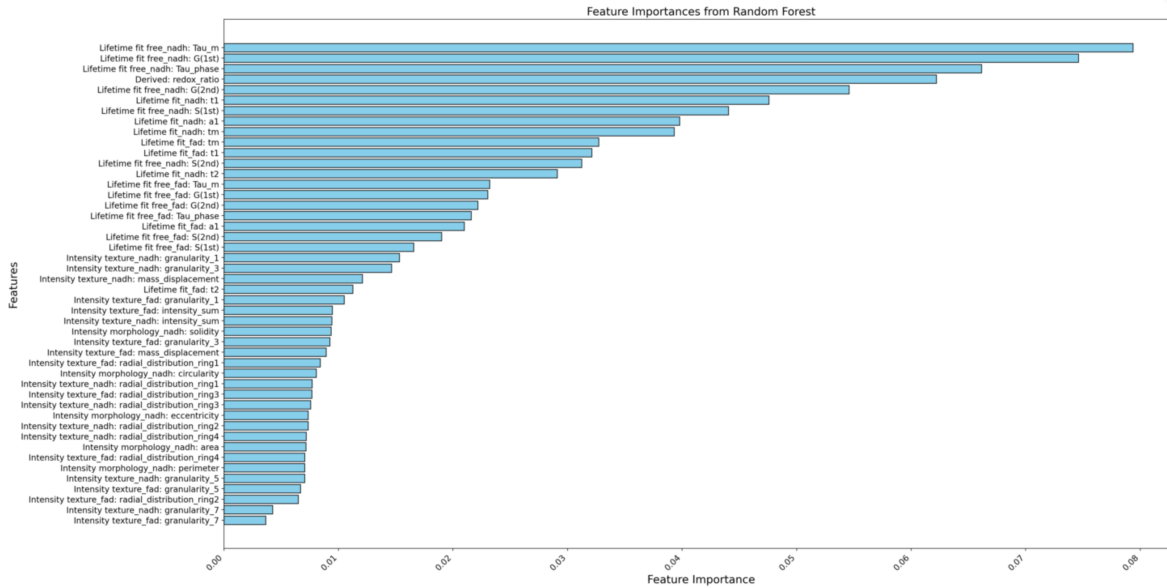


Figure 20.6: The fitted Random Forest assigns an importance score to each selected numerical feature.

20.3 Export the analysis

Click **Export the entire analysis as a Python script** below the results to retain the selected features, filters, target classes, classifier, hyperparameters, train size, sampling, class weights, threshold tuning, and font sizes. The script runs the classification workflow from the original input table, prints the metrics, and saves `roc_curve.svg` and `confusion_matrix.svg`. Random Forest and Gradient Boosting also save `feature_importance.svg`.

Follow the [shared script workflow](#) to run it or change its settings. The script repeats the row-based split and classification decisions used in the app.

21 Common Q & A

Why is a numerical feature missing from a picker?

For a user-provided table, reopen its [column review](#) and check that the column has the **Numerical** role. Look in **Uncategorized Features** if it has no feature group. A numerical picker needs usable numbers: the app tolerates stray text in up to **1%** of non-empty values, replacing those cells with NaN. Above that threshold the column remains text; clean the values and upload it again. See [feature recognition](#).

Why is a categorical column missing?

For a user table, assign **Categorical** in its [profile review](#) and save. For an extraction table, check the active [extraction configuration](#). A filter or visual picker may omit a column with only one remaining value. Other choices can also narrow the options: for example, a column used for **Separate by** cannot also define the color groups, except in Dimension Reduction. See [shared controls](#).

Do I need to add an ID column before uploading my table?

No. Leave **Row ID** unassigned to use generated row numbers. An assigned ID must be nonmissing and unique. Generated numbers follow input row order and are regenerated for each upload; provide your own ID if it must remain stable when rows are reordered. See [identifiers](#).

Why did a different profile open, or why am I asked to review columns?

The uploaded file selects its analysis profile by exact column headers, including ignored columns. A single valid exact match can apply automatically. Otherwise, choose a candidate or start a new profile and review the columns. Header order does not matter; case, spaces, and punctuation do. See [profile matching](#).

How do I average observations by a category, such as dish or patient?

Use [Collapse by](#) with that categorical column. The app calculates one mean per category within each displayed group. Statistics then use those category means. The [SuperPlot overlay](#) can show original observations behind them without changing the analysis unit.

Can I reuse a calibration and choose Local fitting?

Yes. Save the [updated metadata after calibration](#), then upload it on a later run. Confirmed or saved shifts enable the choice between **Hybrid** and **Local** for cell-level fitting. Shift estimation itself uses Hybrid.

Where are exported files saved?

Browser download buttons use your browser's download location. Extraction workflows can also save directly into the selected local folder; the page reports the path. For example, categorical merging writes a timestamped combined CSV to its input folder. Run extraction on a [local installation](#) so these folders are available. The [analysis Python export](#) reproduces the current view and saves SVG figures when run.

How do I use a light or dark theme?

The app follows its Streamlit theme setting, which can follow the system or select Light or Dark. Use the app's **Settings** → **Theme** when available. The documentation examples use Light.

1. Datta, R., Heaster, T. M., Sharick, J. T., Gillette, A. A. & Skala, M. C. [Fluorescence lifetime imaging microscopy: fundamentals and advances in instrumentation, analysis, and applications](#). *Journal of Biomedical Optics* **25**, 071203 (2020).
2. Stringer, C., Wang, T., Michaelos, M. & Pachitariu, M. Cellpose: A generalist algorithm for cellular segmentation. *Nature methods* **18**, 100–106 (2021).
3. Gohlke, C., Pannunzio, B., Schüty, B. & Blanco, R. Phasorpy/phasorpy: v0.7. Zenodo <https://doi.org/10.5281/zenodo.16923774> (2025).
4. Gottlieb, D., Asadipour, B., Kostina, P., Ung, T. P. L. & Stringari, C. [FLUTE: A python GUI for interactive phasor analysis of FLIM data](#). *Biological Imaging* **3**, e21 (2023).
5. Kapsiani, S. *et al.* FLIMPA: A versatile software for fluorescence lifetime imaging microscopy phasor analysis. *Analytical Chemistry* (2025).
6. Becker, W. *The Bh TCSPC Handbook*. (Becker & Hickl GmbH, 2021).
7. Stirling, D. R. *et al.* [CellProfiler 4: Improvements in speed, utility and usability](#). *BMC Bioinformatics* **22**, 433 (2021).
8. Stirling, D. R., Carpenter, A. E. & Cimini, B. A. [CellProfiler analyst 3.0: Accessible data exploration and machine learning for image analysis](#). *Bioinformatics* **37**, 3992–3994 (2021).
9. Samimi, K. *et al.* Segmentation-guided photon pooling enables robust single cell analysis and fast fluorescence lifetime imaging microscopy. *bioRxiv* <https://doi.org/10.1101/2025.09.30.679660> (2025) doi:[10.1101/2025.09.30.679660](https://doi.org/10.1101/2025.09.30.679660).
10. Samimi, K. *et al.* [Autofluorescence lifetime flow cytometry with time-correlated single photon counting](#). *Cytometry Part A* **105**, 607–620 (2024).
11. Warren, S. C. *et al.* [Rapid global fitting of large fluorescence lifetime imaging microscopy datasets](#). *PLoS ONE* **8**, e70687 (2013).
12. Cleveland, W. S. & McGill, R. [Graphical perception: Theory, experimentation, and application to the development of graphical methods](#). *Journal of the American Statistical Association* **79**, 531–554 (1984).
13. Shannon, C. E. [A mathematical theory of communication](#). *The Bell System Technical Journal* **27**, 379–423 (1948).
14. Pham, D. L. *et al.* [Autofluorescence imaging reveals the impact of cryopreservation on t cell metabolism and activation response](#). *Molecular Therapy Advances* **34**, 201704 (2026).